

Effective Software Testing



Effective Software Testing



Topics

Software Development Life Cycle

Software quality problem

Test-first vs Test-last vs Test later

Testing strategies

Good testing

Test design

Test techniques

Working with AI



What problems in your team ?

Think about testing issues !!



Other's problems

Team silo

Poor communication

Testing is a phase (never enough time !!)

Requirements change but ...

Tester not involved early

Don't understand the requirements

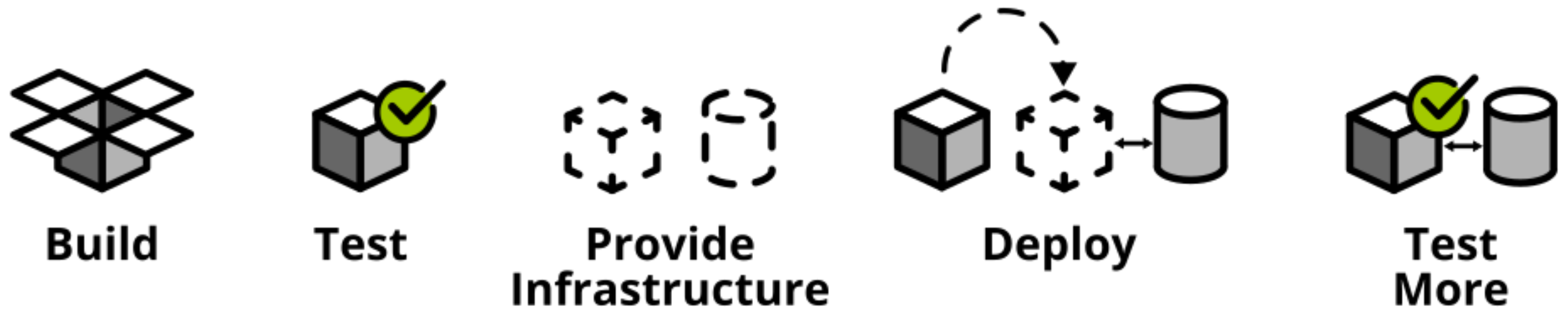
Developers don't do their test !!



Quality vs. Quantity



Software Delivery



<https://martinfowler.com/articles/practical-test-pyramid.html>



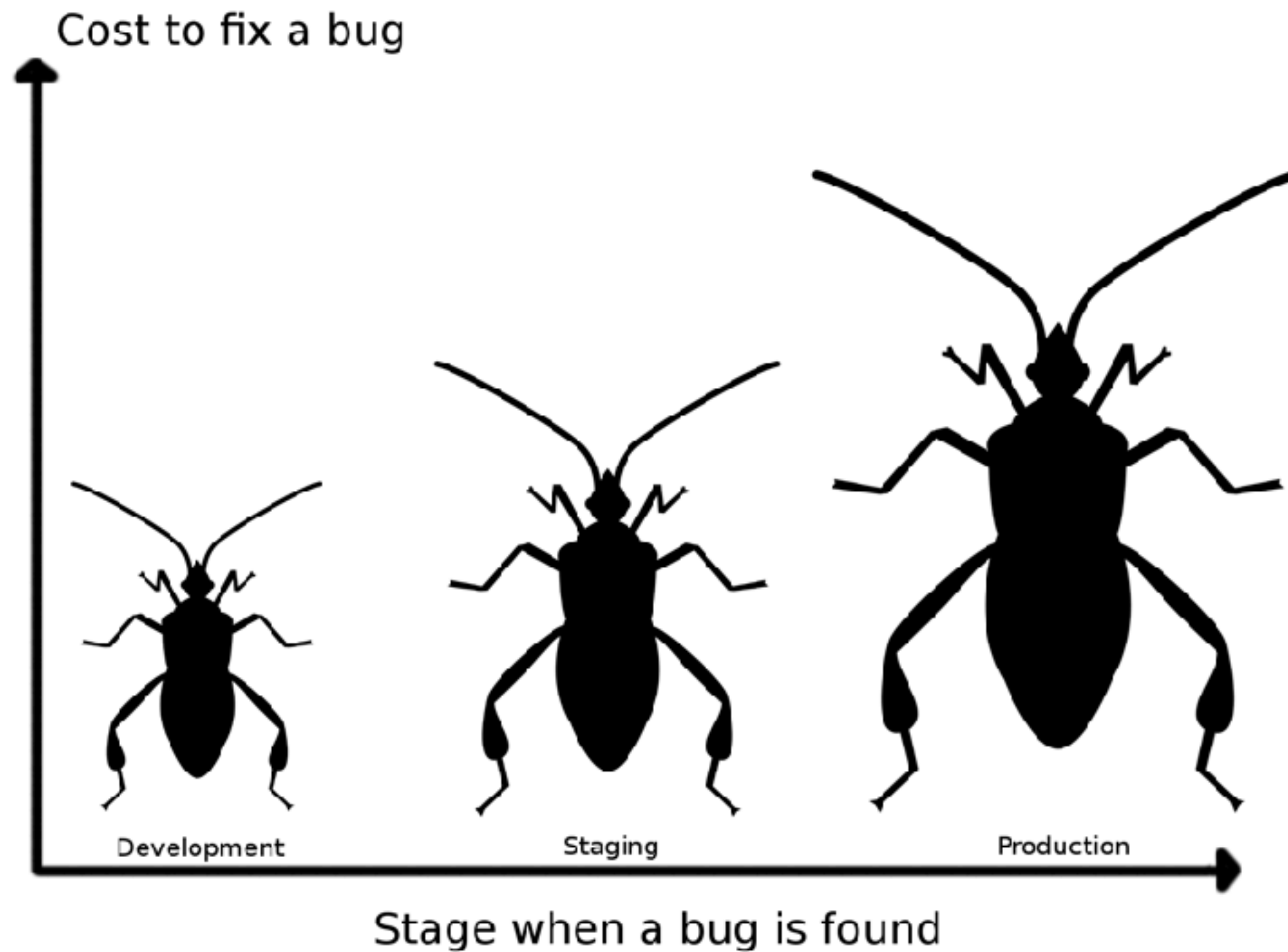
Start with Why ...



"Program testing can be used to show the presence of bugs, but never their absence."

Edsger W. Dijkstra, 1970, Notes on Structured Programming







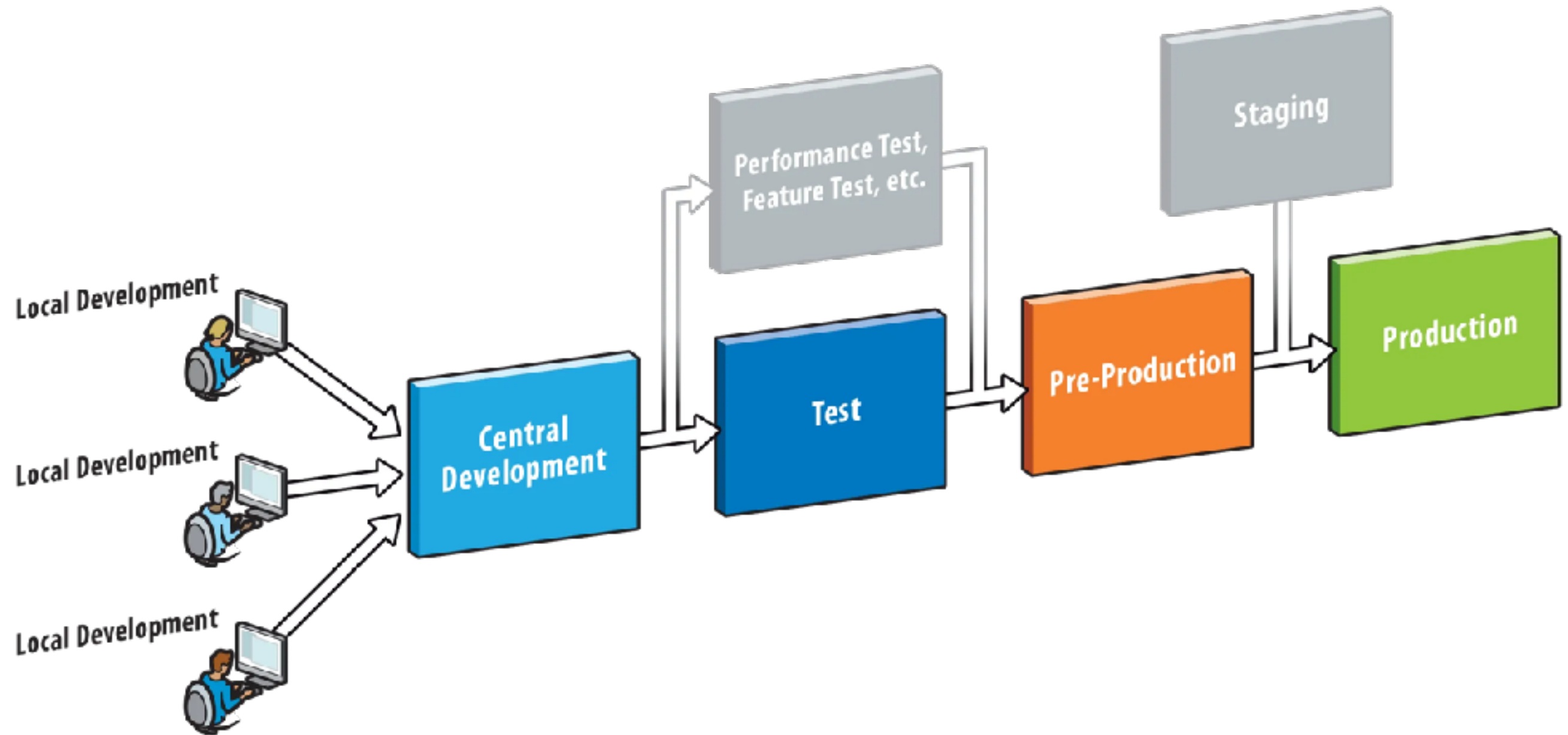
www.cartoonistshilpa.com



Every time a developer changing the code !!



Environments to deliver software



Why we need to test ?

Help you to **catch bugs**

Boosted confidence

Quality code

Enforce **modularity** of your project

Develop features **faster**

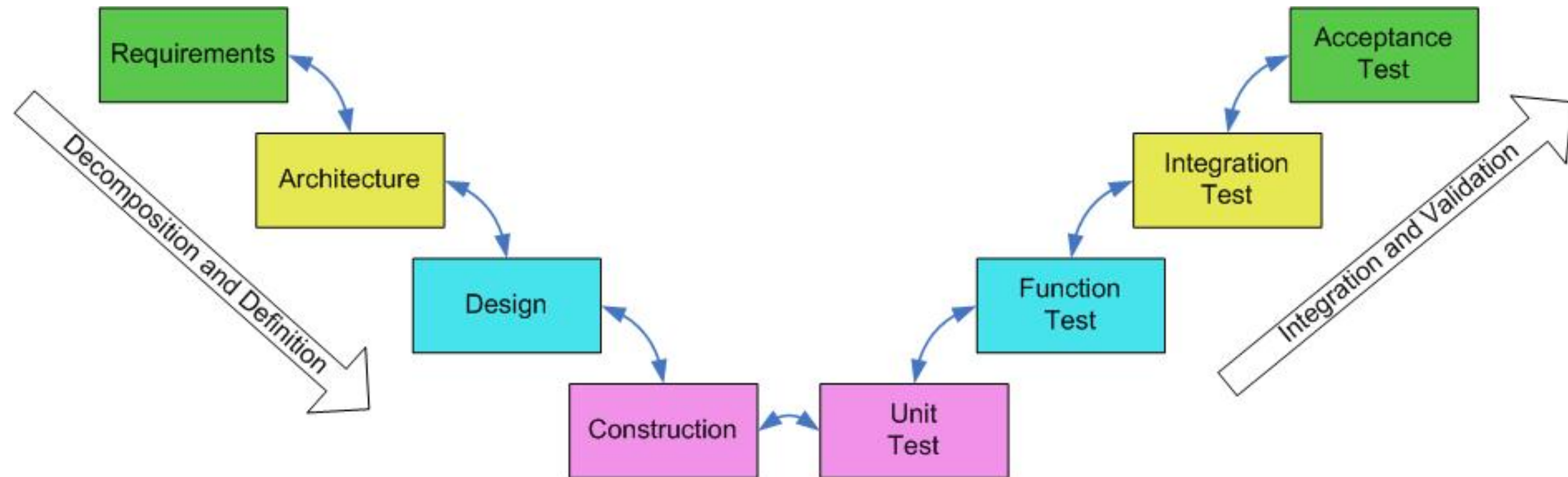
Better documentation



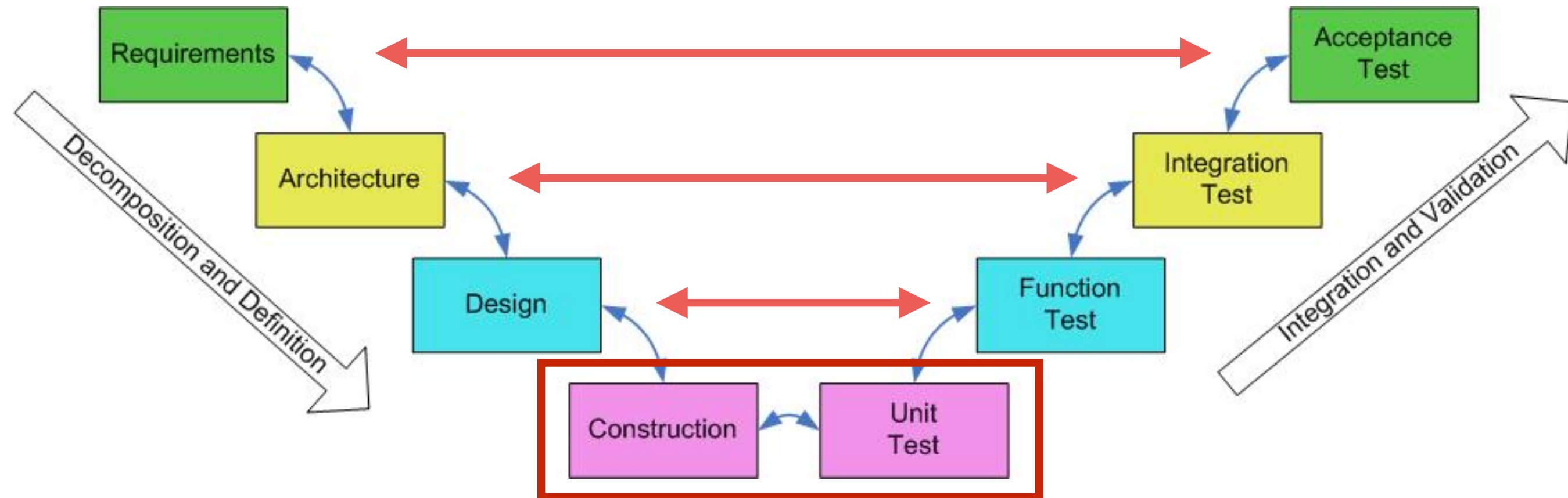
What kind of test should we write ?



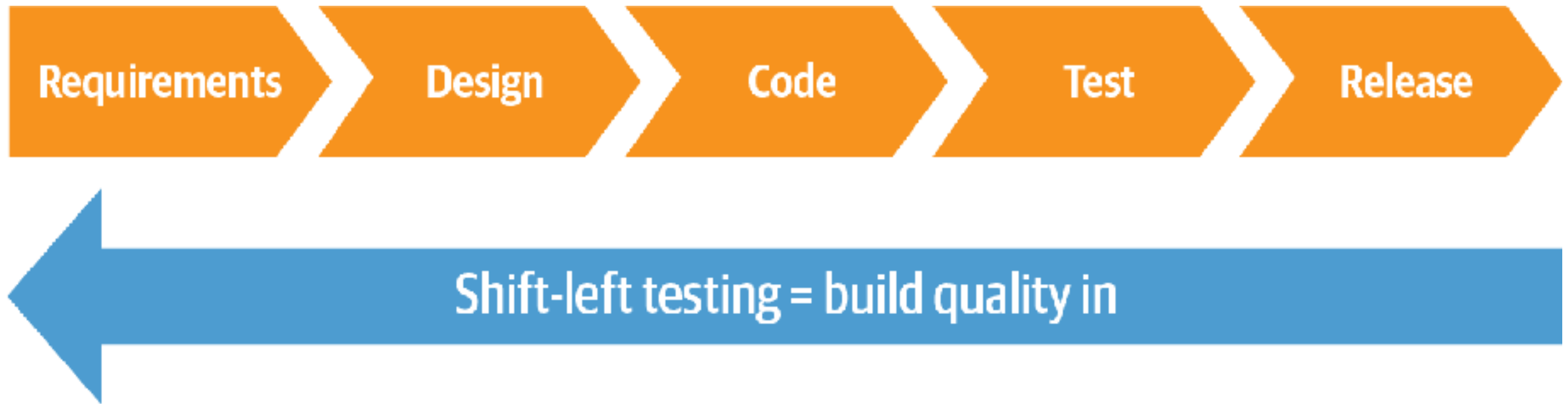
V Model



V Model



Shift Left Testing

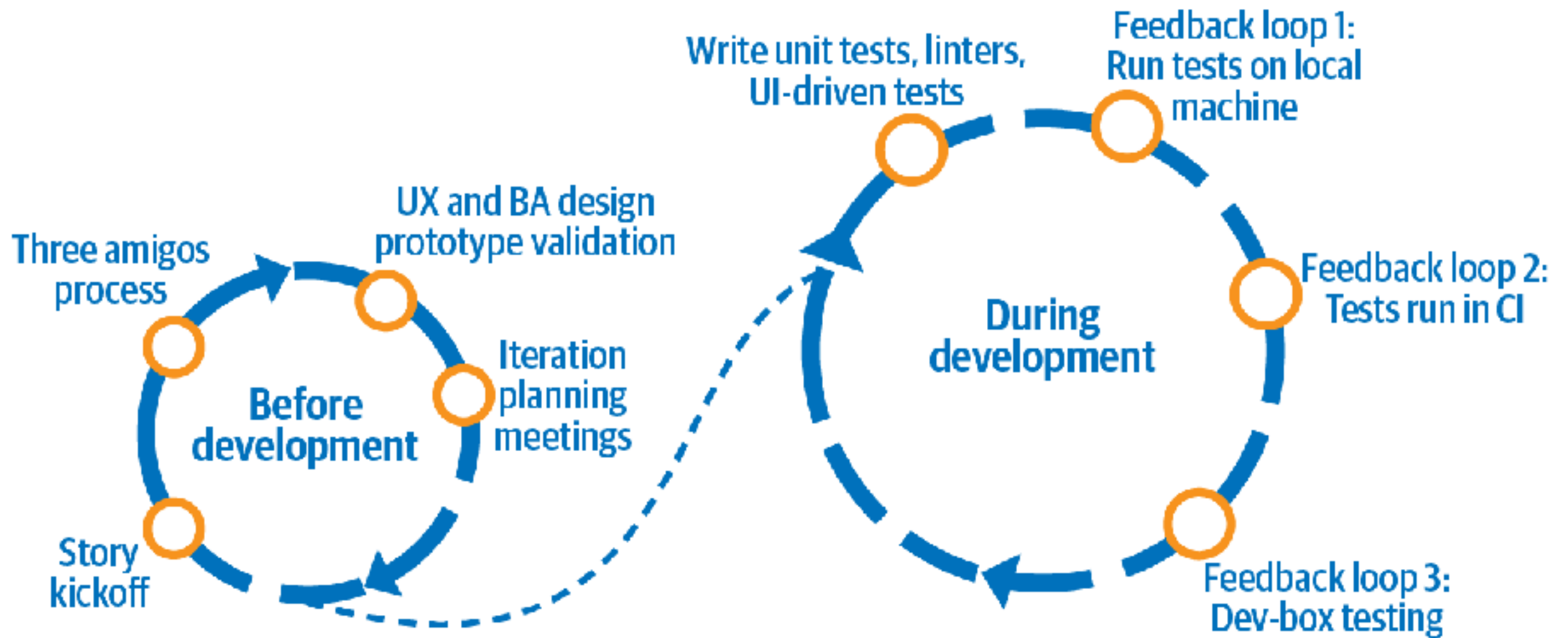


Functional
testing

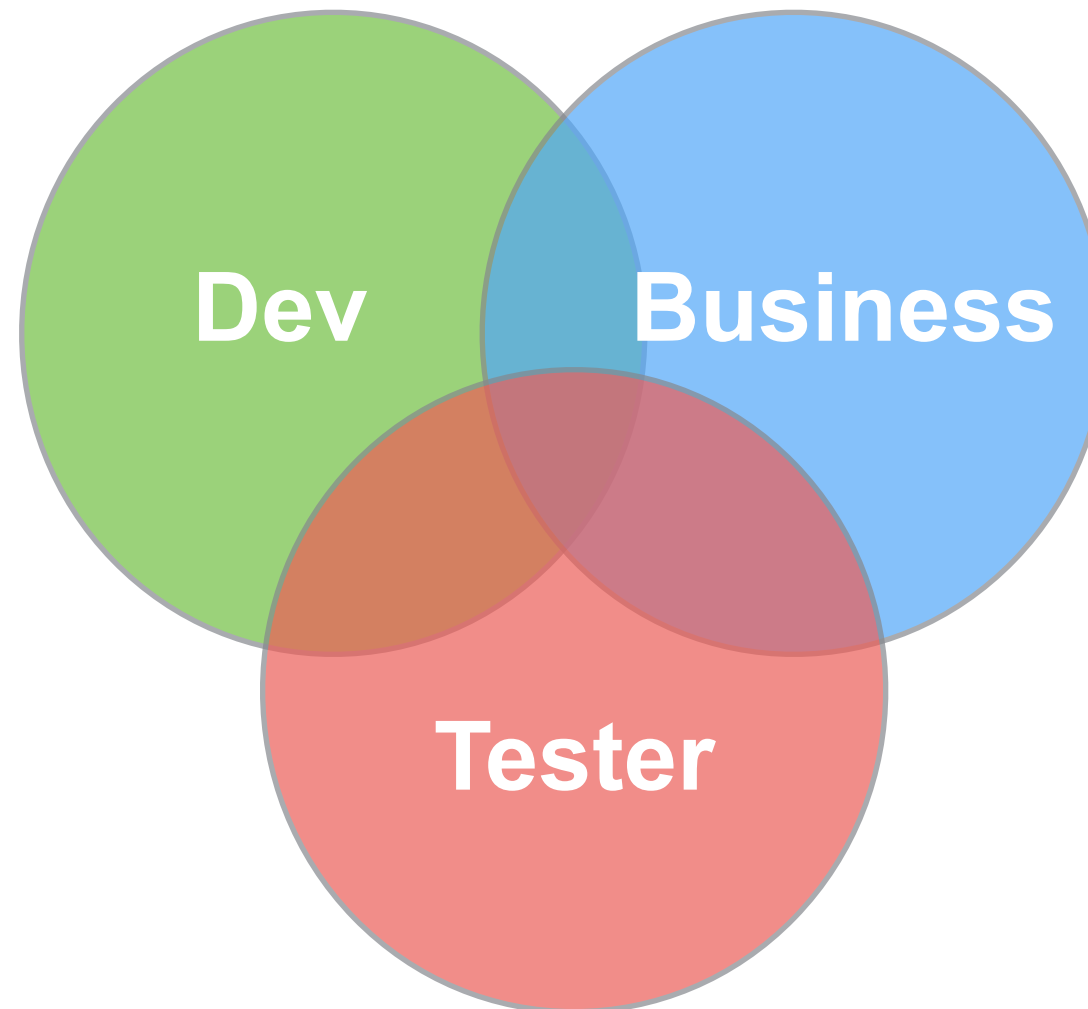
Non-functional
testing



Quality Check ?



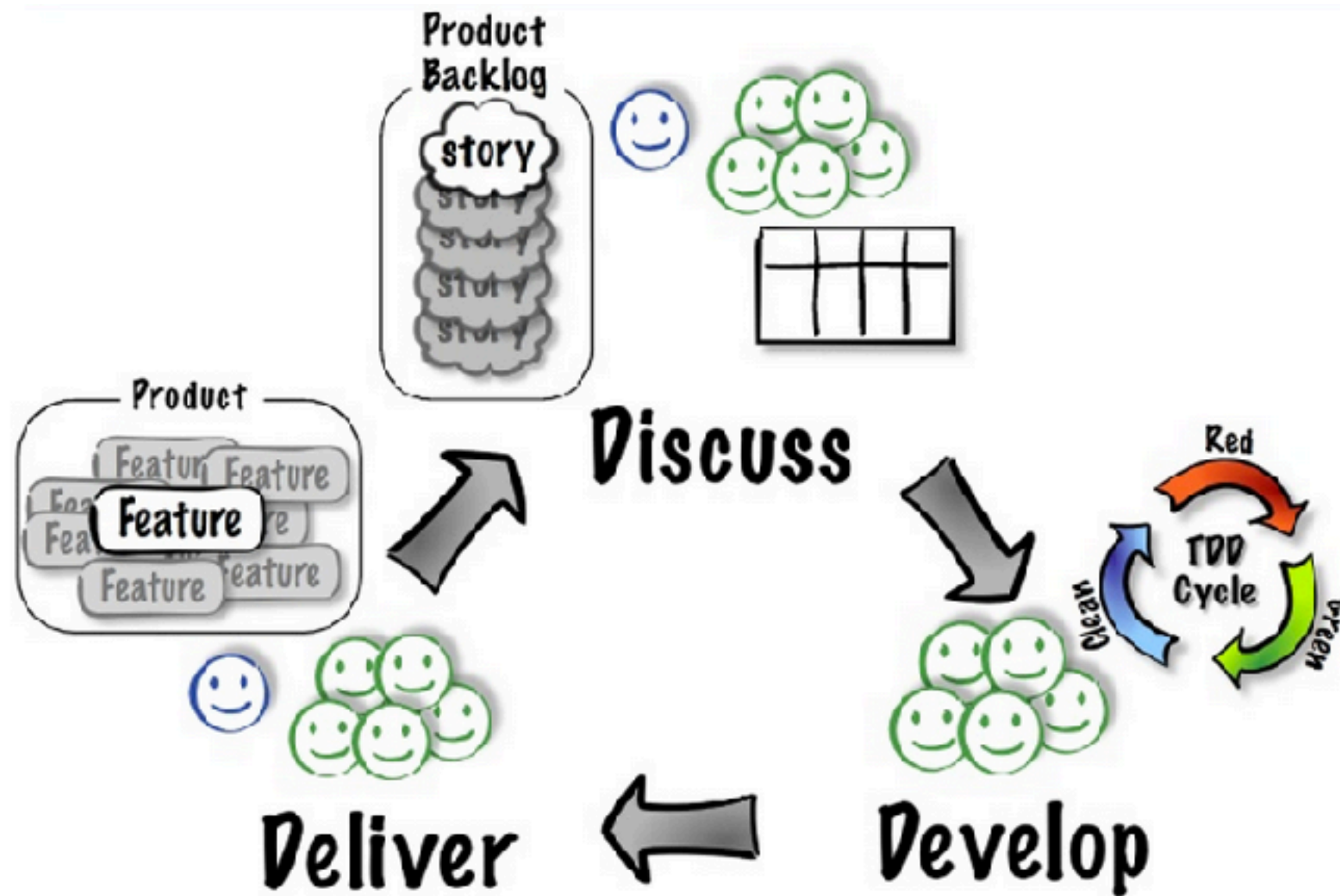
Power of Three



THINK before coding



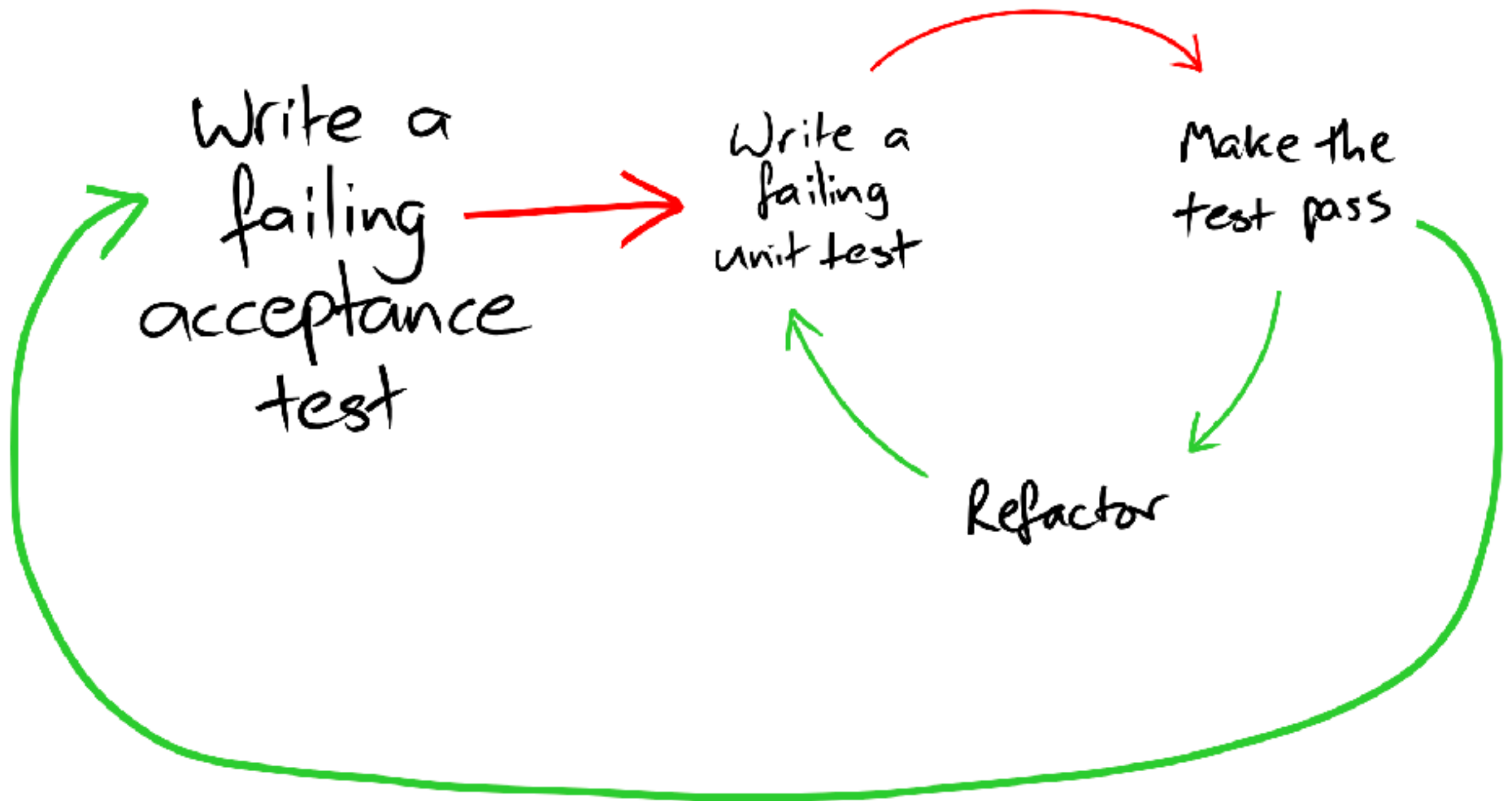
Acceptance Test-Driven Development



(Model developed with Pekka Klärck, Bas Vodde, and Craig Larman.)



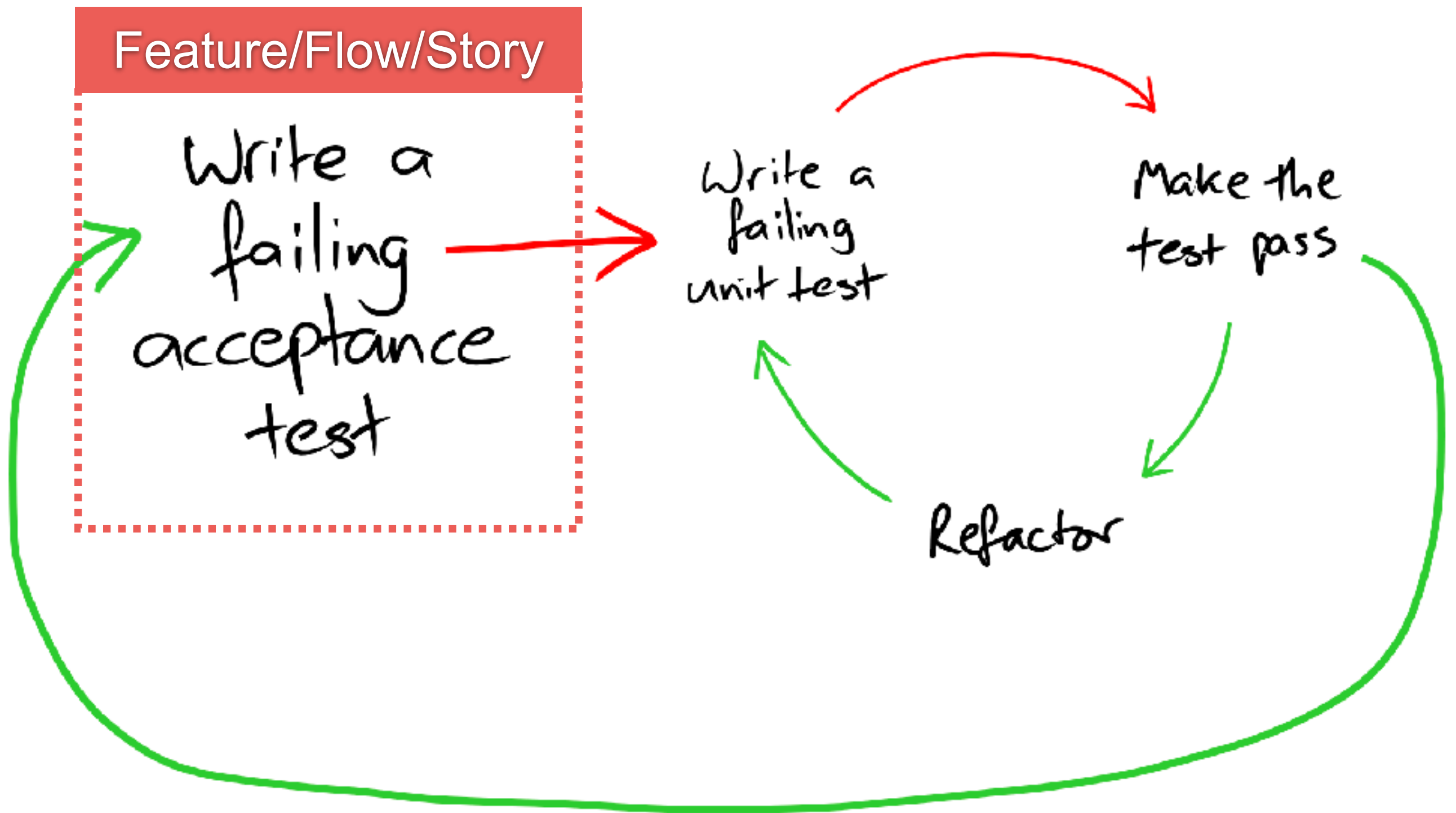
Outside-in develop/test



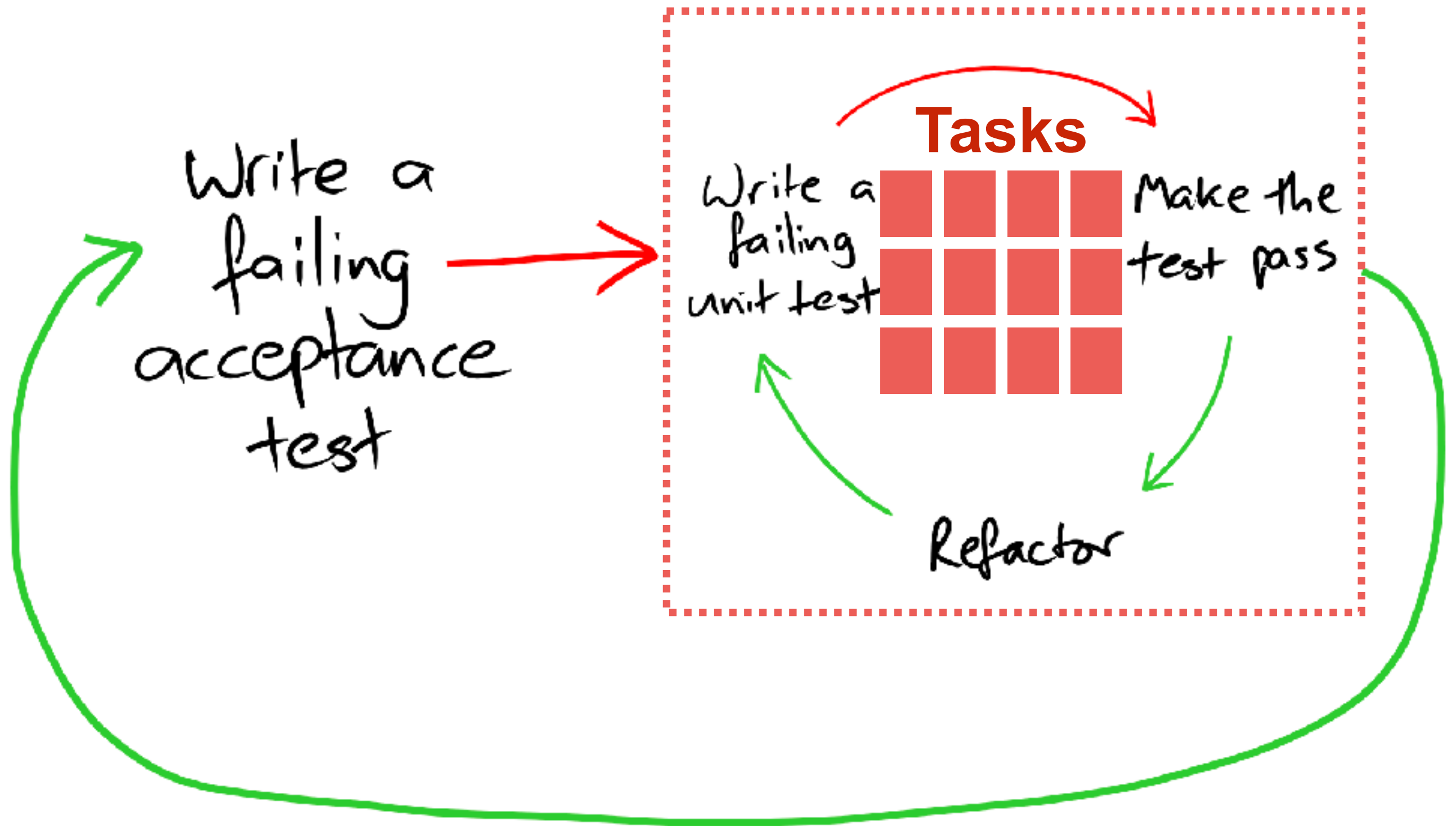
<http://www.growing-object-oriented-software.com/>



Outside-in develop/test



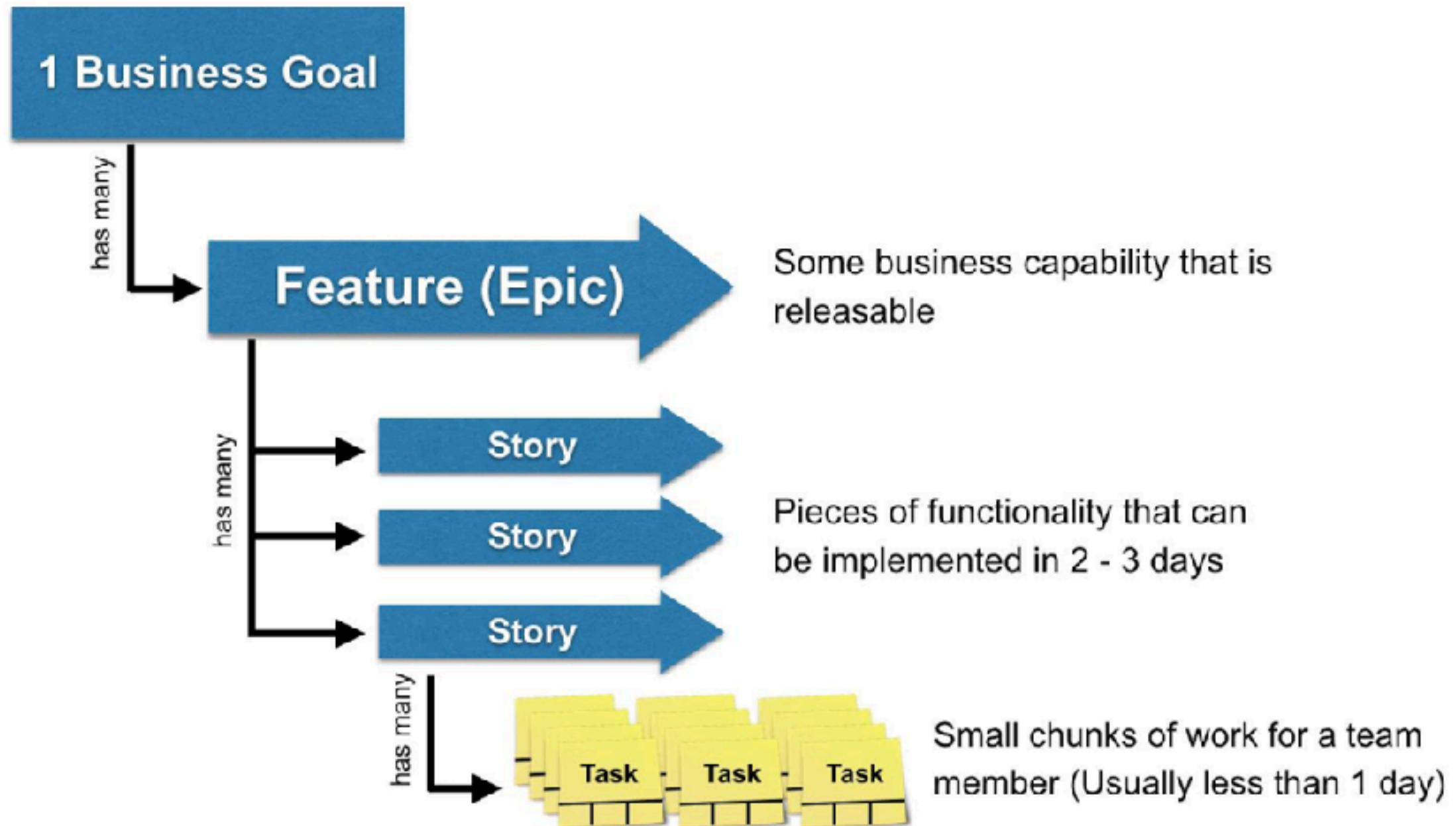
Outside-in develop/test



Tests Cases
=
Business Criteria
+
Examples (real data)



Work break down



Key success factors

Whole team solve problems

Whole team thinks about **testing**

Whole team **committed to quality**

Everyone **collaborates**

Improve technical **skills**

Better testing process (improvement)



Whole team approach

Functional

Common functional expertise



System analysts



Developers



Testers

Cross - Functional

Representatives from the various functions



Development Team



Iterative and Incremental process



Iterative and incremental process

Feature 1 is Done ?

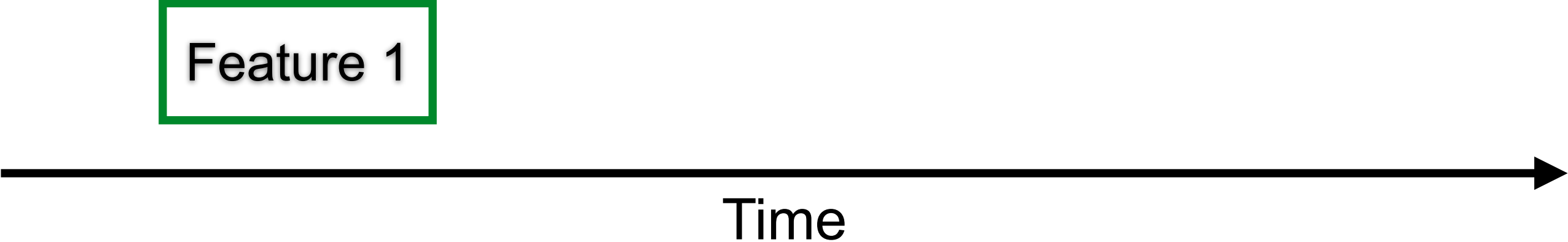
Feature 1

Time



Iterative and incremental process

Done = coded and tested



The diagram illustrates an iterative and incremental process. A horizontal black arrow points to the right, representing the progression of time. Above the arrow, on the left side, is a green rectangular box containing the text "Feature 1". This indicates that Feature 1 has been completed (coded and tested) at an early stage in the process.

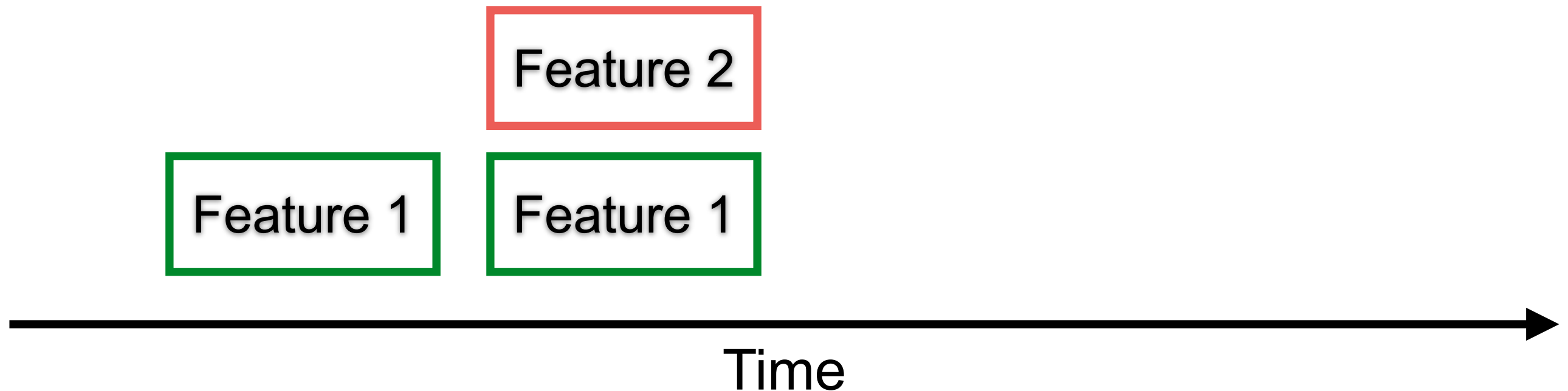
Feature 1

Time



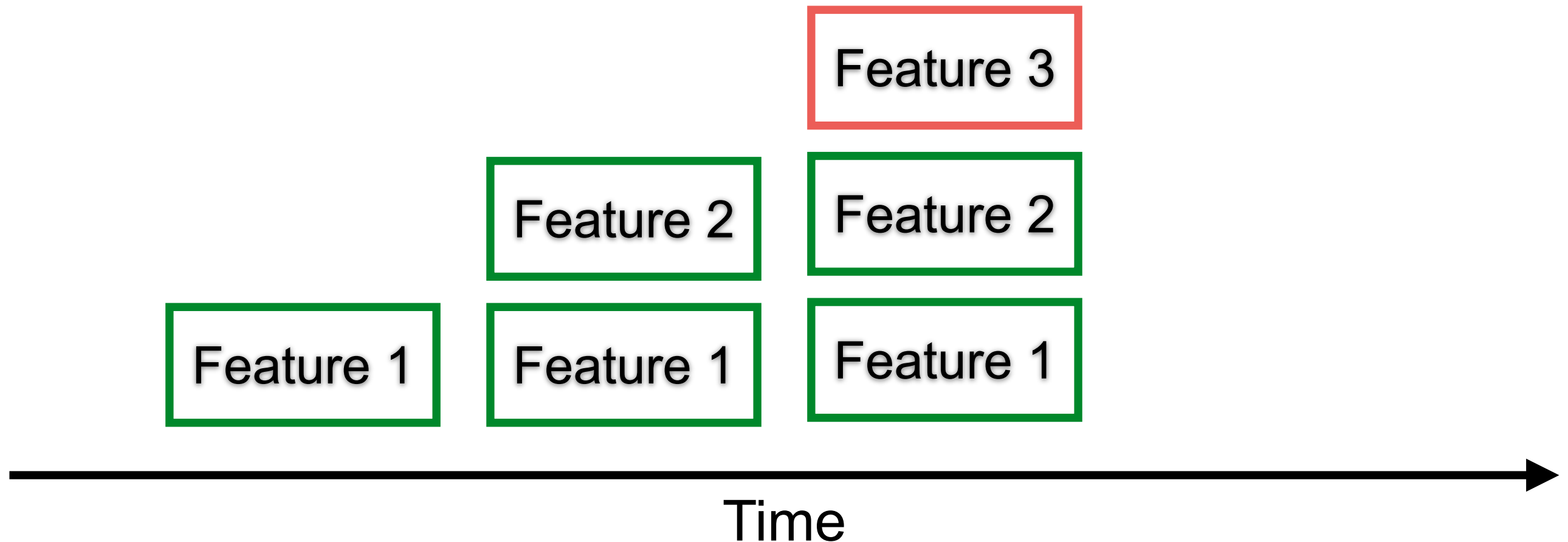
Iterative and incremental process

Done = coded and tested



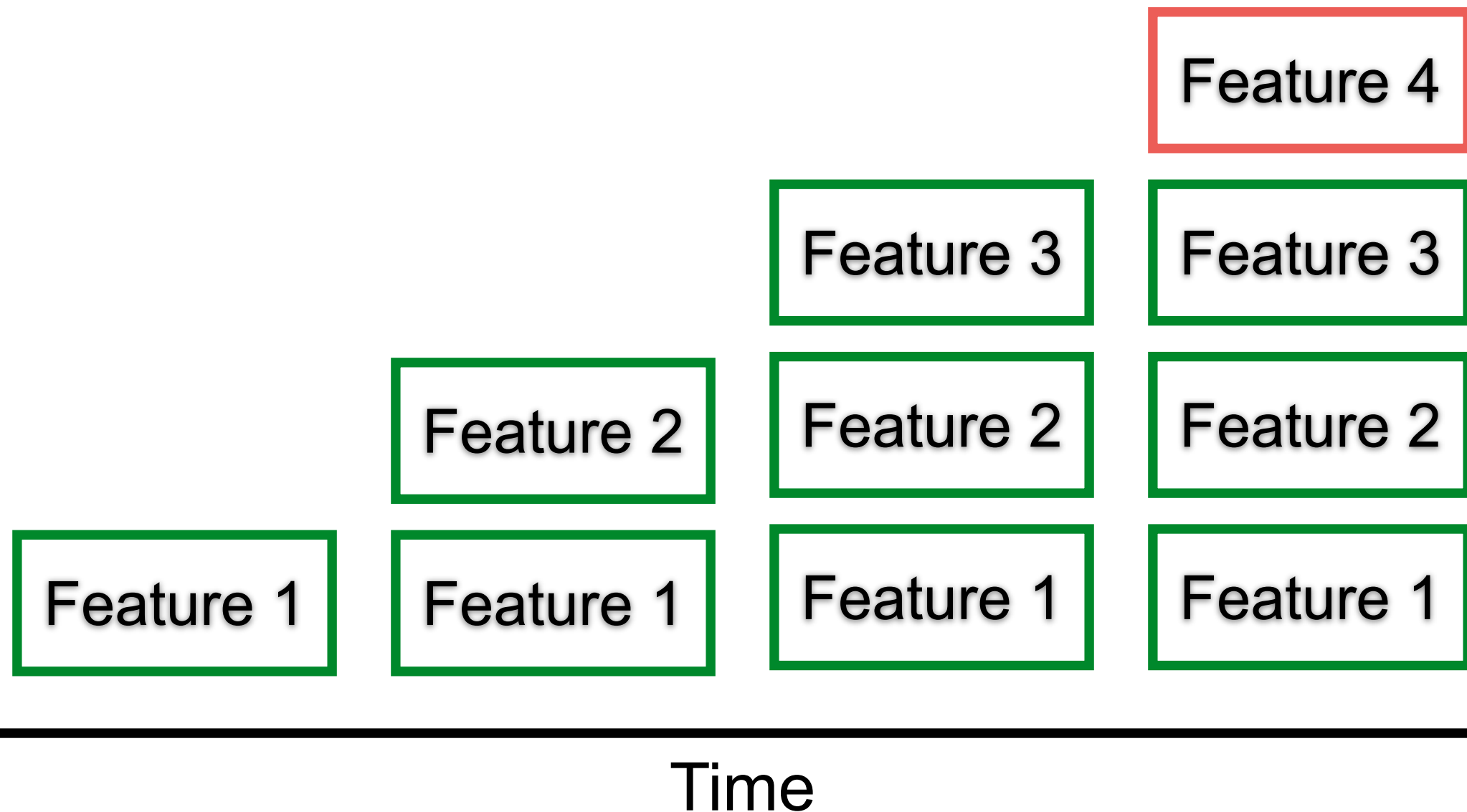
Iterative and incremental process

Done = coded and tested



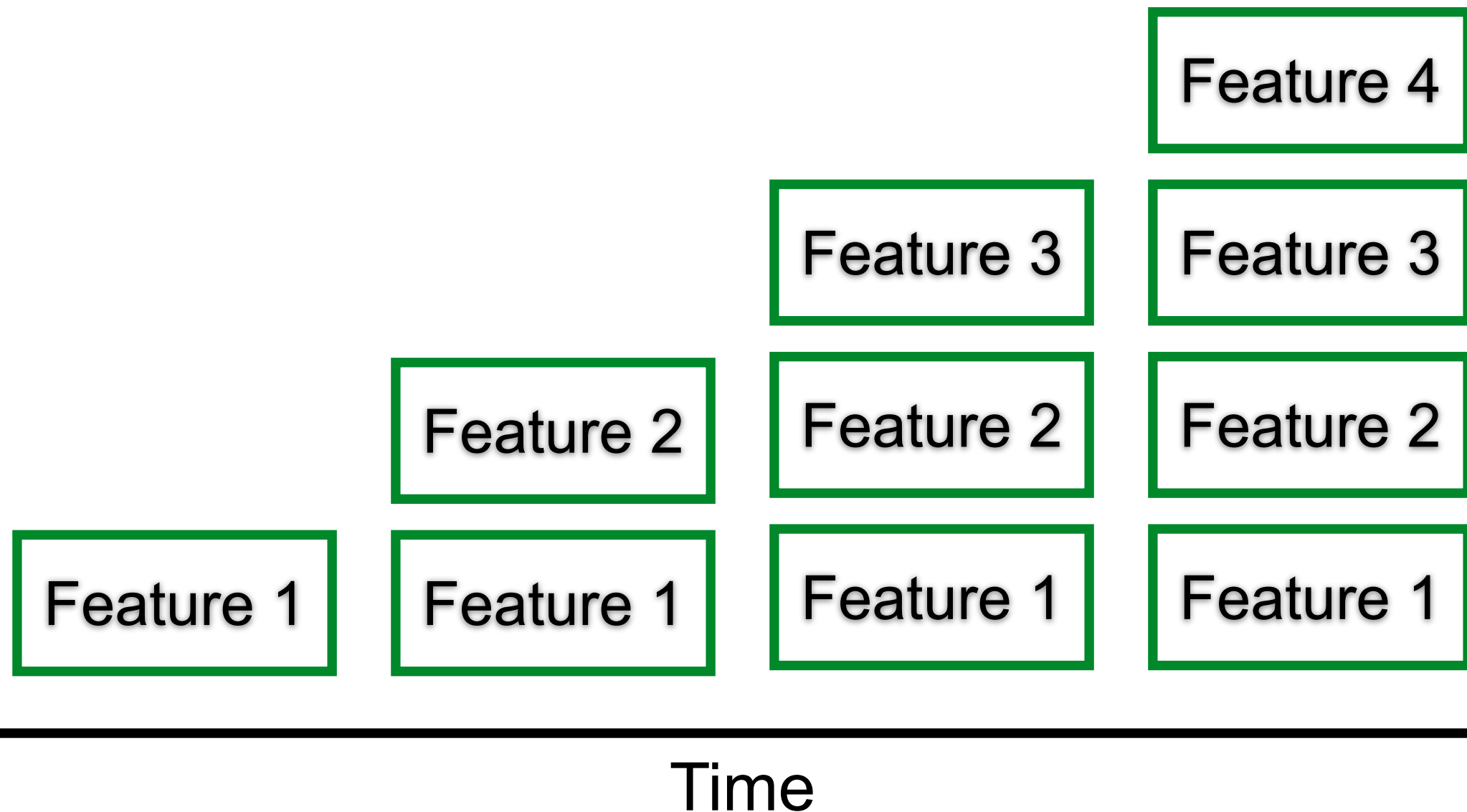
Iterative and incremental process

Done = coded and tested

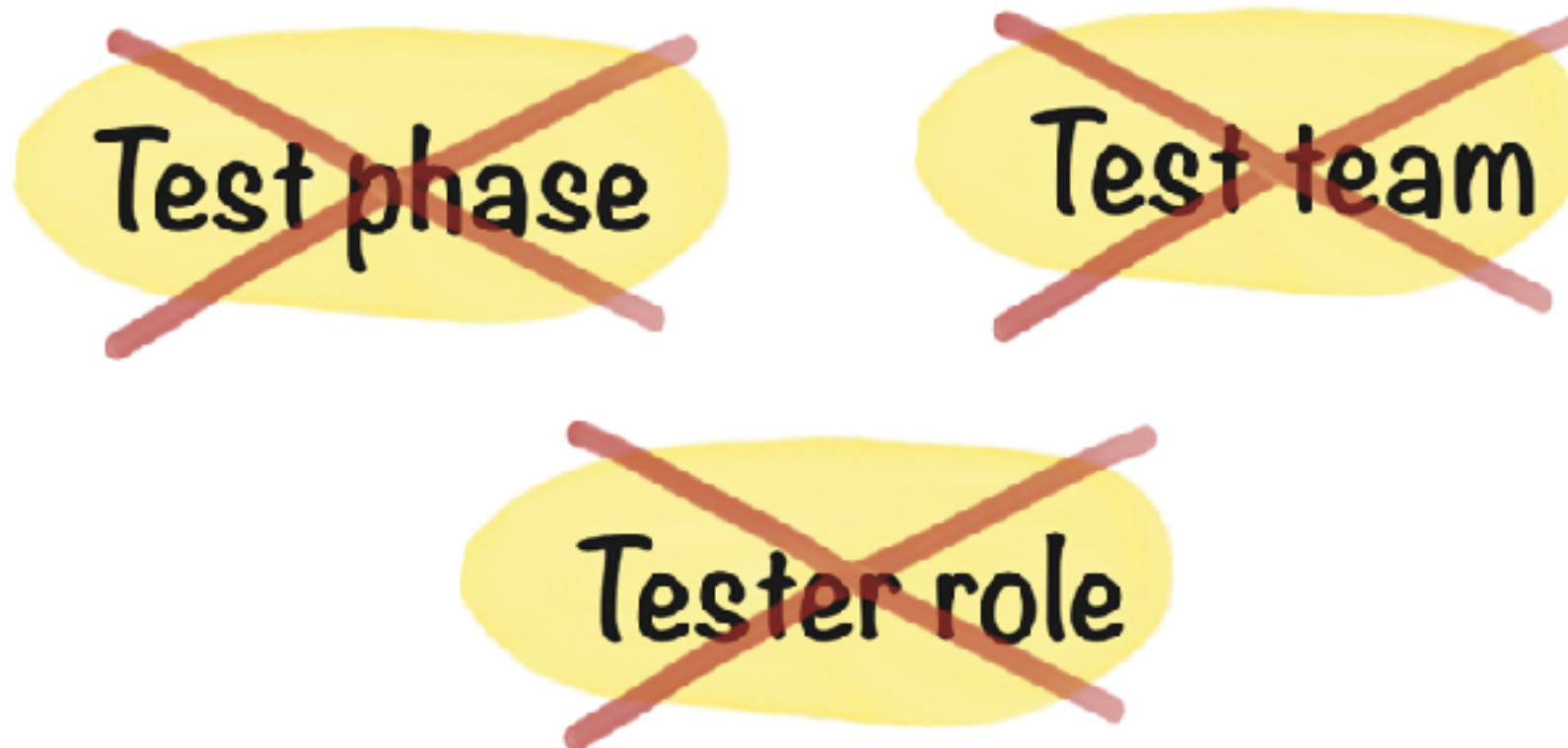


Iterative and incremental process

Done = coded and tested

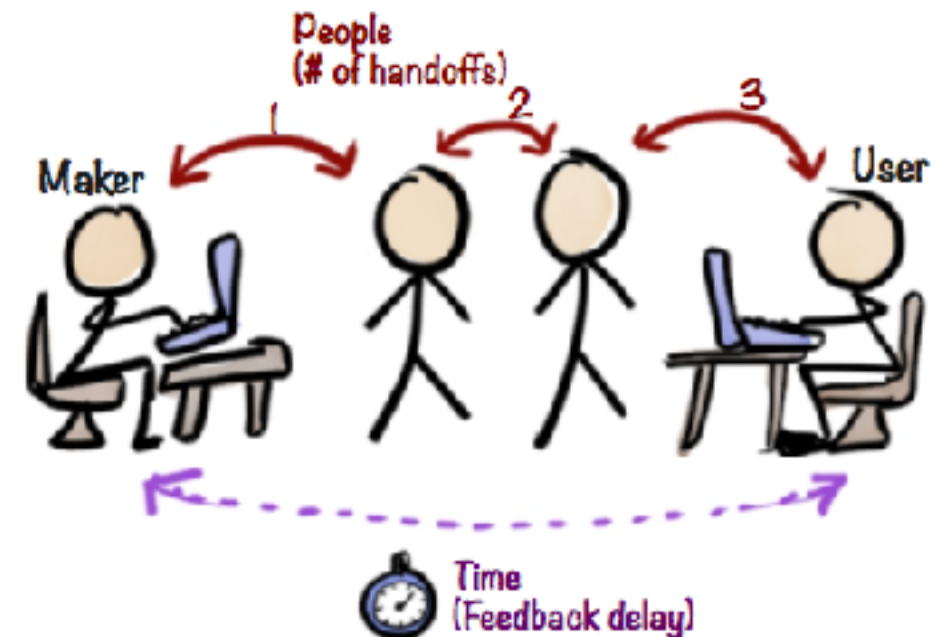
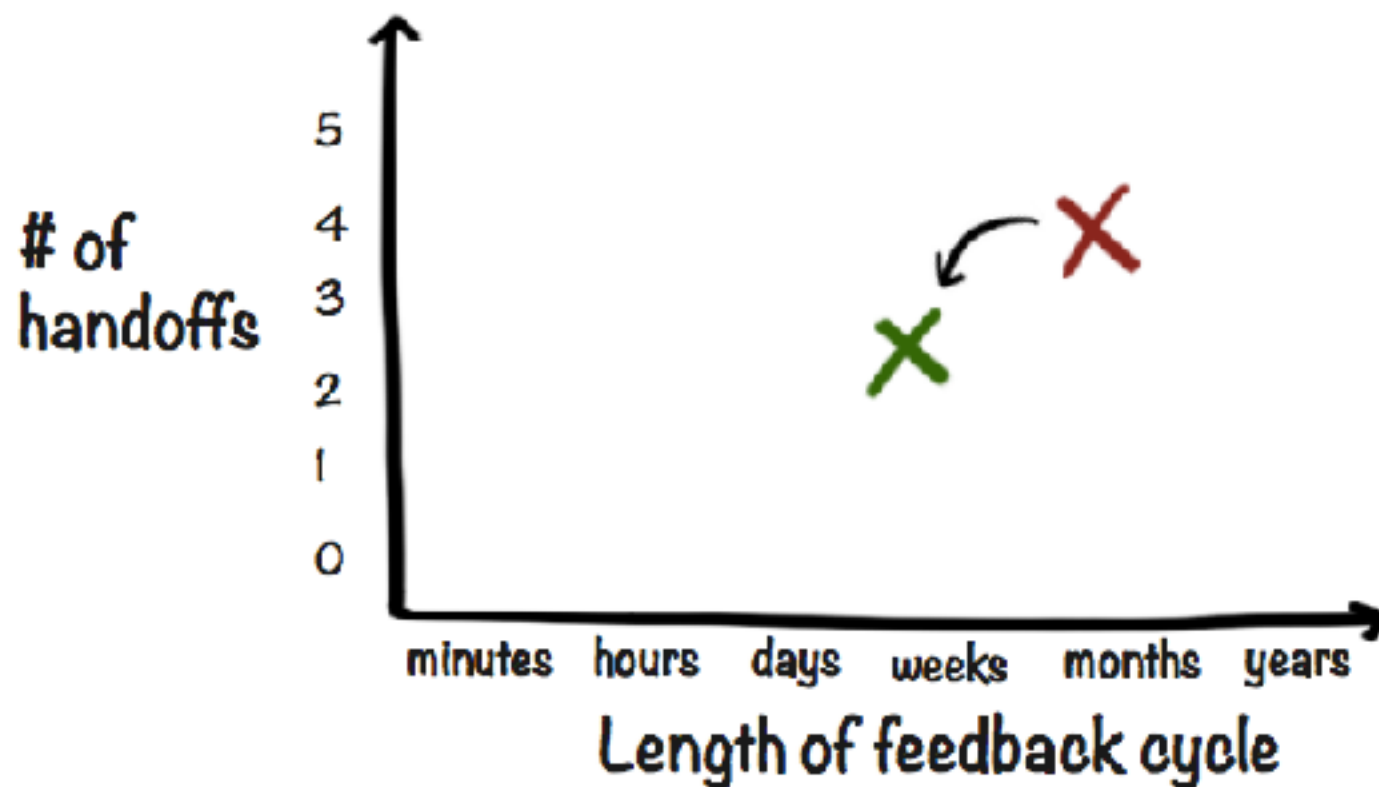


Testing is activity

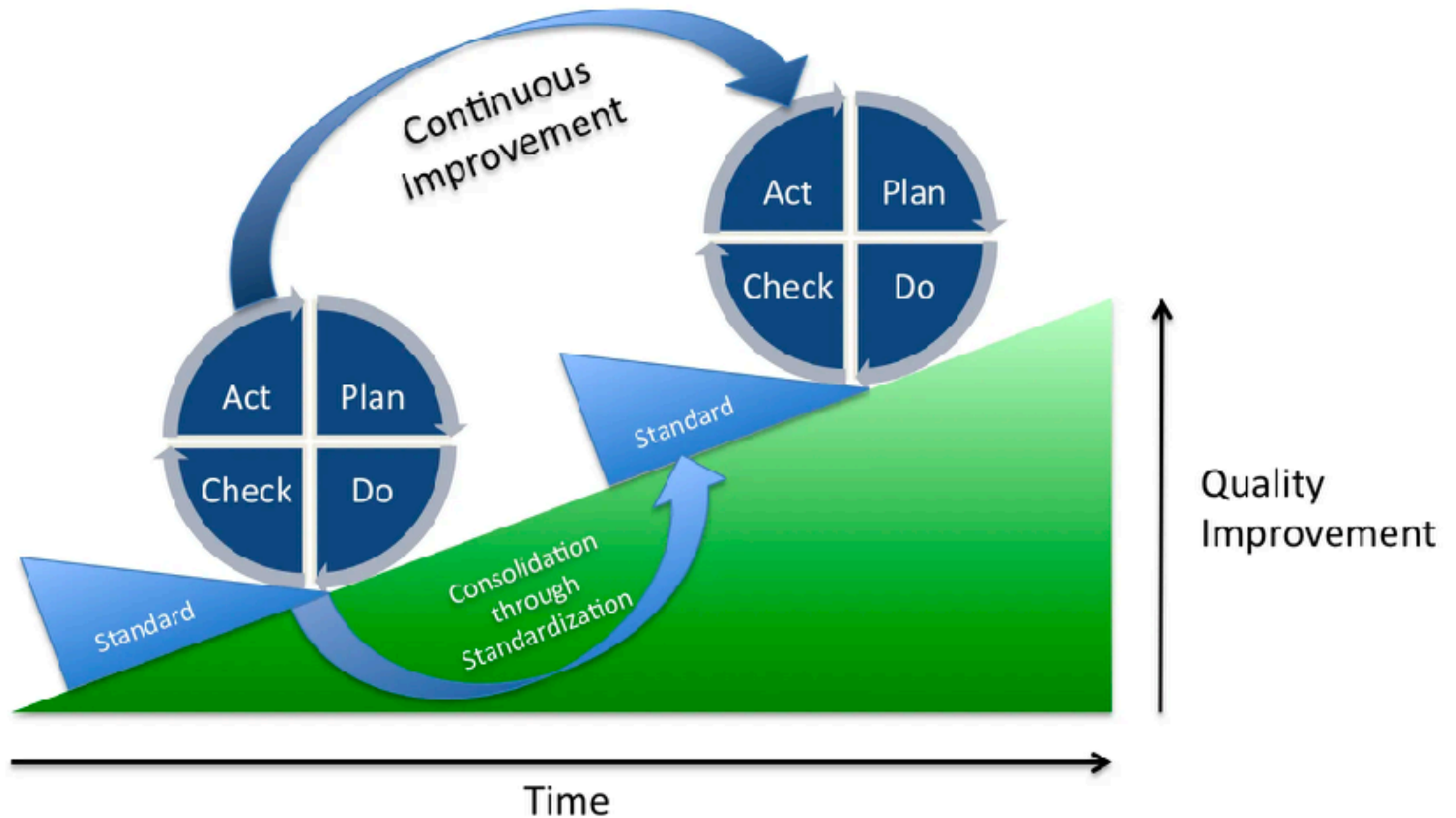


Fast feedback loop

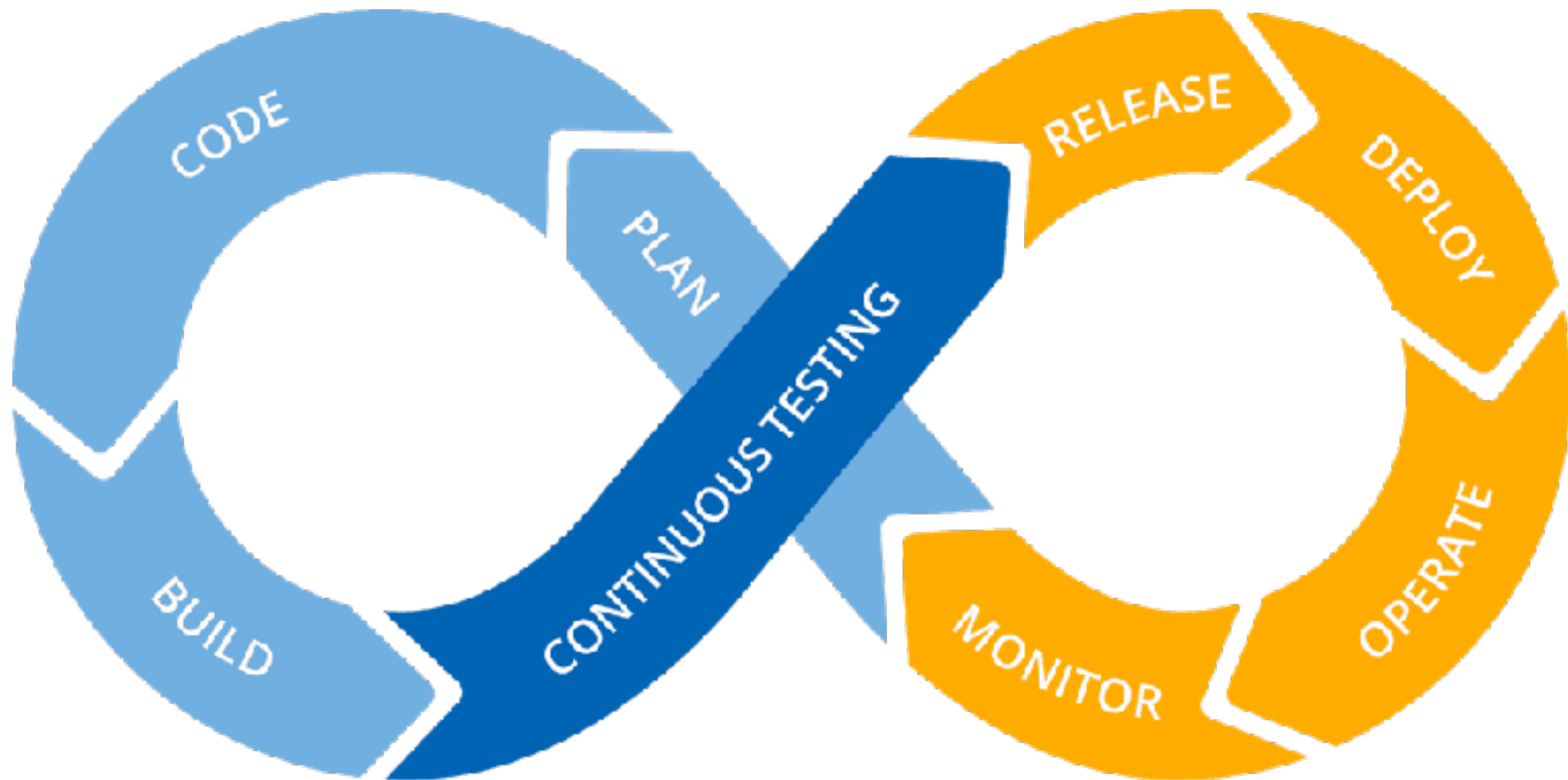
Shorten the feedback loop



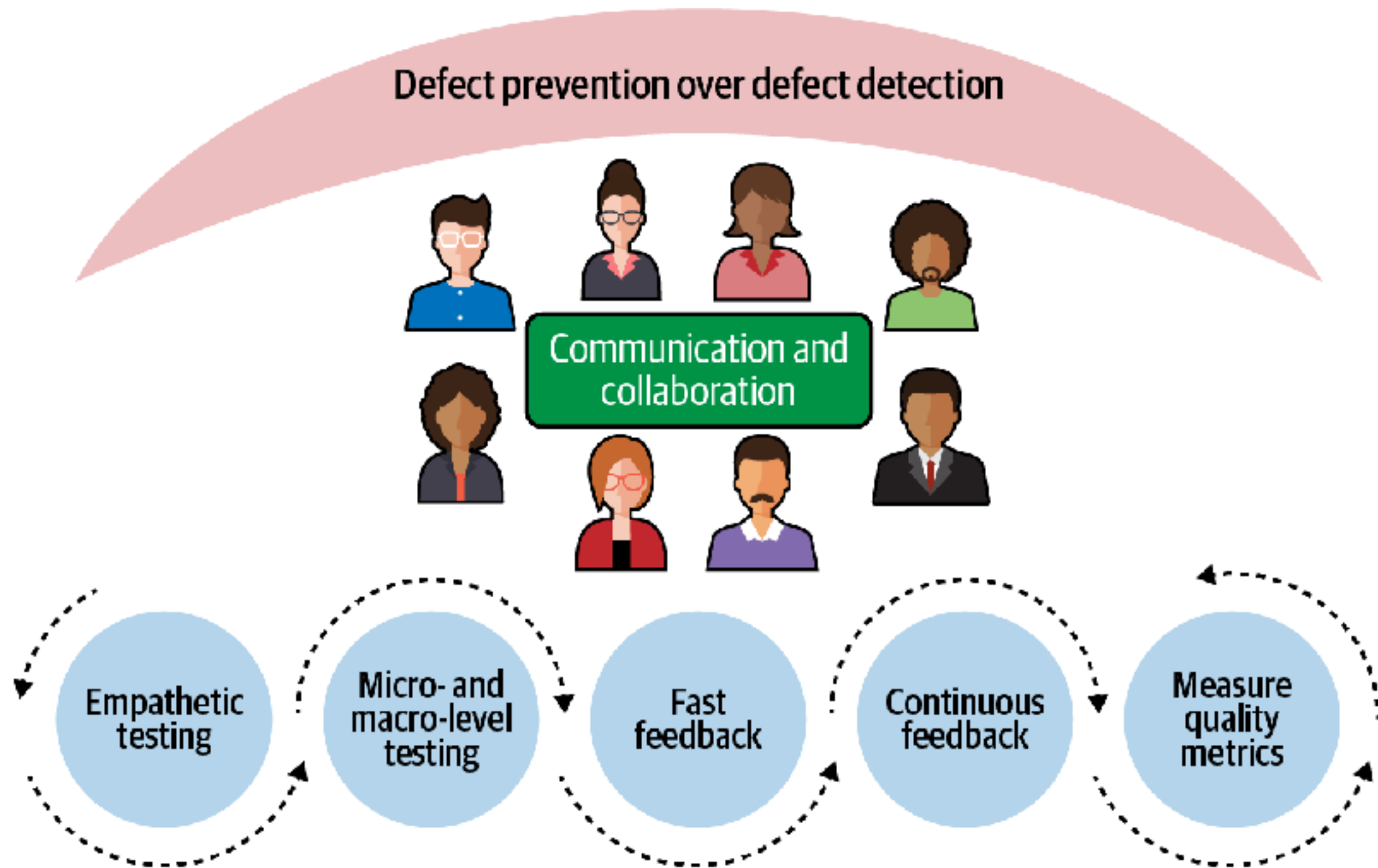
Continuous improvement



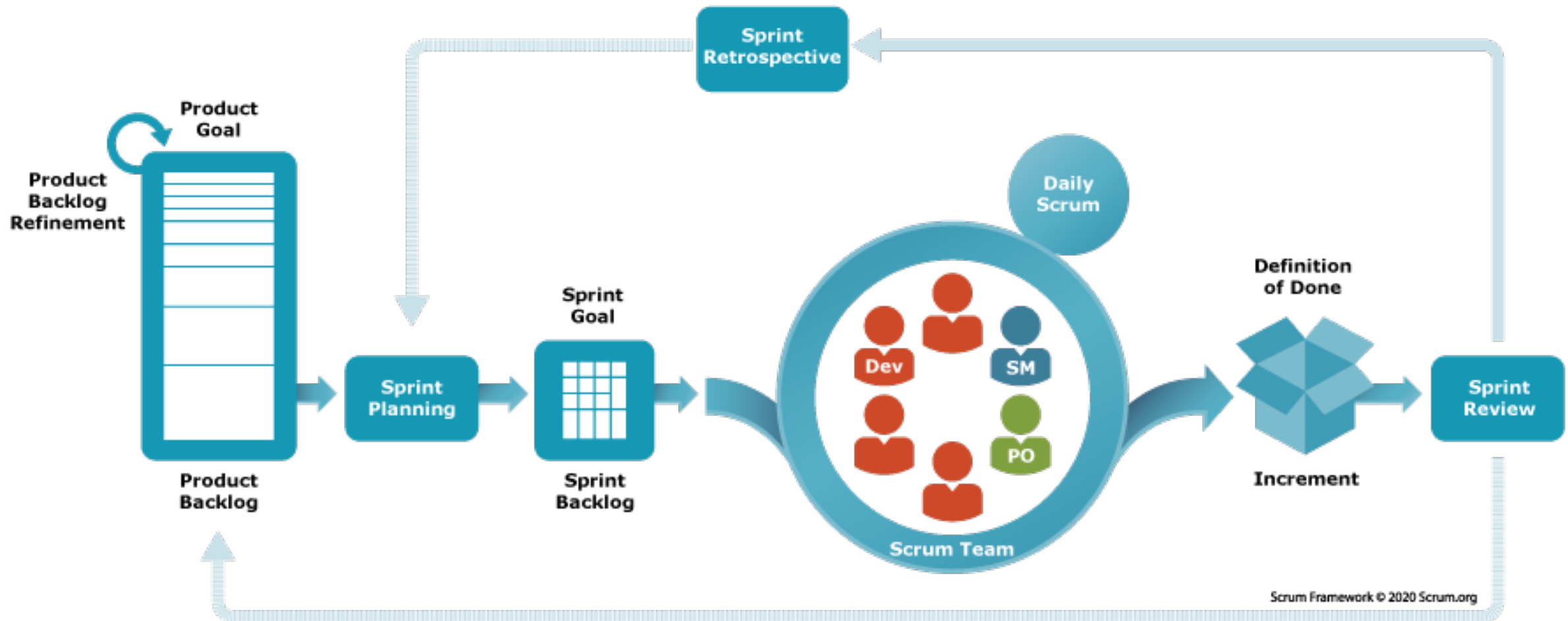
Continuous testing and feedback



Principles of Testing



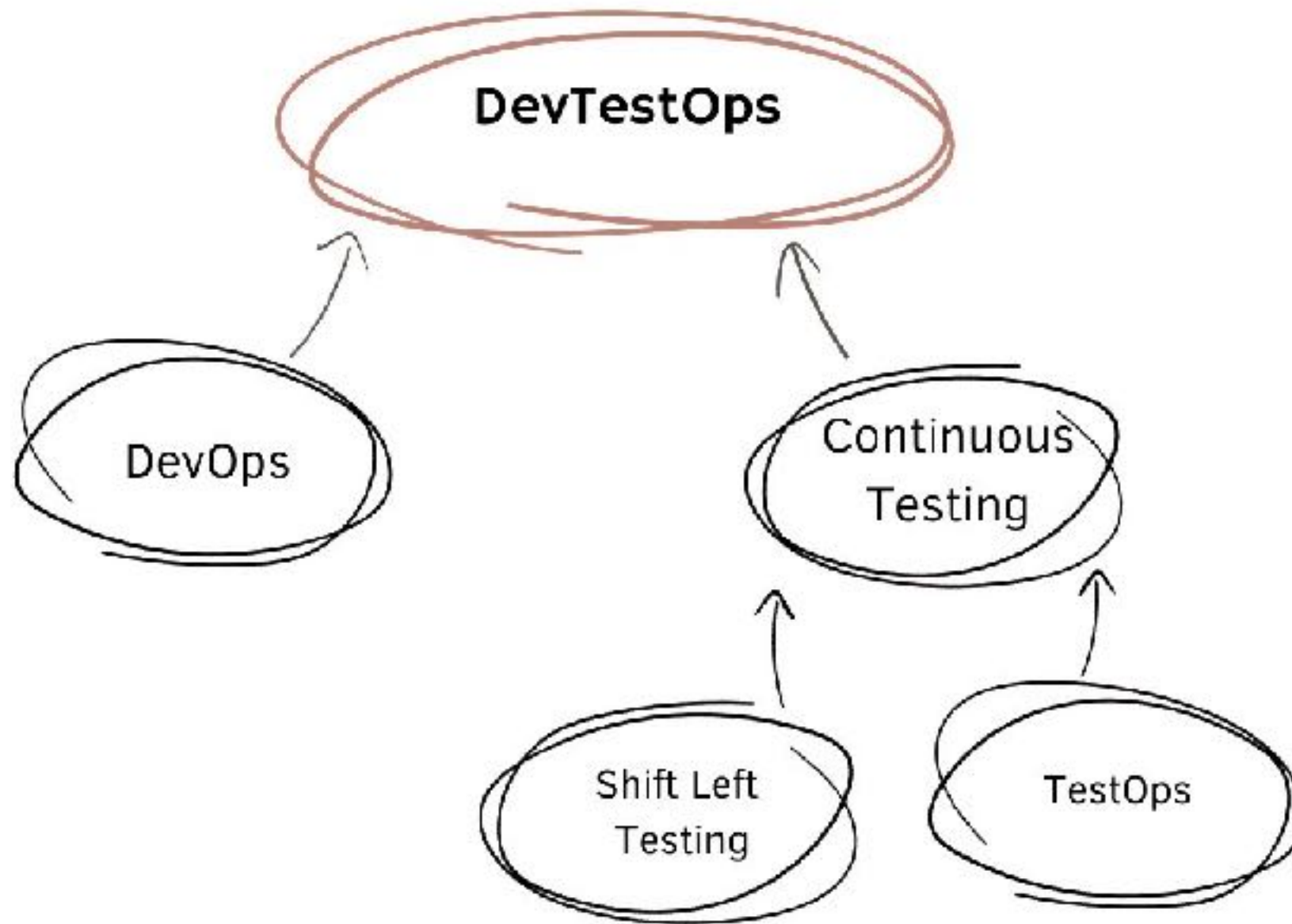
Scrum framework



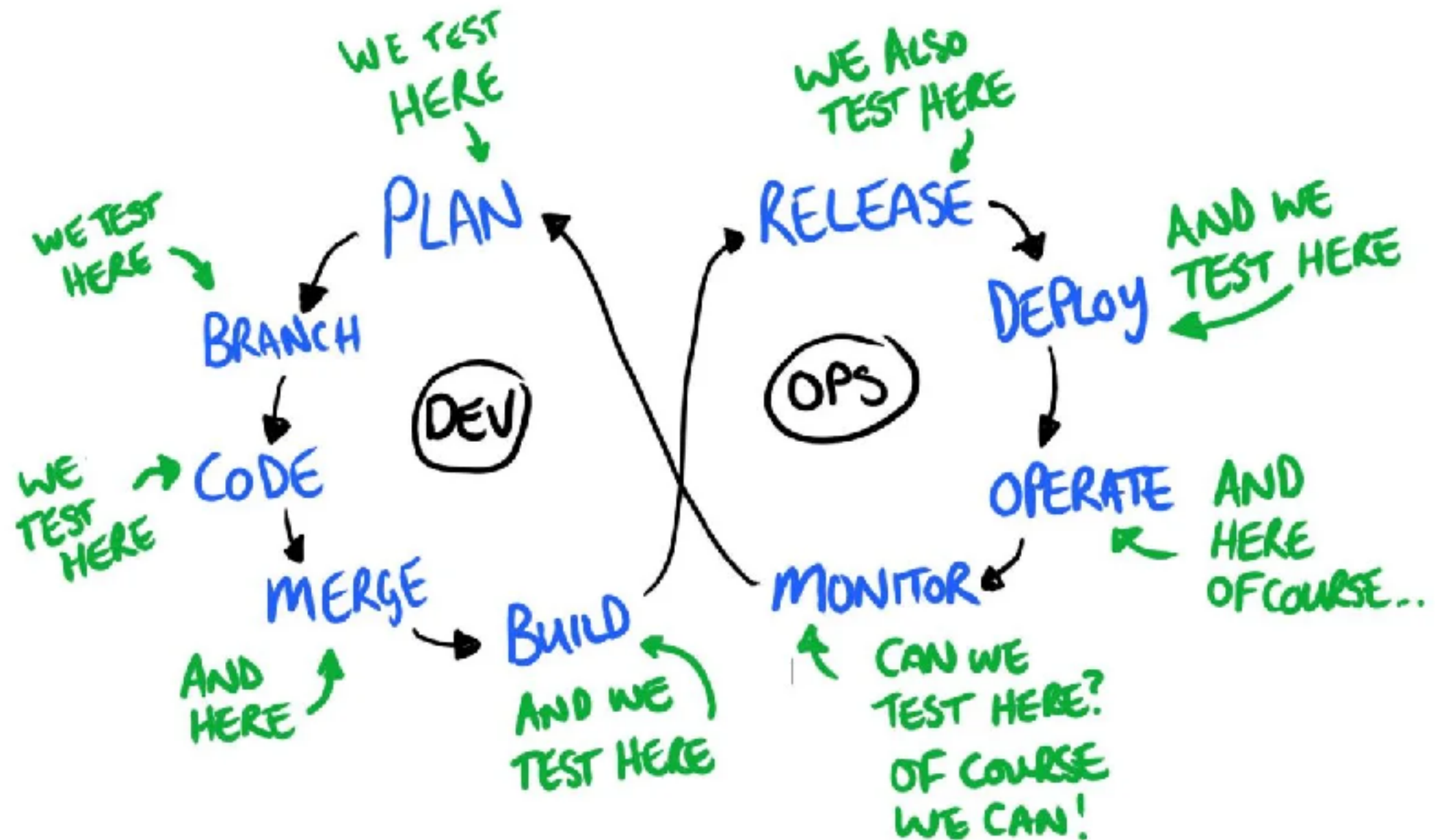
<https://www.scrum.org/resources/what-scrum-module>



DevOps + Continuous Testing



DevTestOps



Manual testing



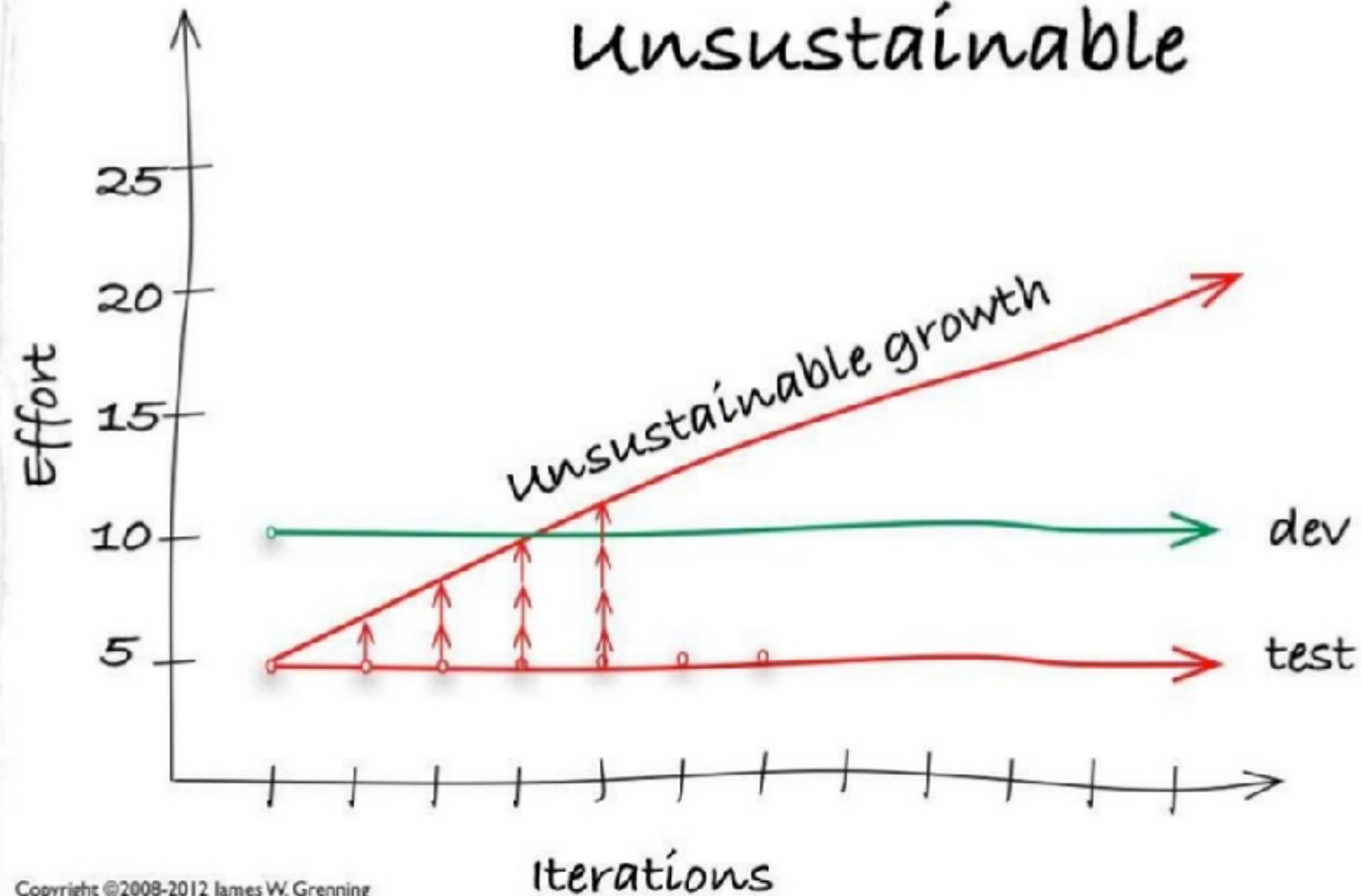
CAUTION



Human error



Manual Test is Unsustainable

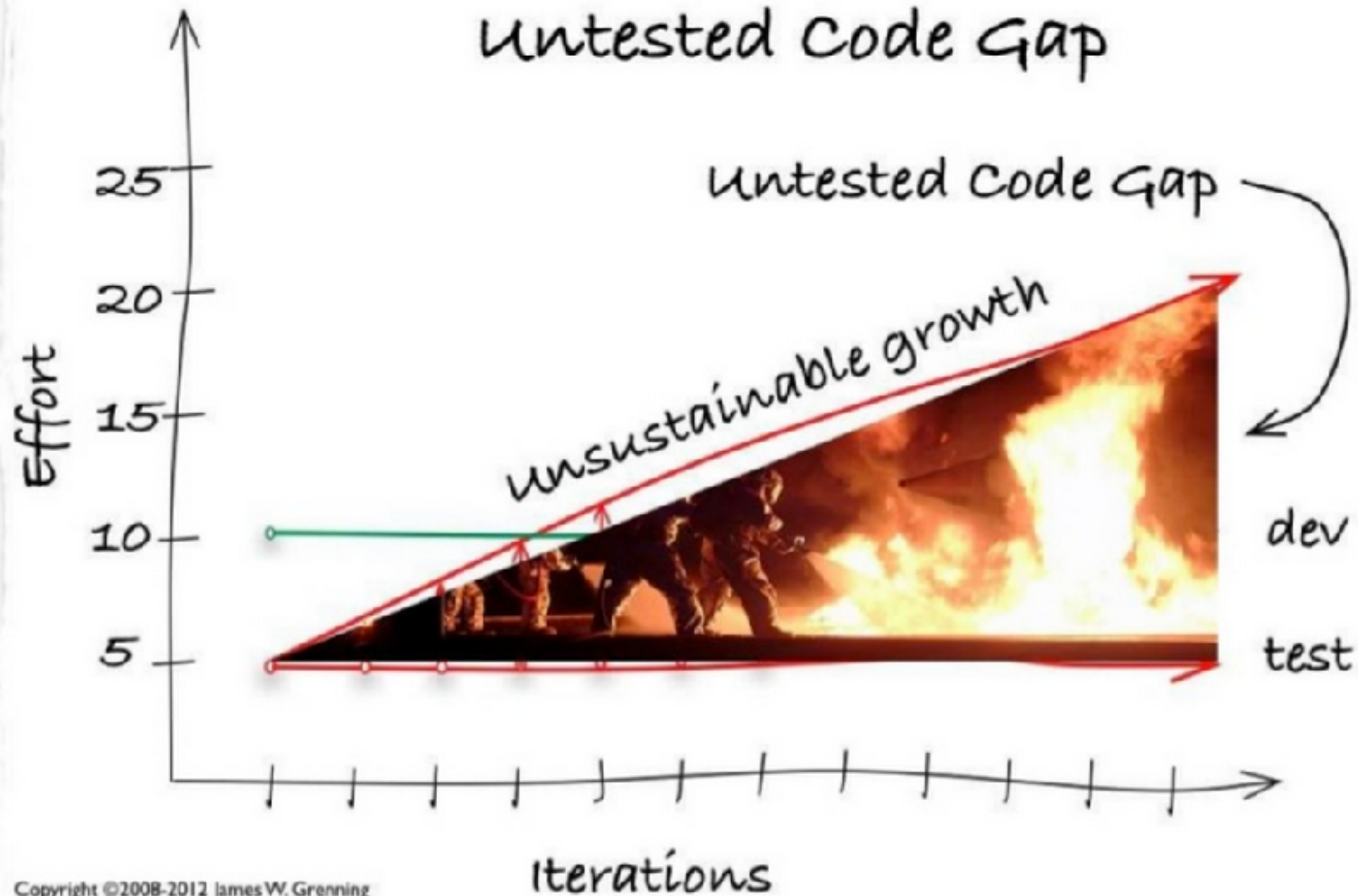


Copyright ©2008-2012 James W. Grenning
All Rights Reserved.

<https://wingman-sw.com/>



Risk Accumulates in the Untested Code Gap



<https://wingman-sw.com/>



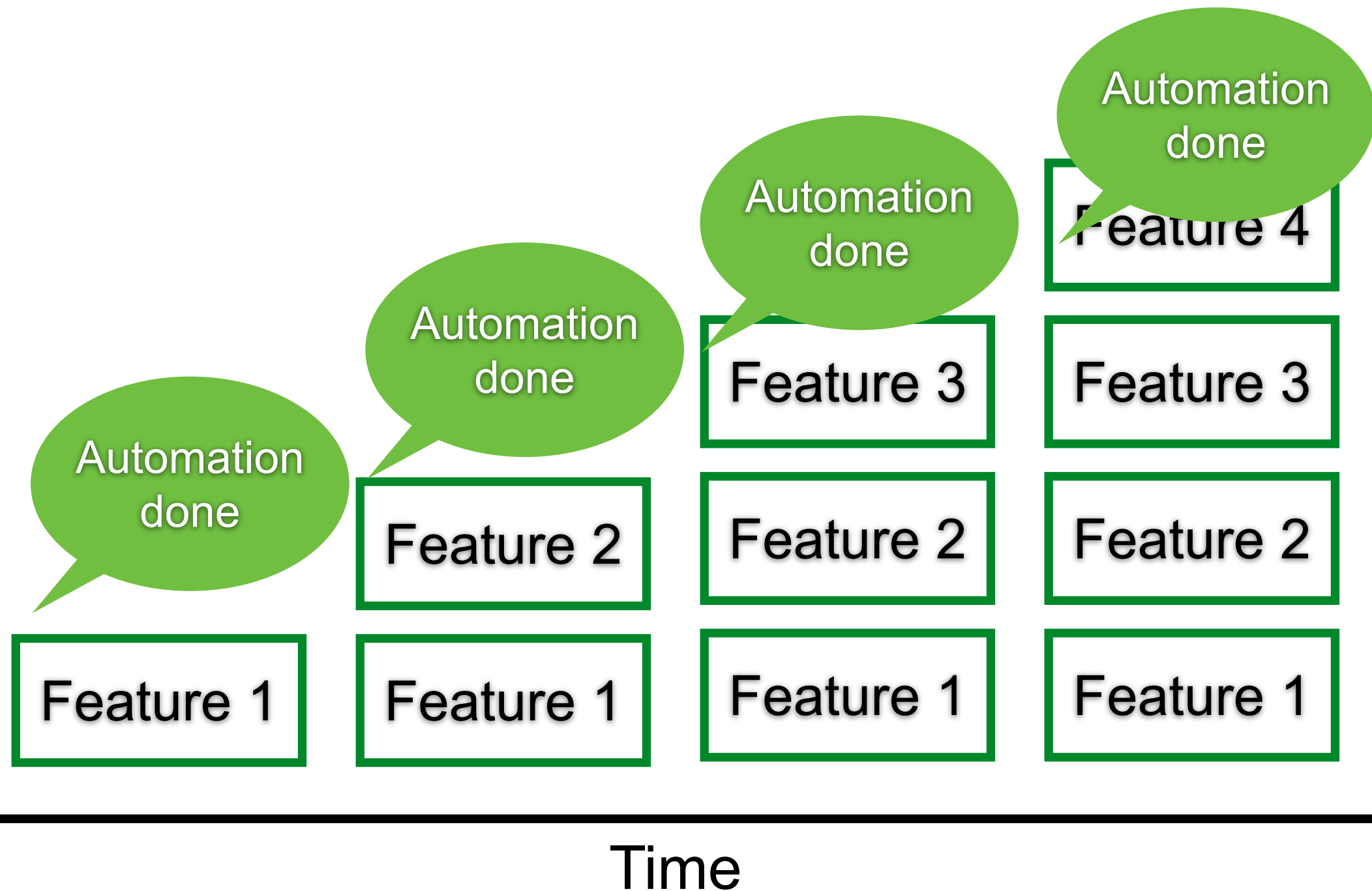
Regression Testing !!

Runs after every change to ensure that the change introduces no unintended breaks



Iterative and incremental process

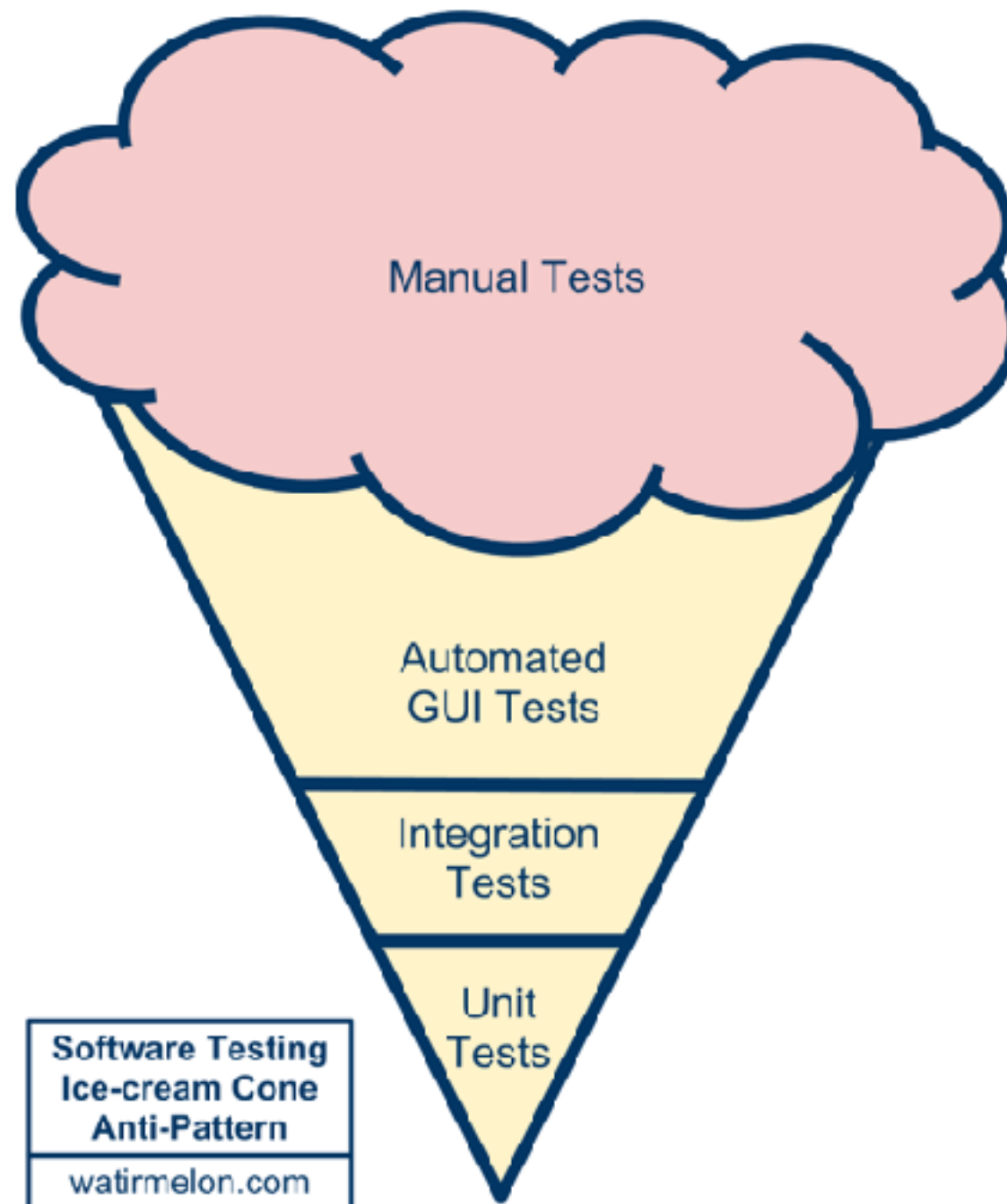
Done = coded and tested



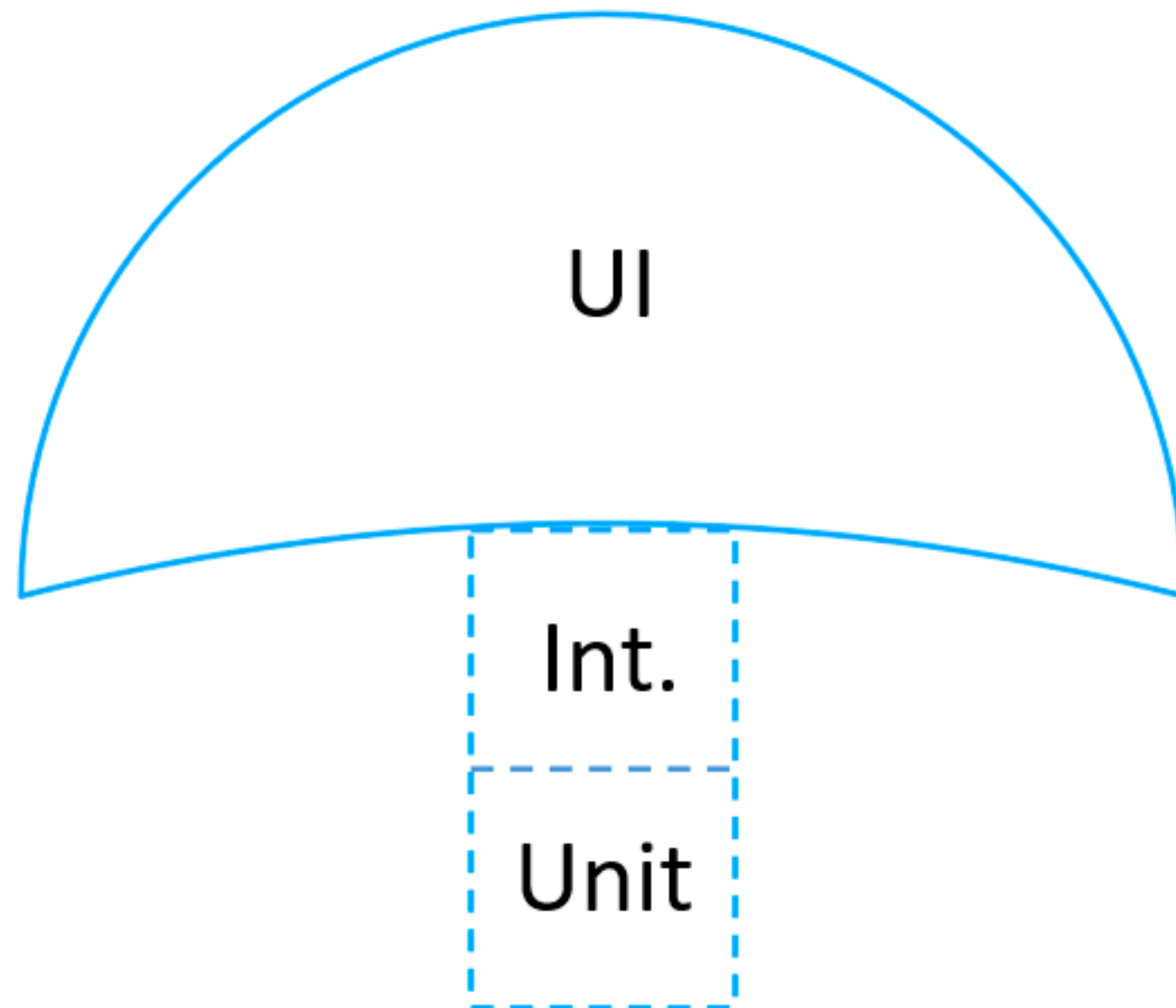
Testing pyramid



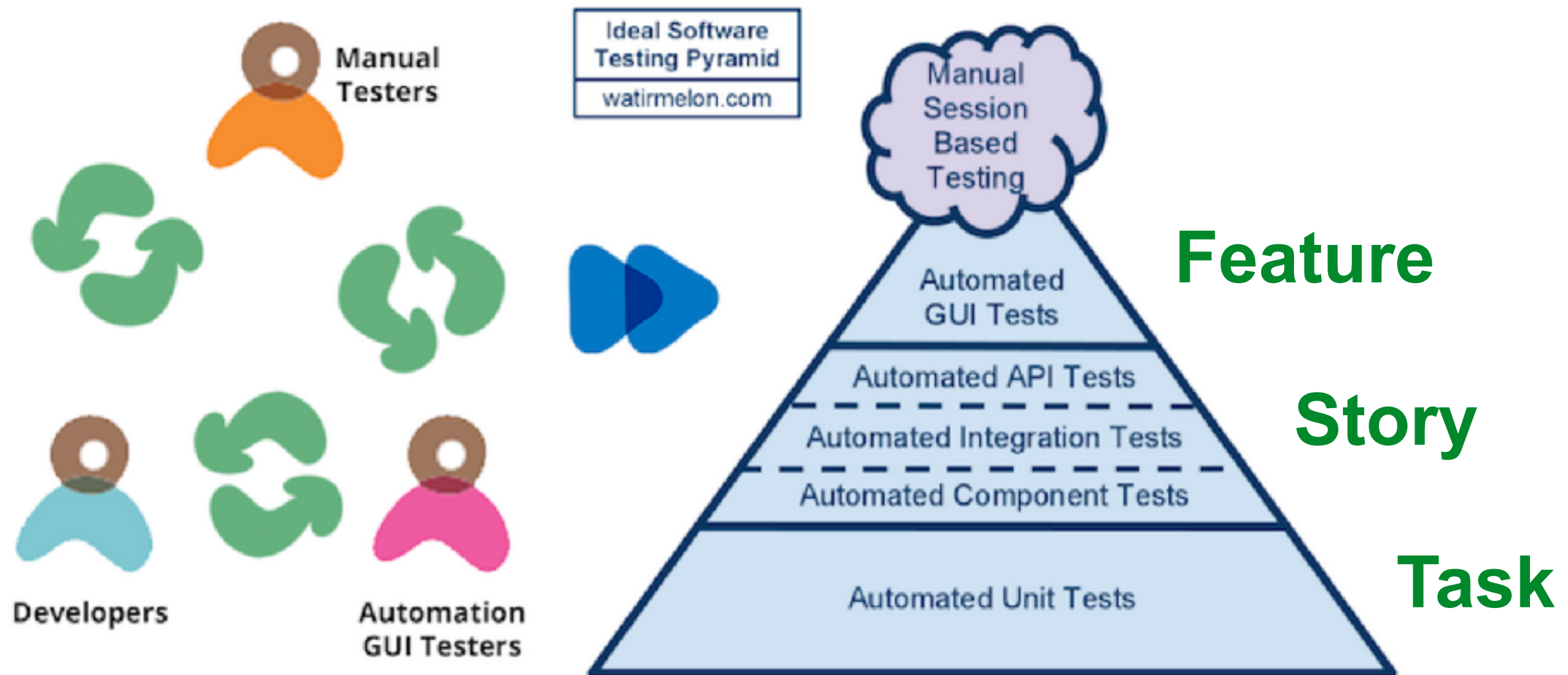
Ice-cream testing



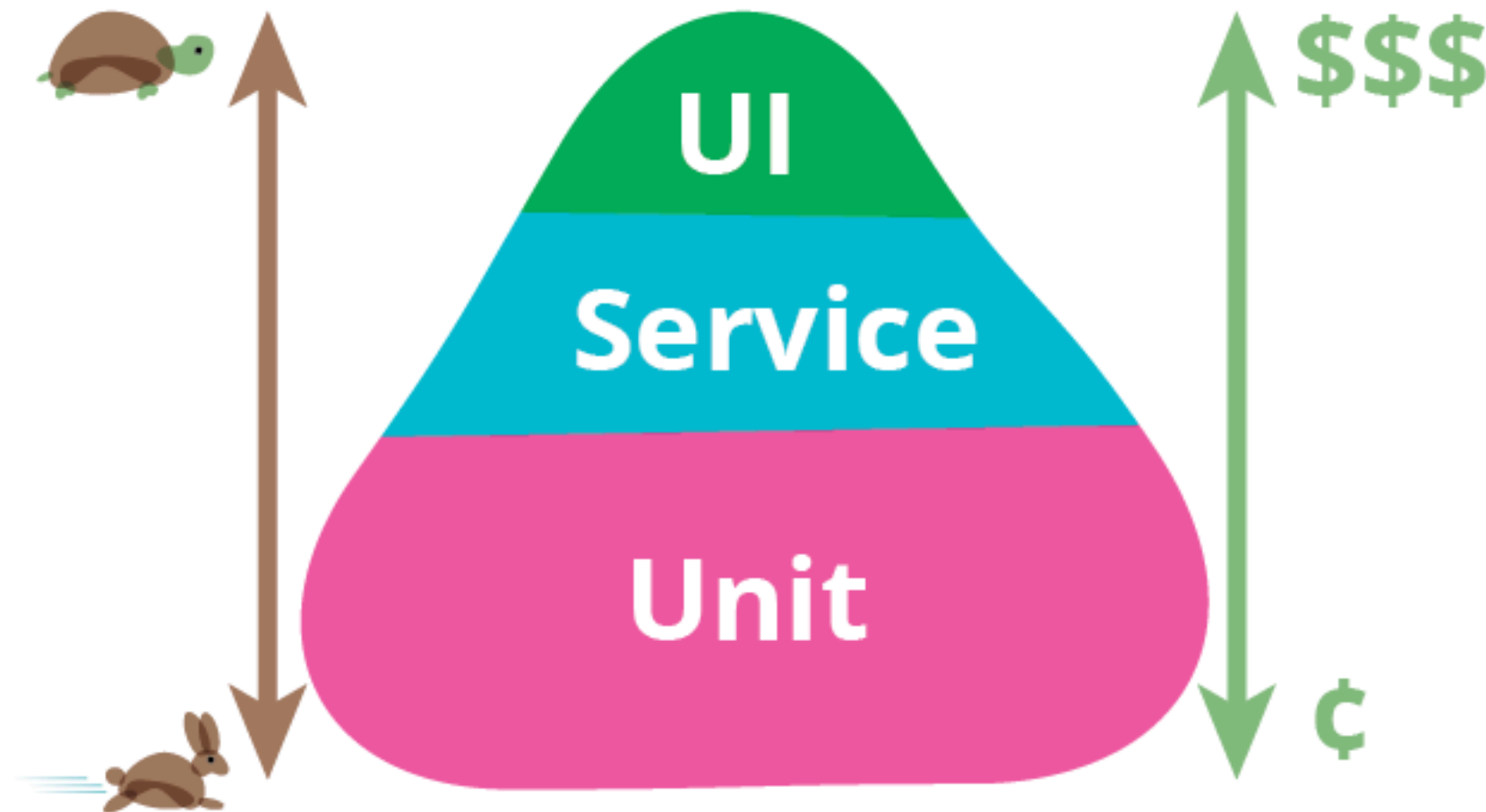
Mashroom testing



Testing Pyramid



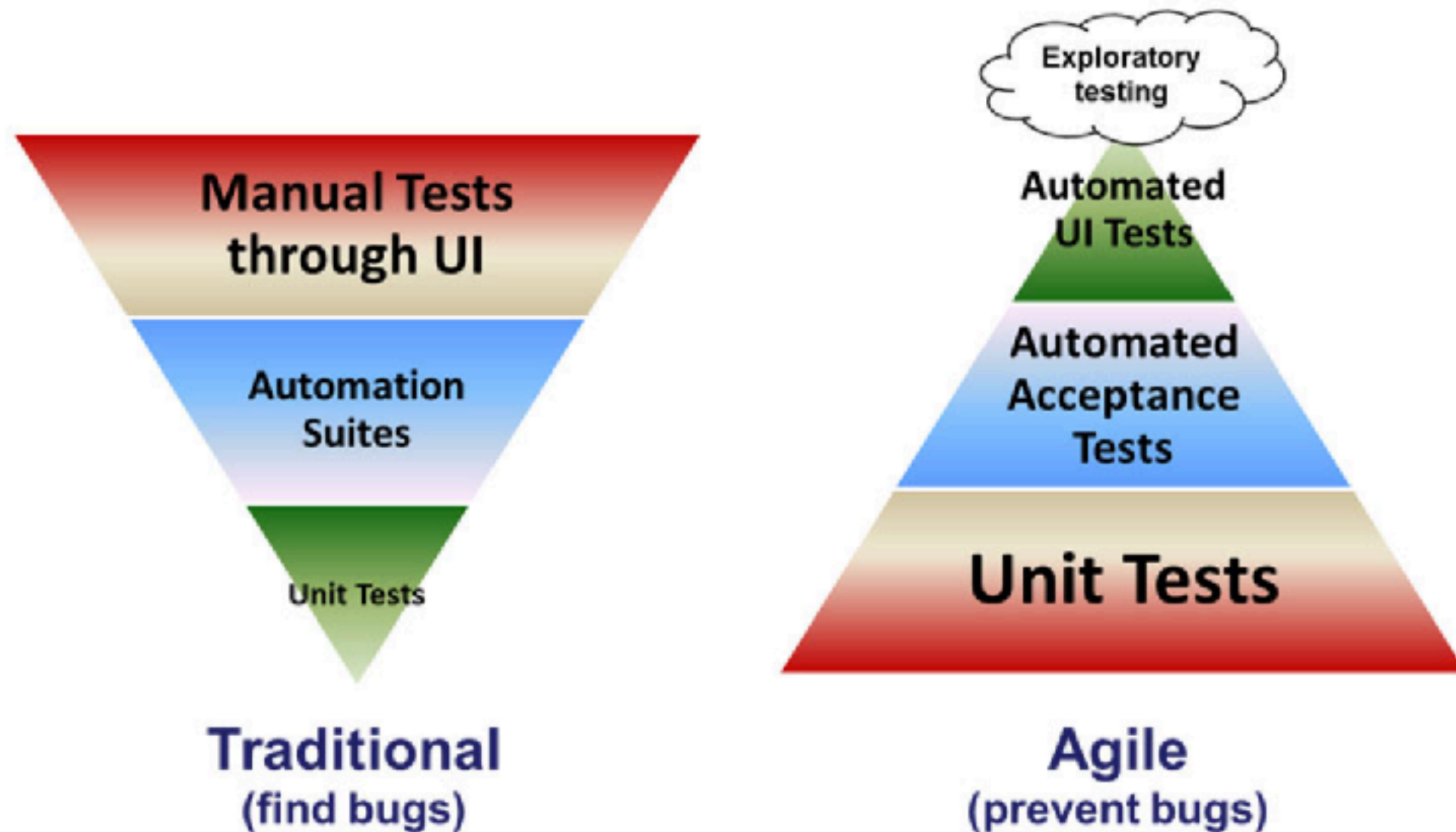
Testing Pyramid



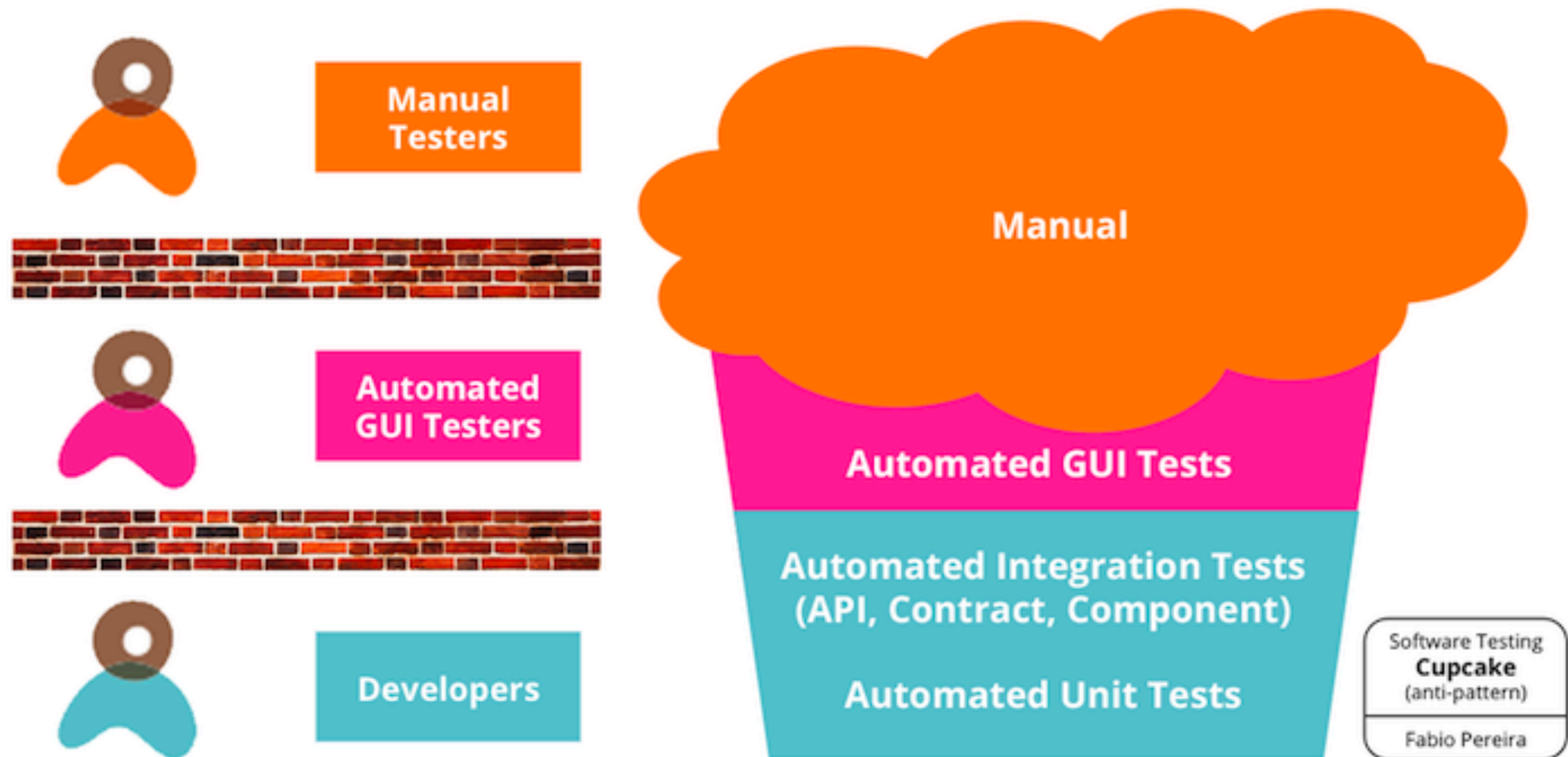
<https://martinfowler.com/bliki/TestPyramid.html>



Find vs Prevent



Cupcake testing !!



<https://www.thoughtworks.com/insights/blog/introducing-software-testing-cupcake-anti-pattern>



Trophy testing

THE FOUR TYPES OF TESTS

End to End

A helper robot that behaves like a user to click around the app and verify that it functions correctly.

Sometimes called "functional testing" or e2e.

Integration

Verify that several units work together in harmony.

Unit

Verify that individual, isolated parts work as expected.

Static

Catch typos and type errors as you write the code.



<https://kentcdodds.com/blog/the-testing-trophy-and-testing-classifications>



Understand What and Why to test ?

Discussion is very important



Remember !!

Test pyramid is a tool

To talk about automation tasks

How to prioritize and help to do automation ?

Way to make **visible to the whole team**



Where to start ?



What are the
biggest obstacles ?

Time/Tools/System/People



What should be careful ?

Automating end-to-end tests
User Interface are slow
Working with database
Working with external system
Automated every paths



Thinking about automation

Incremental testing

Easy to test

Testable

Fast customer feedback

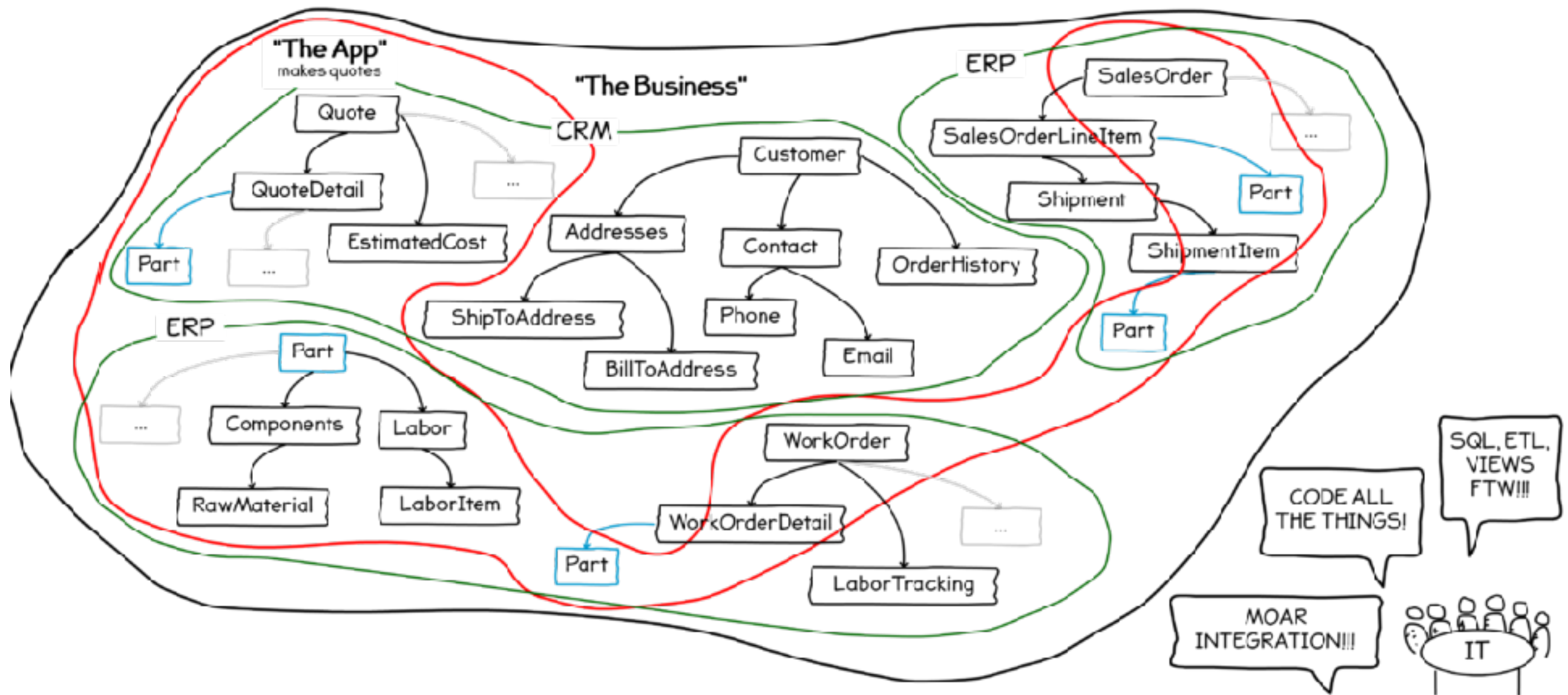


Know your System Architecture

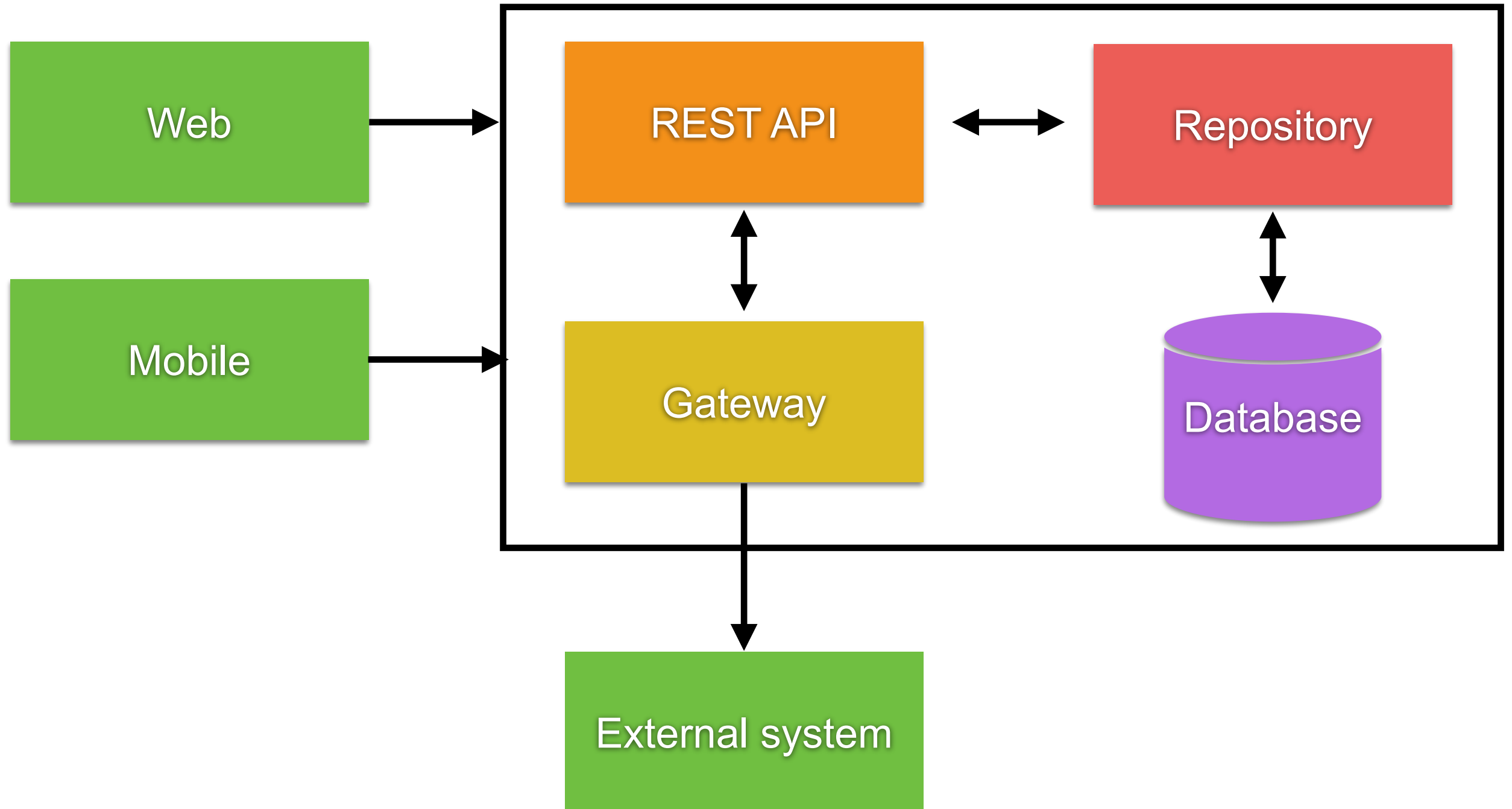


System impacts !!

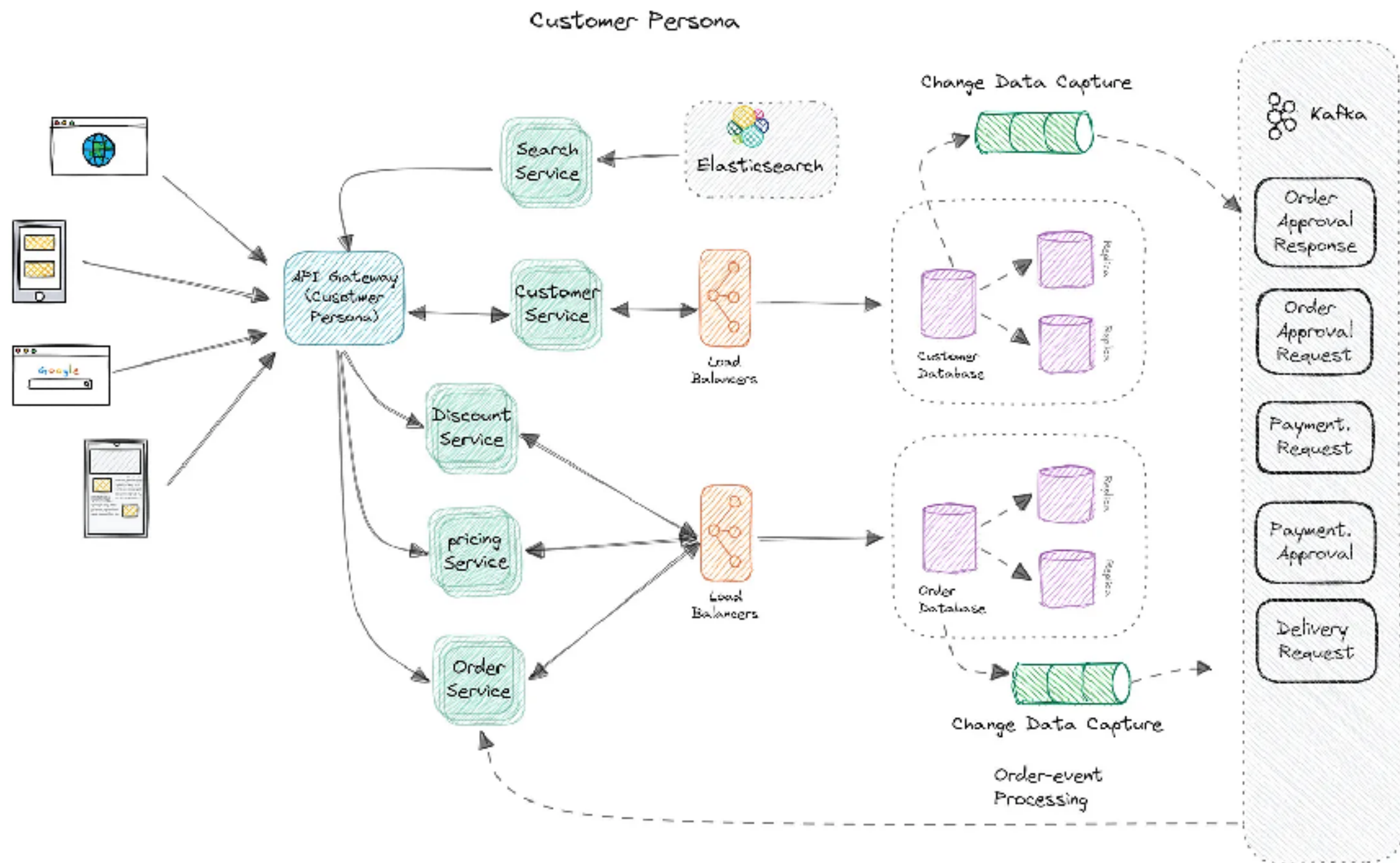
Know your systems



Simple Architecture



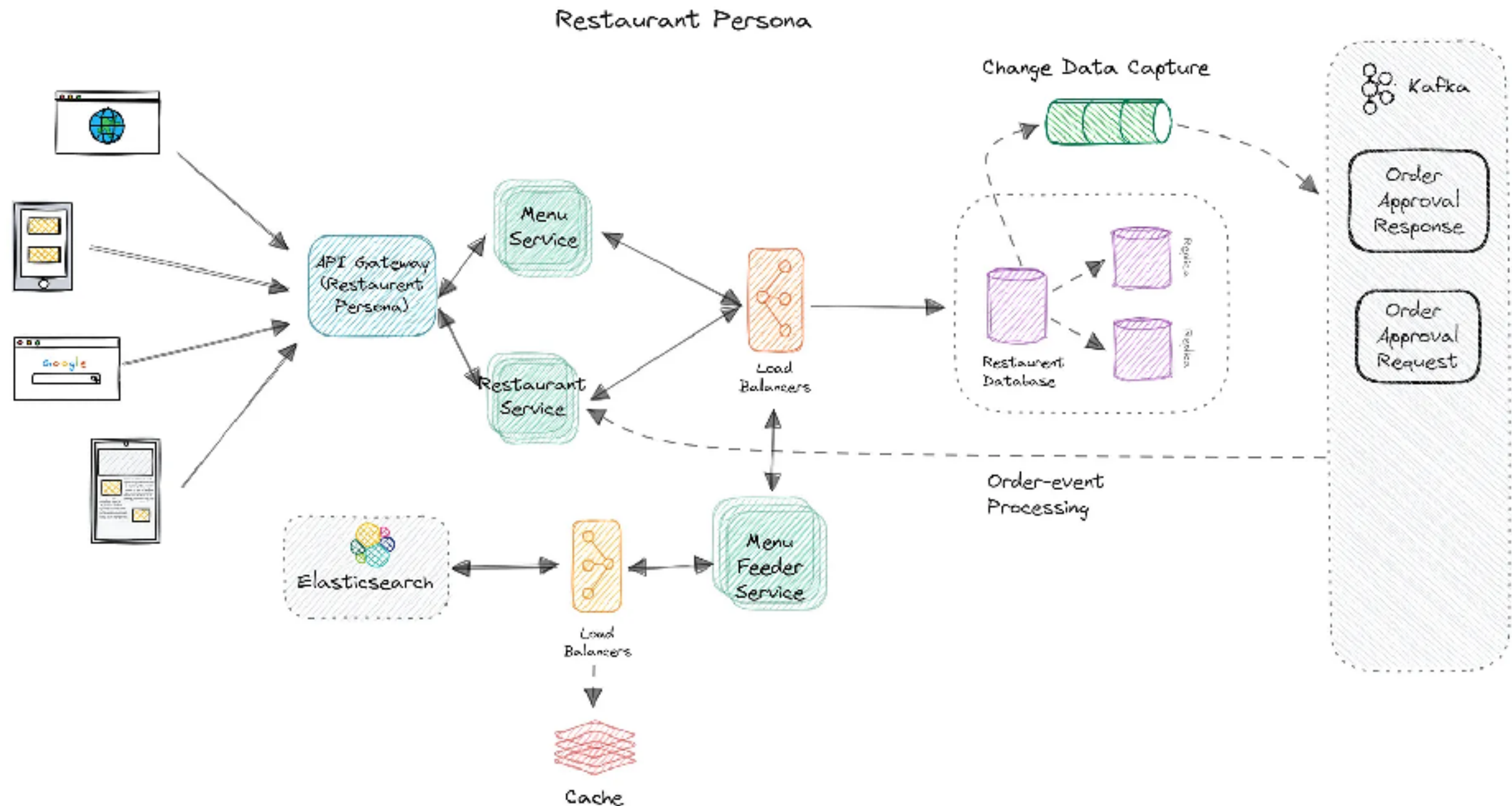
Food Delivery (Customer)



<https://medium.com/@systemdesignbychk/system-design-a-deep-dive-into-the-food-ordering-system-f84ae6375ce3>



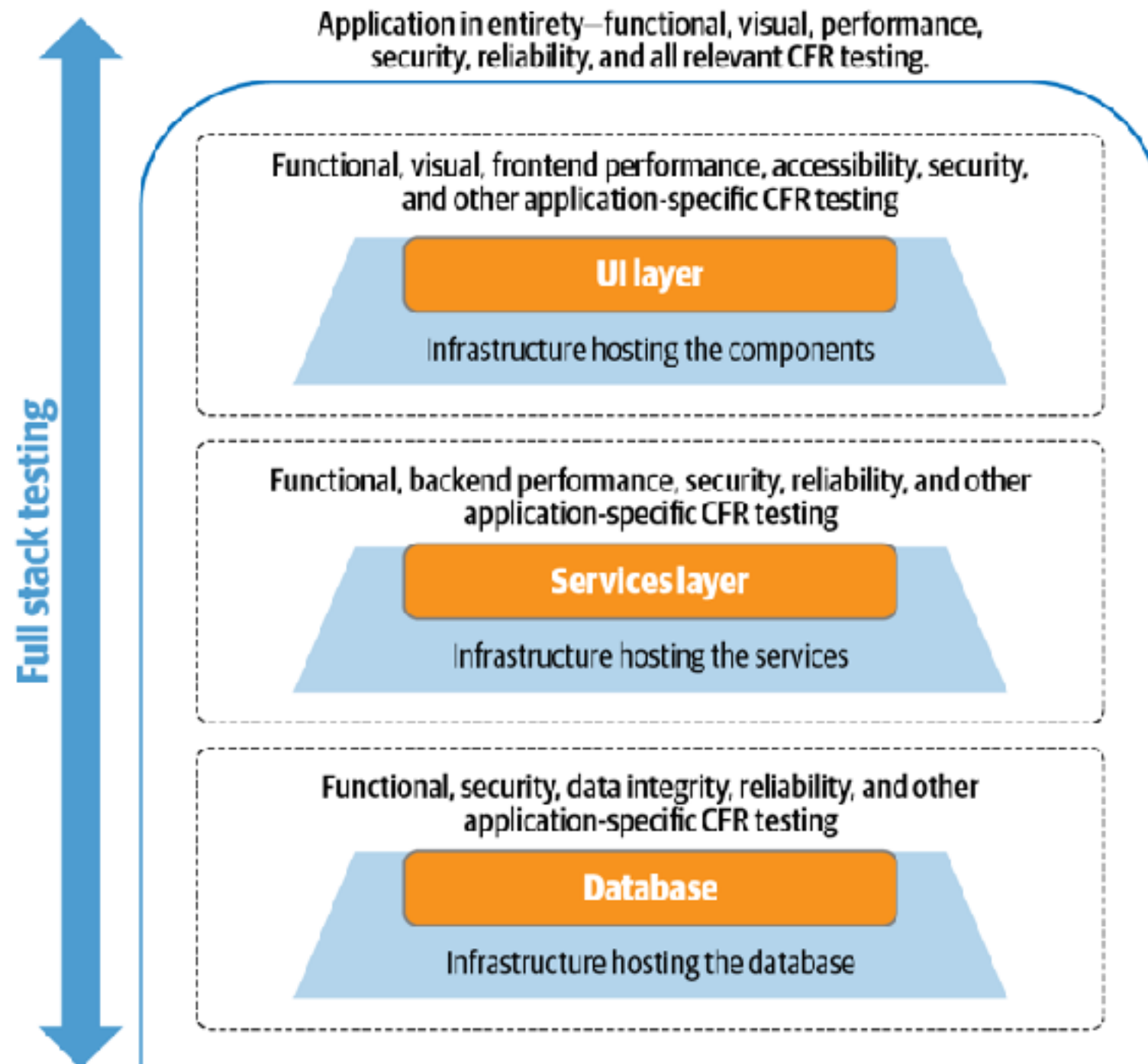
Food Delivery (Restaurant)



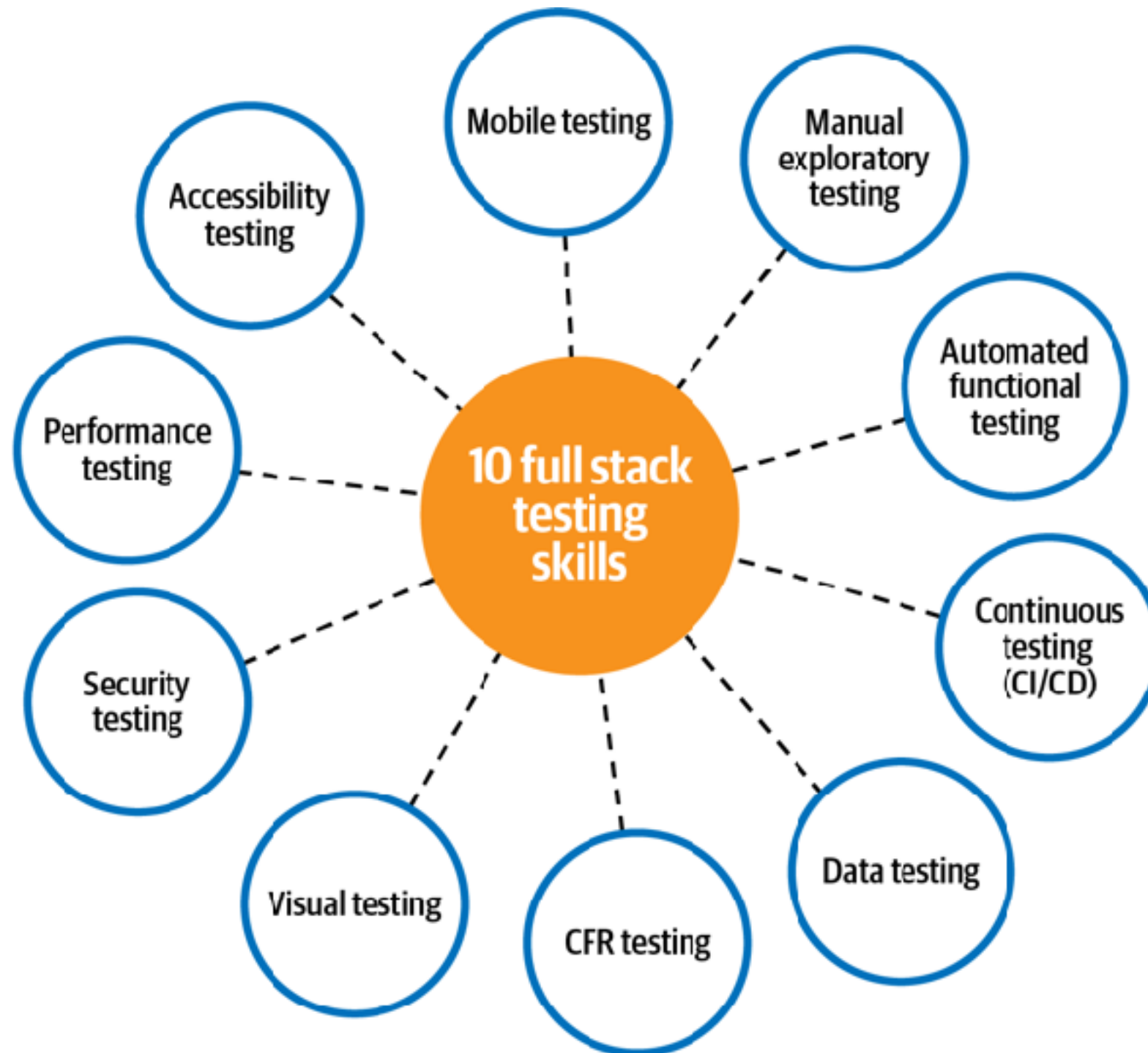
<https://medium.com/@systemdesignbychk/system-design-a-deep-dive-into-the-food-ordering-system-f84ae6375ce3>



Full Stack Testing

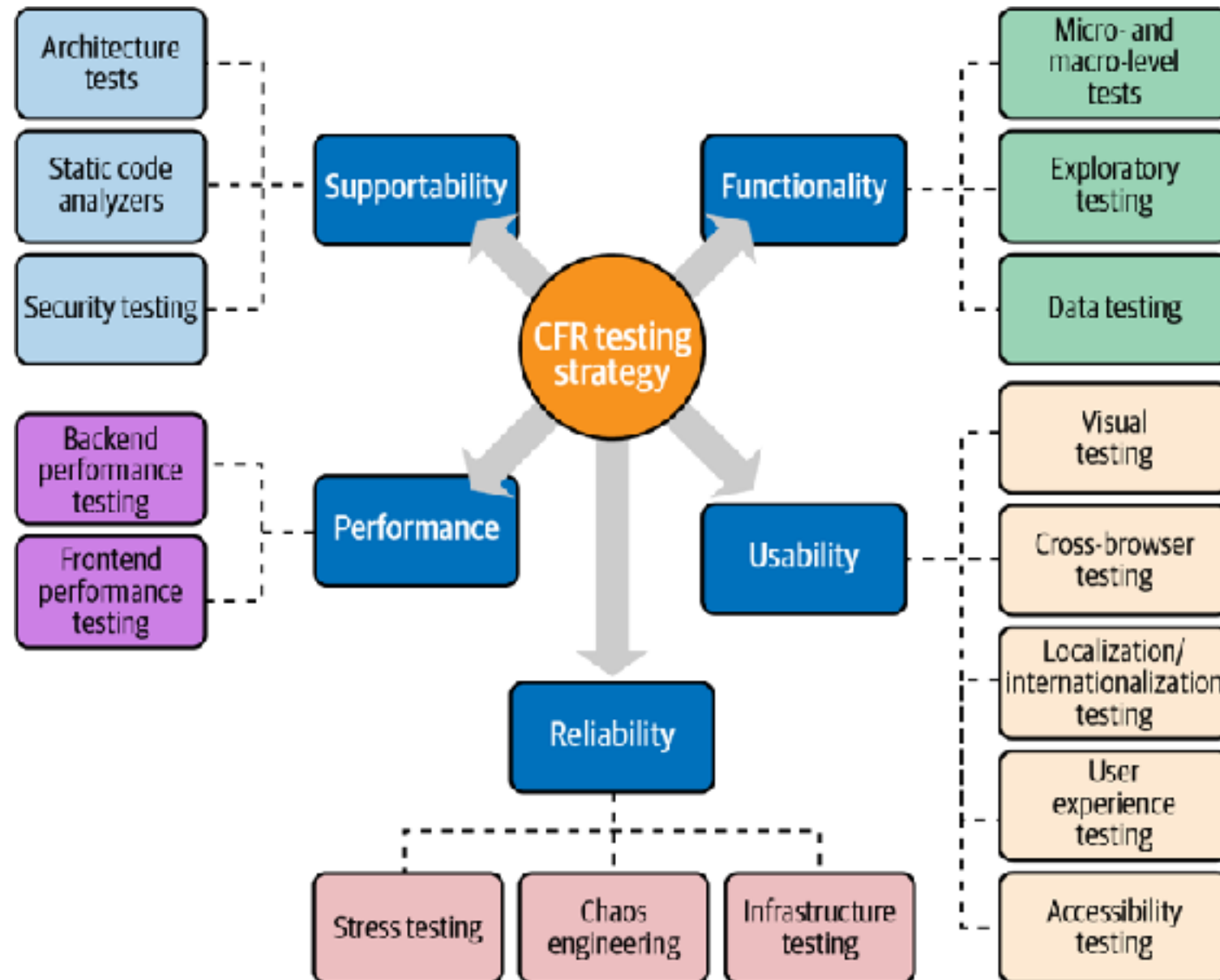


Full Stack Testing Skills



CFR testing

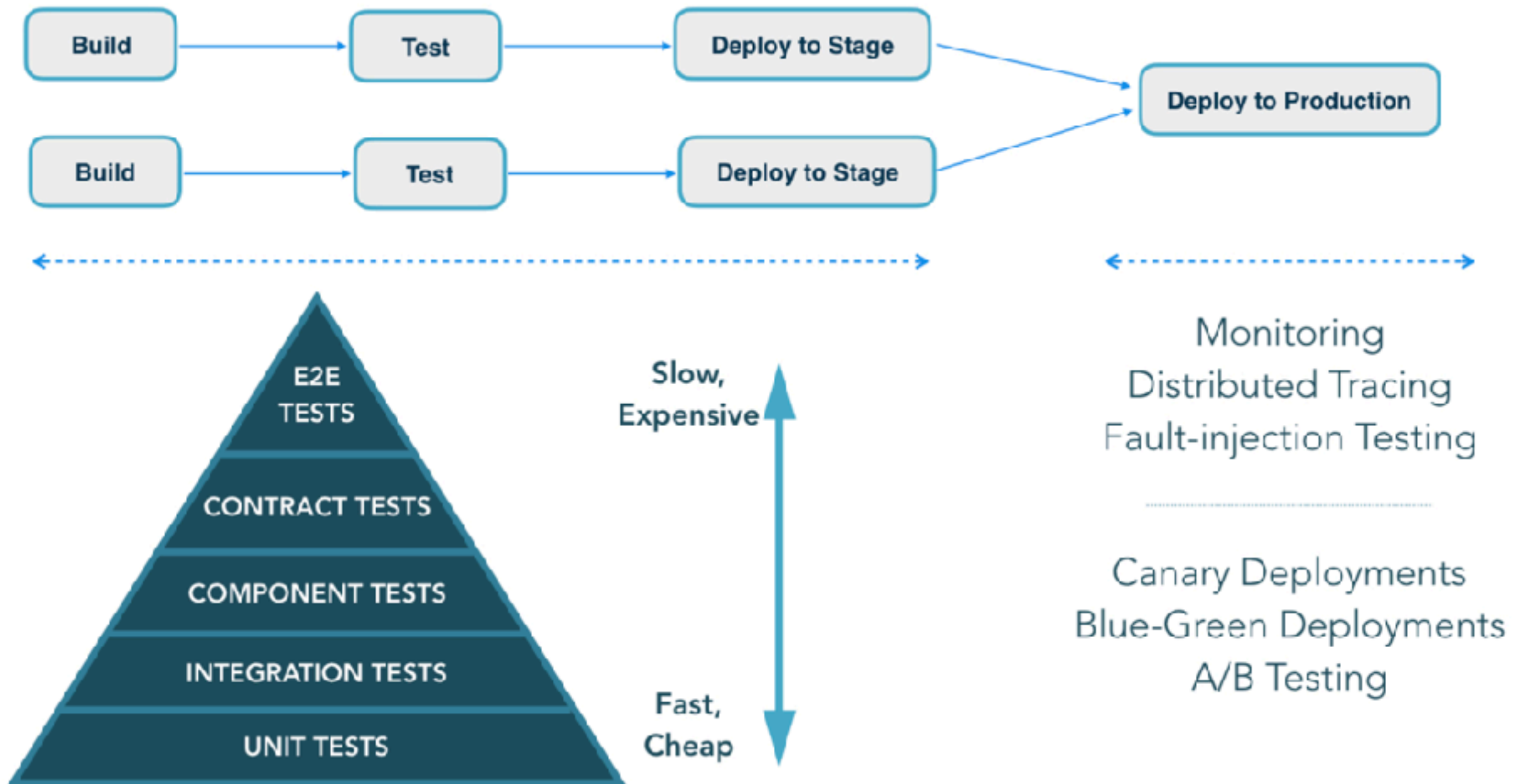
Cross Functional Requirement



Test Strategies



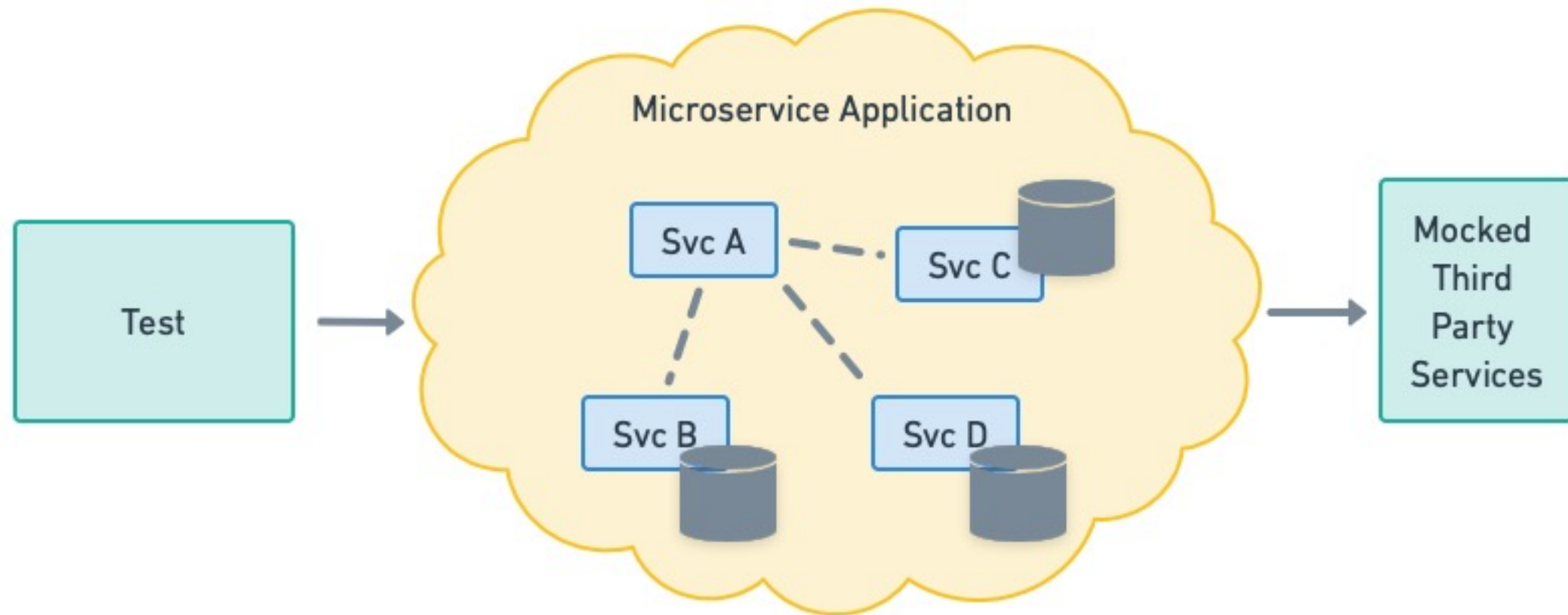
Test strategy



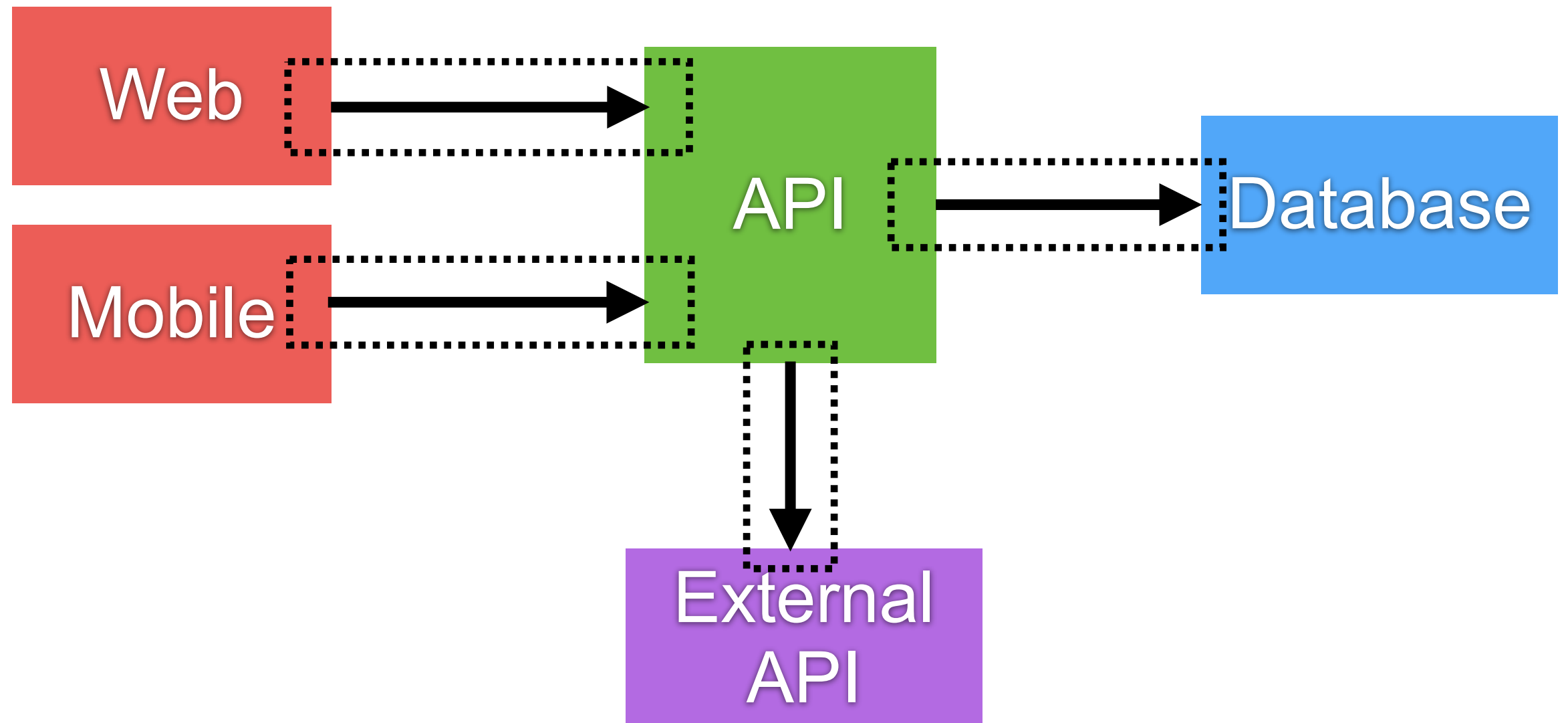
<https://martinfowler.com/articles/microservice-testing/>



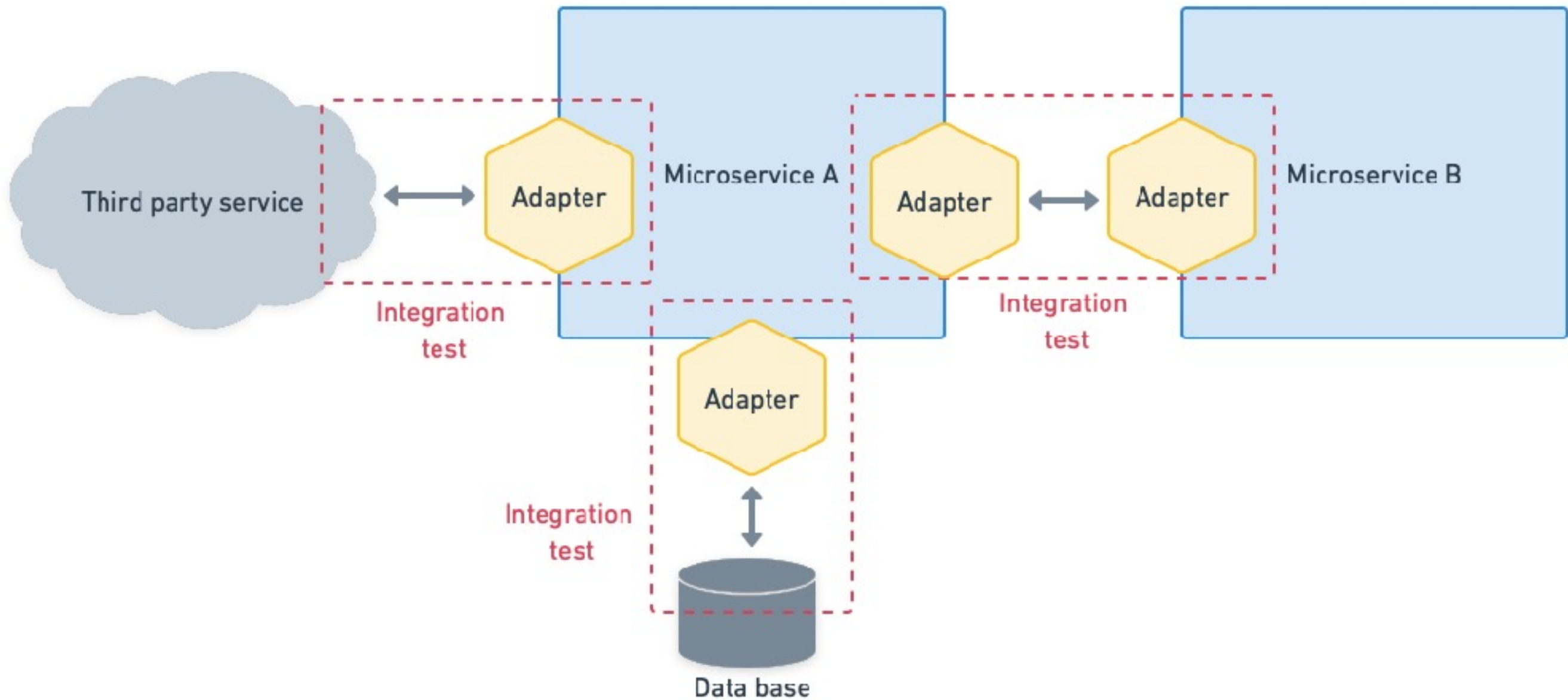
End-to-End testing



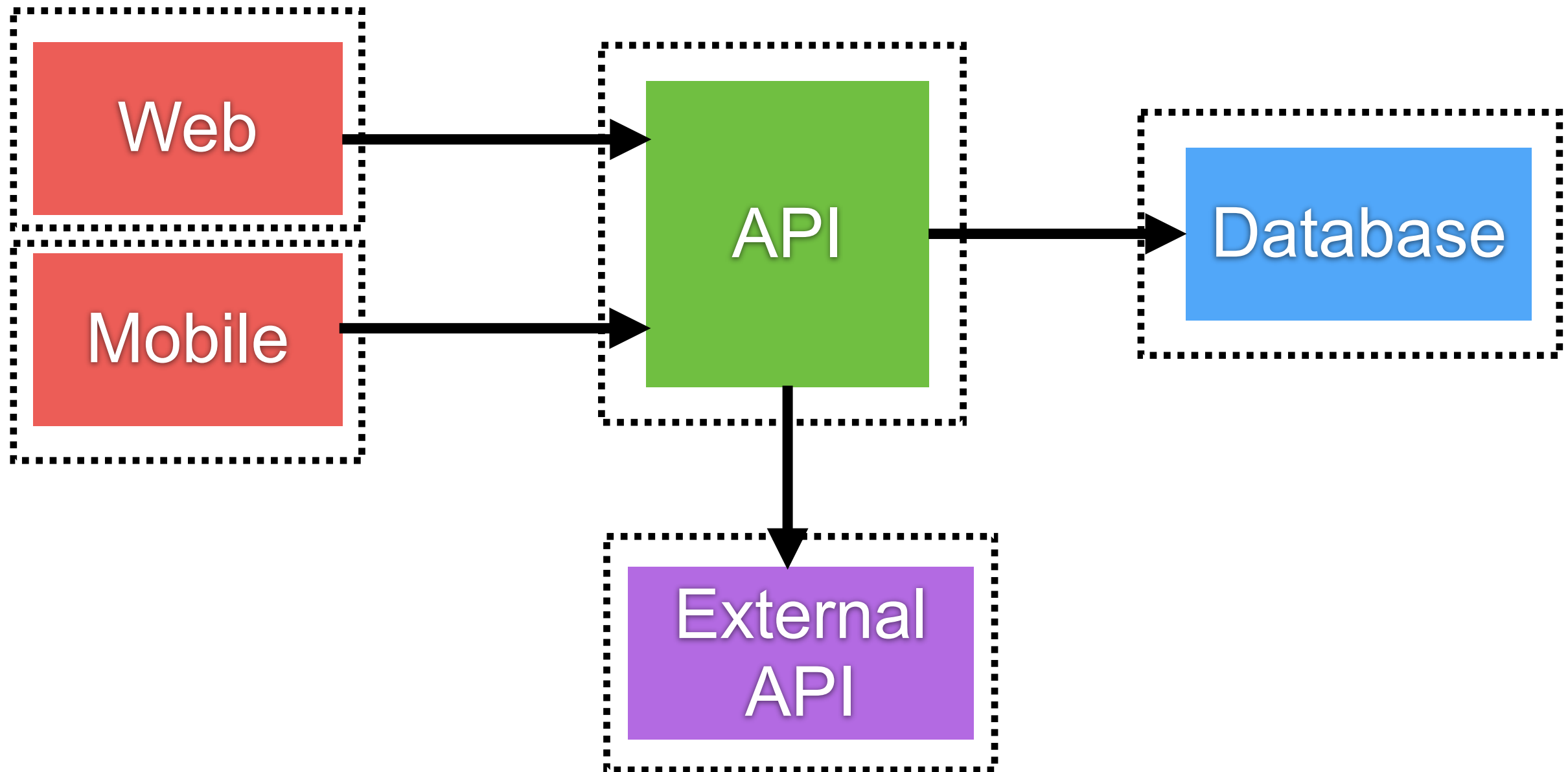
Integration testing



Integration testing

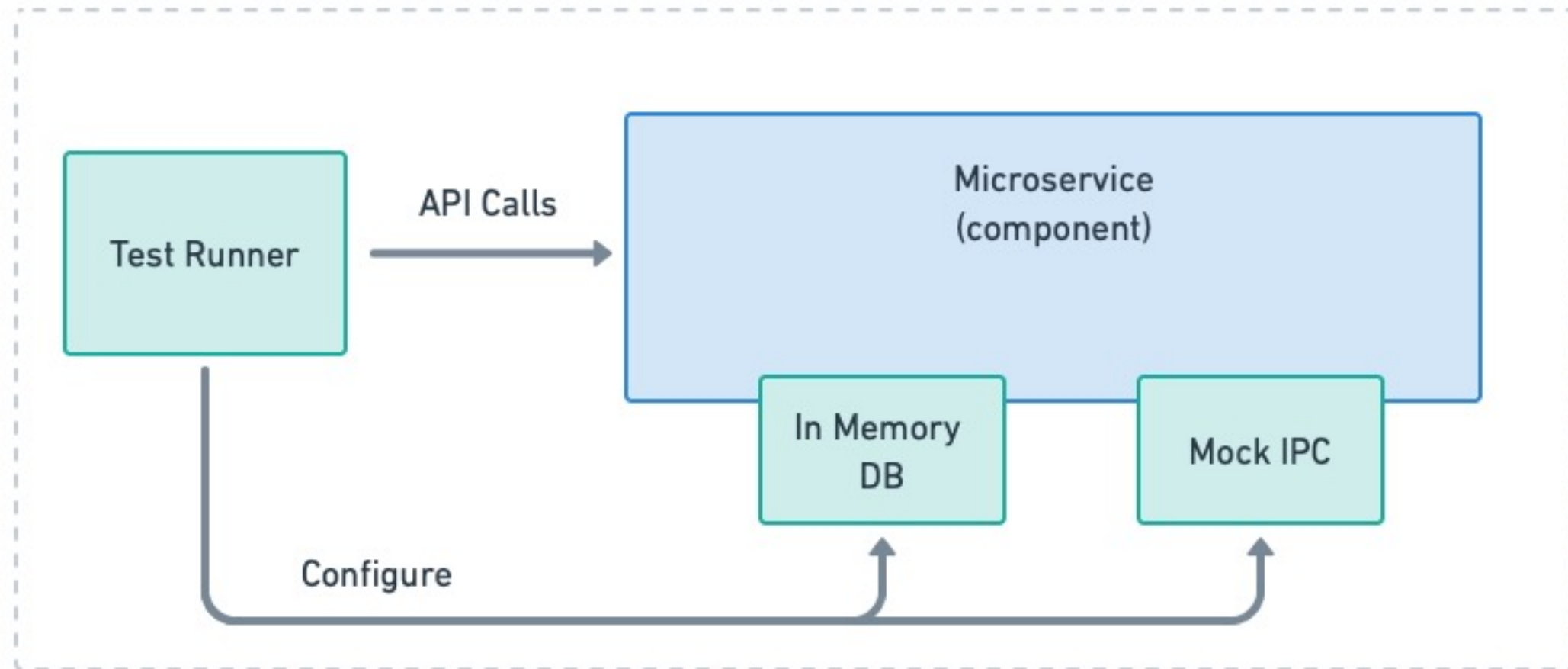


Service component testing



Service component testing

In-process component test



Contract testing

Consumer



Provider



Contract

Request GET /user/1
Response 200 OK

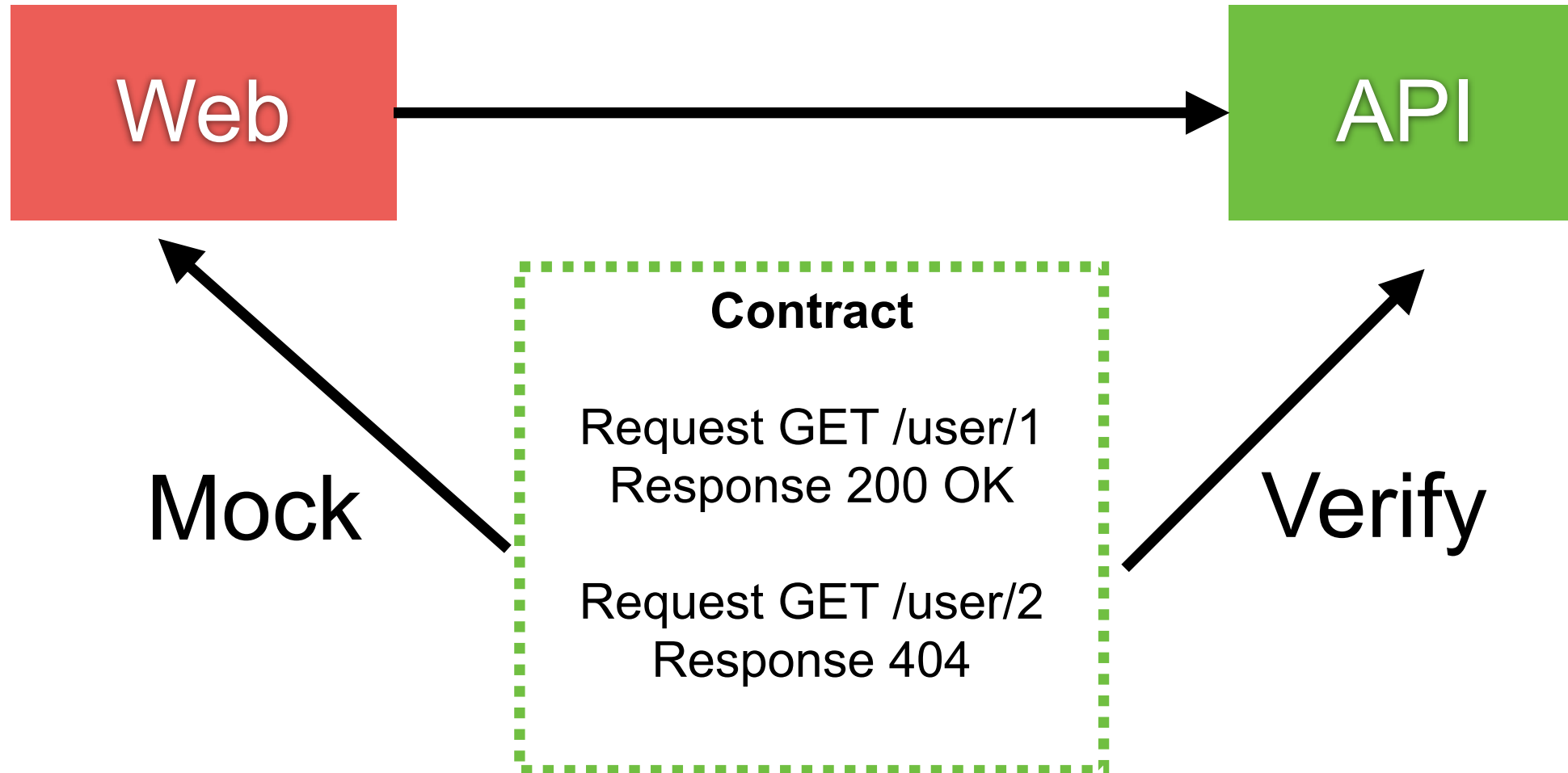
Request GET /user/2
Response 404



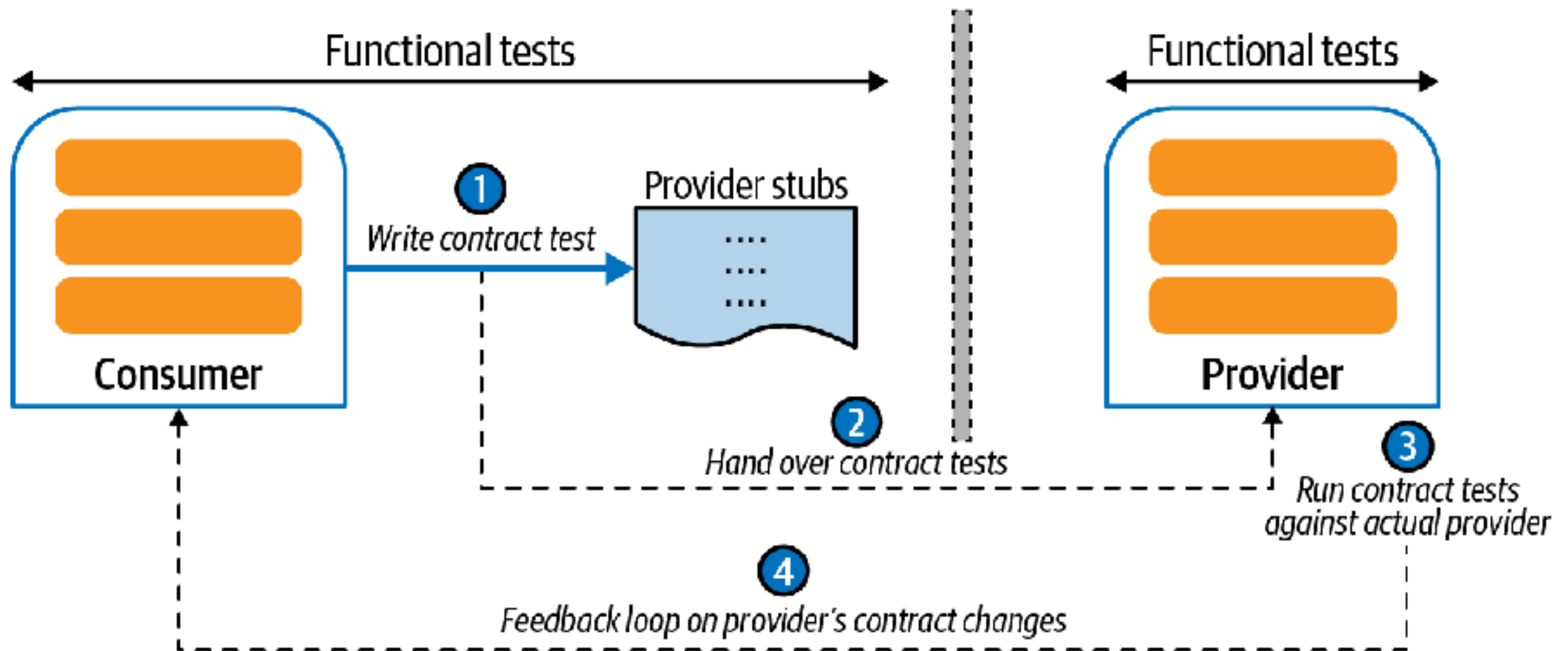
Contract testing

Consumer

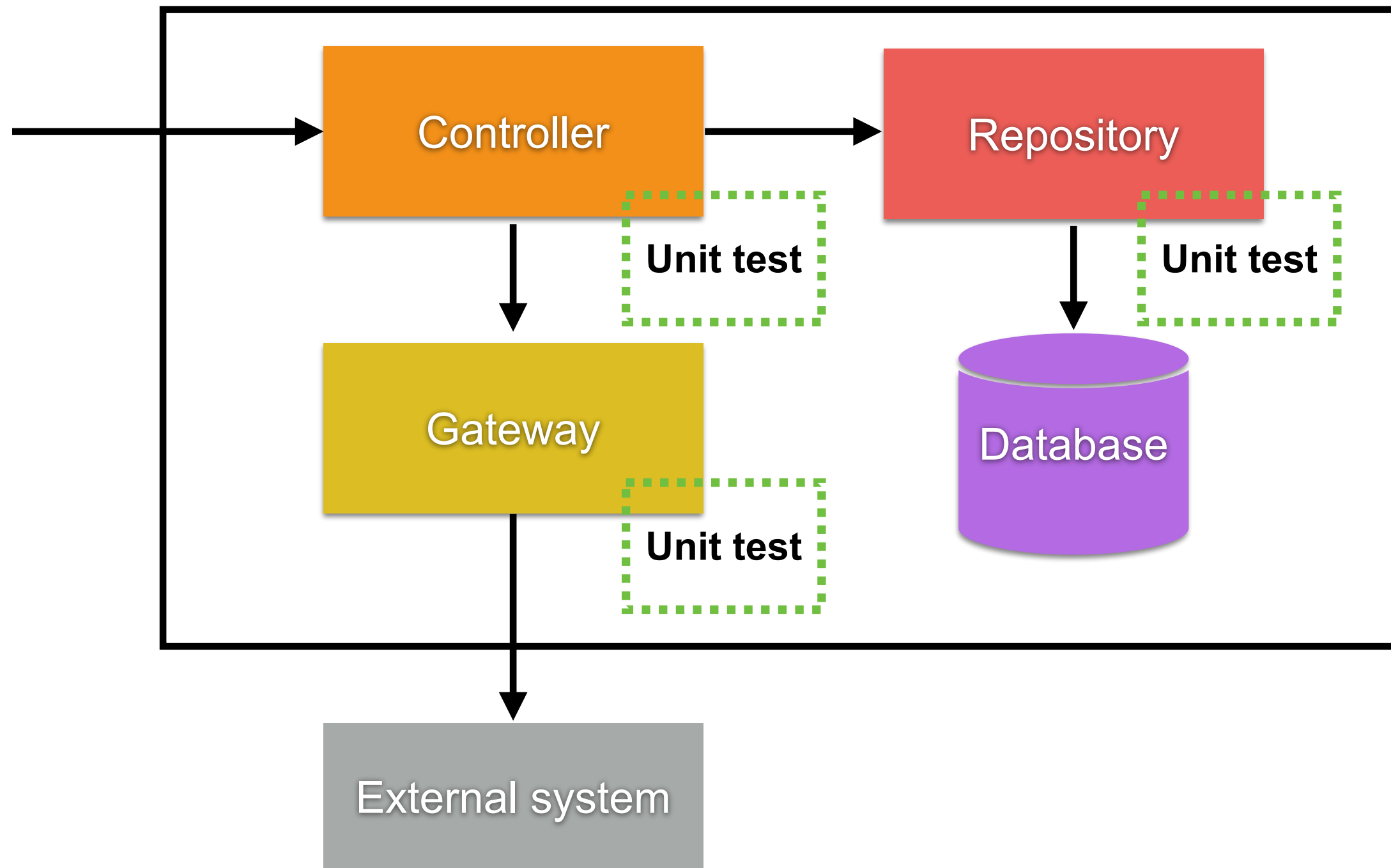
Provider



Contract testing



Unit testing



Test Double

Dummy

Stub

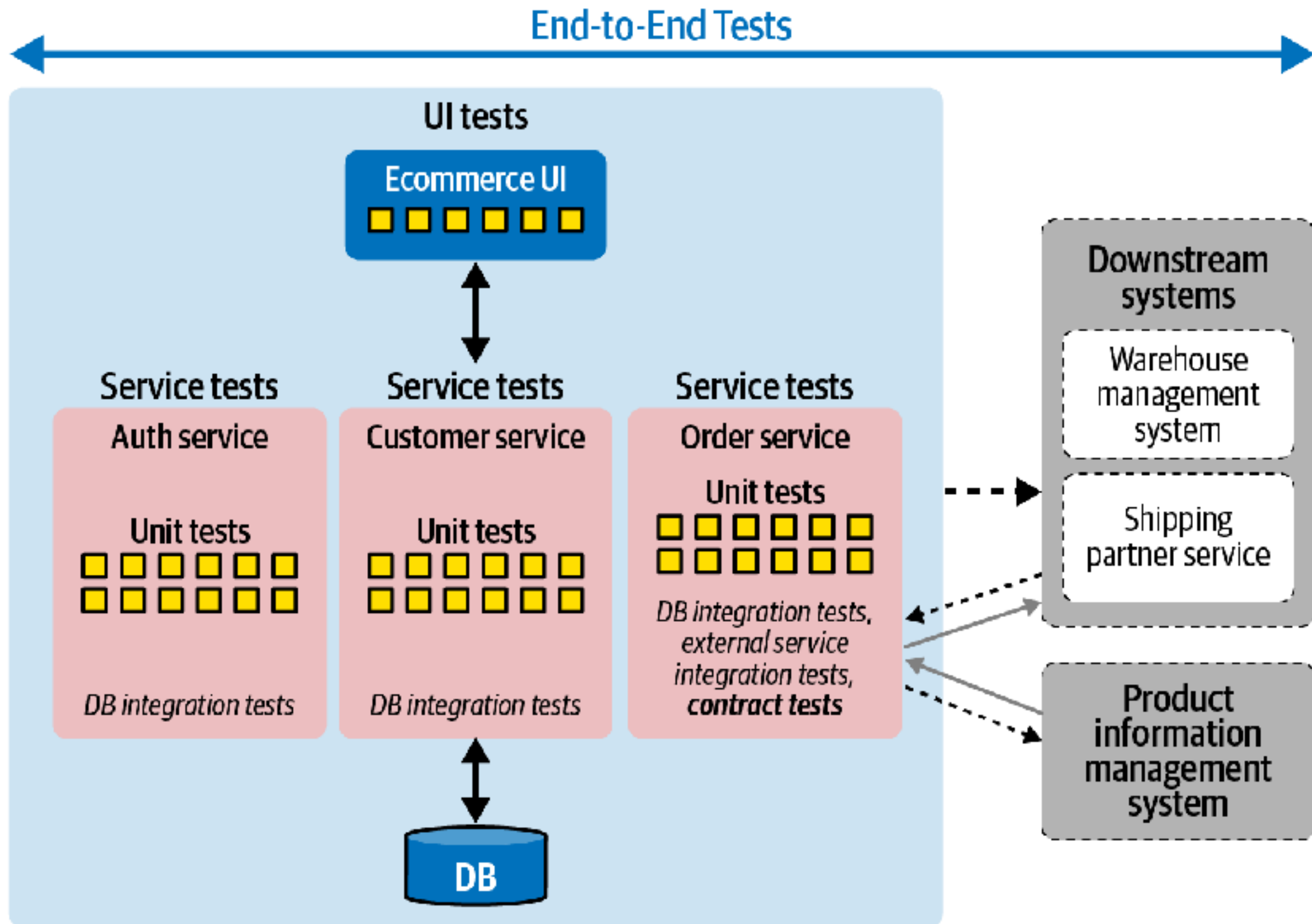
Spy

Mock

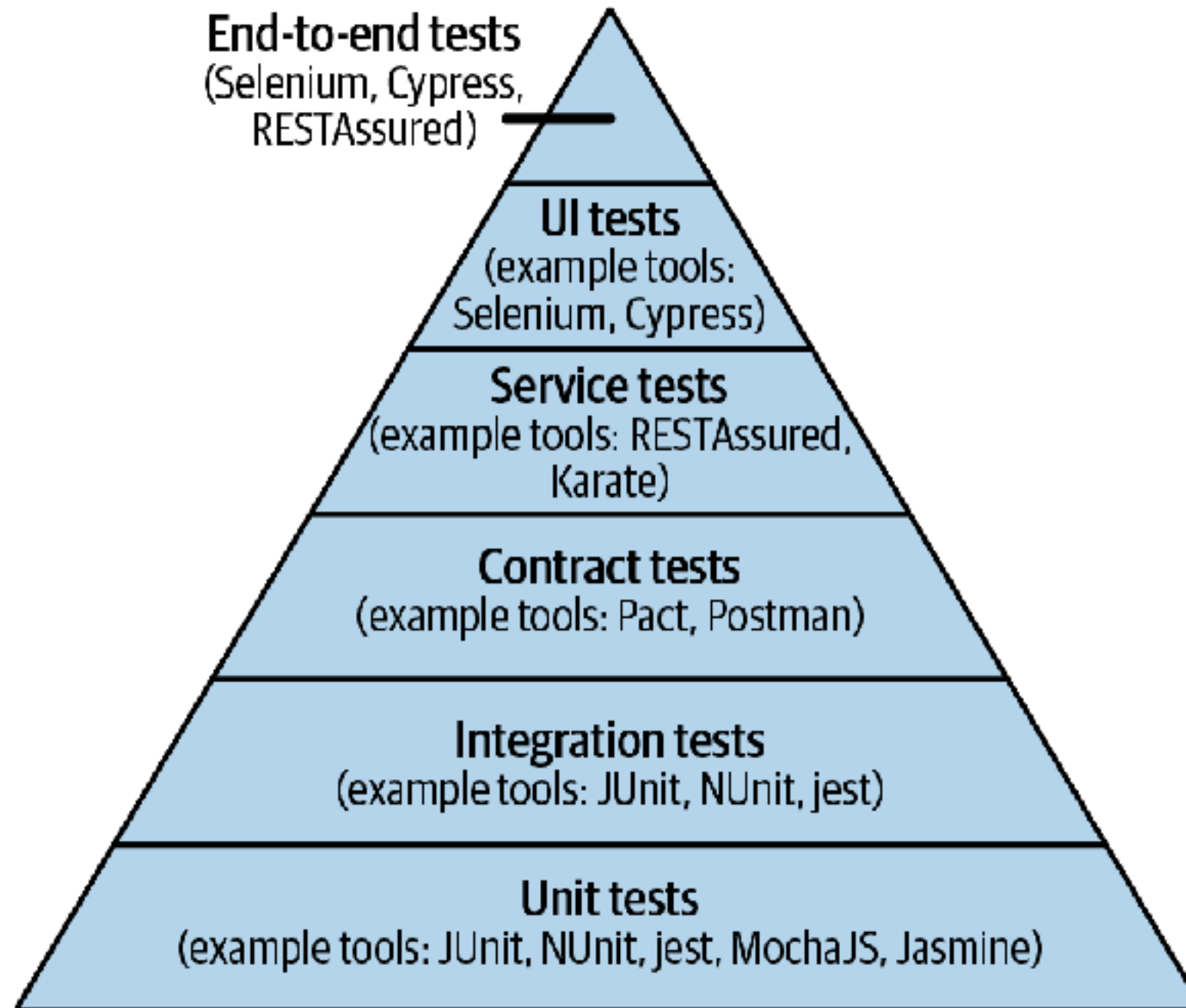
Fake

<http://xunitpatterns.com/Test%20Double.html>





Example Testing Tools



Test design



Economy of Test Design

Easy to understand

Easy to maintain

Readable by the business

One purpose per test

Repeatable

Poor test practices reduce the benefits



Good Test

F.I.R.S.T + U

Fast

Isolate

Repeat

Self-verify

Timely

Understand



Respect your tests

Don't ignore it if it fails

Fix the code or fix the test

Always refactor or improve

**100% of regression tests
must pass all the time**

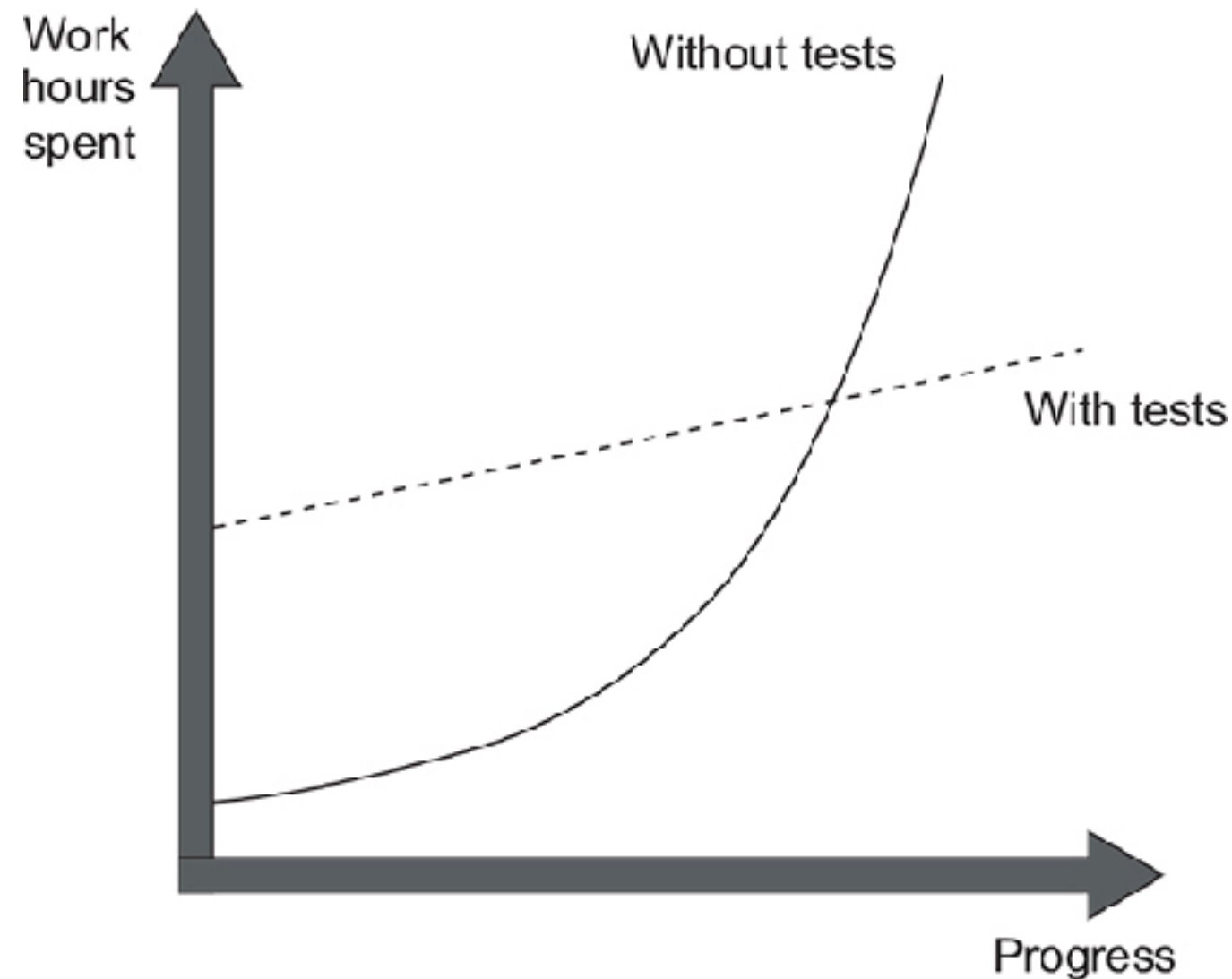


Regression testing

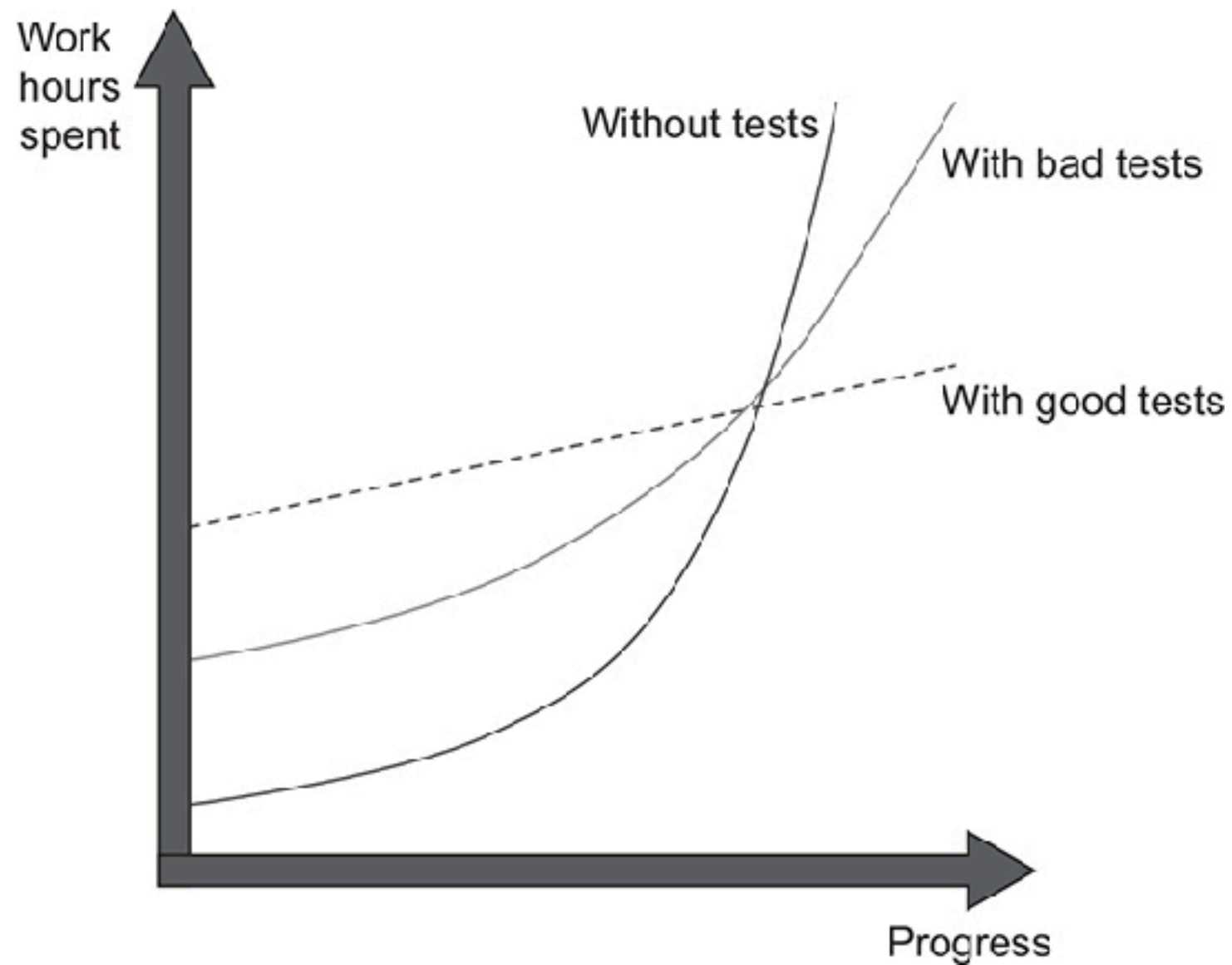
Complete
Selective or Partial



Reduce work hours with tests



Good vs Bad tests



Test data !!

Avoid database/external system access

Setup/ tear down test data

Use production-like data

Need to control your data test



Start with simple
Use feedback to improve



Workshop



Register workshop !!

As a new user,

**I want to create an account with
User name and strong password**

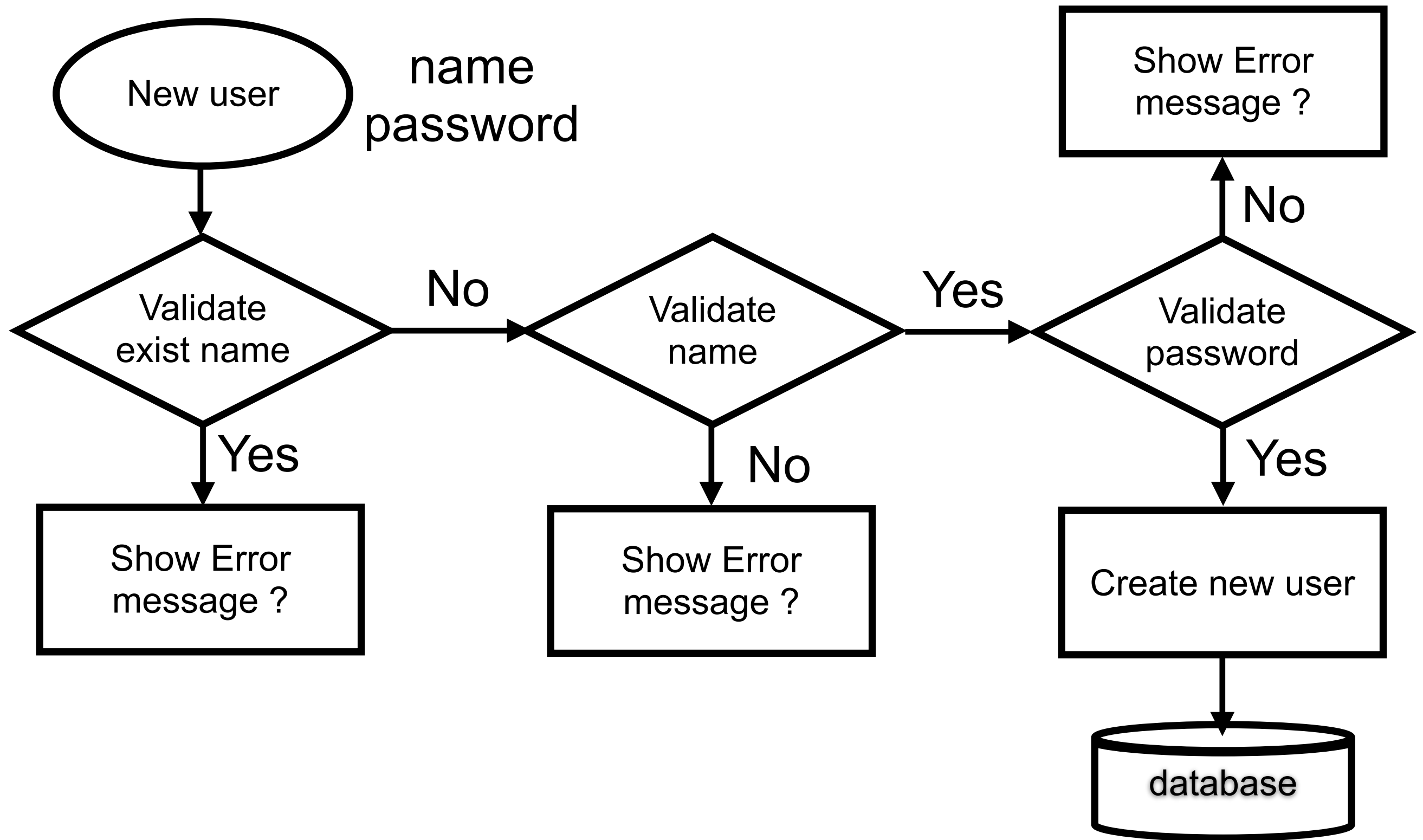
So that only I can access my information



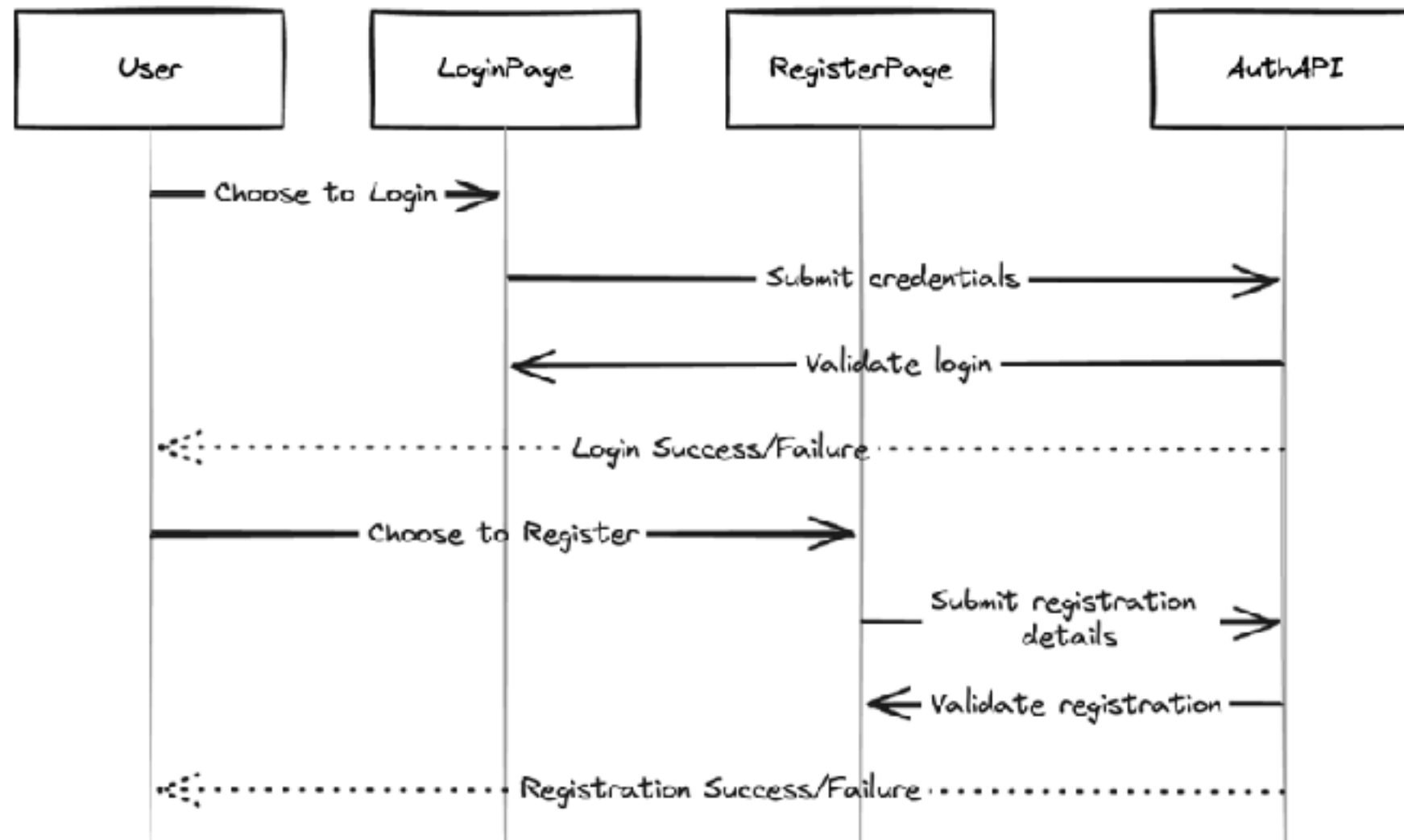
Design your Test cases ?



Flow ?



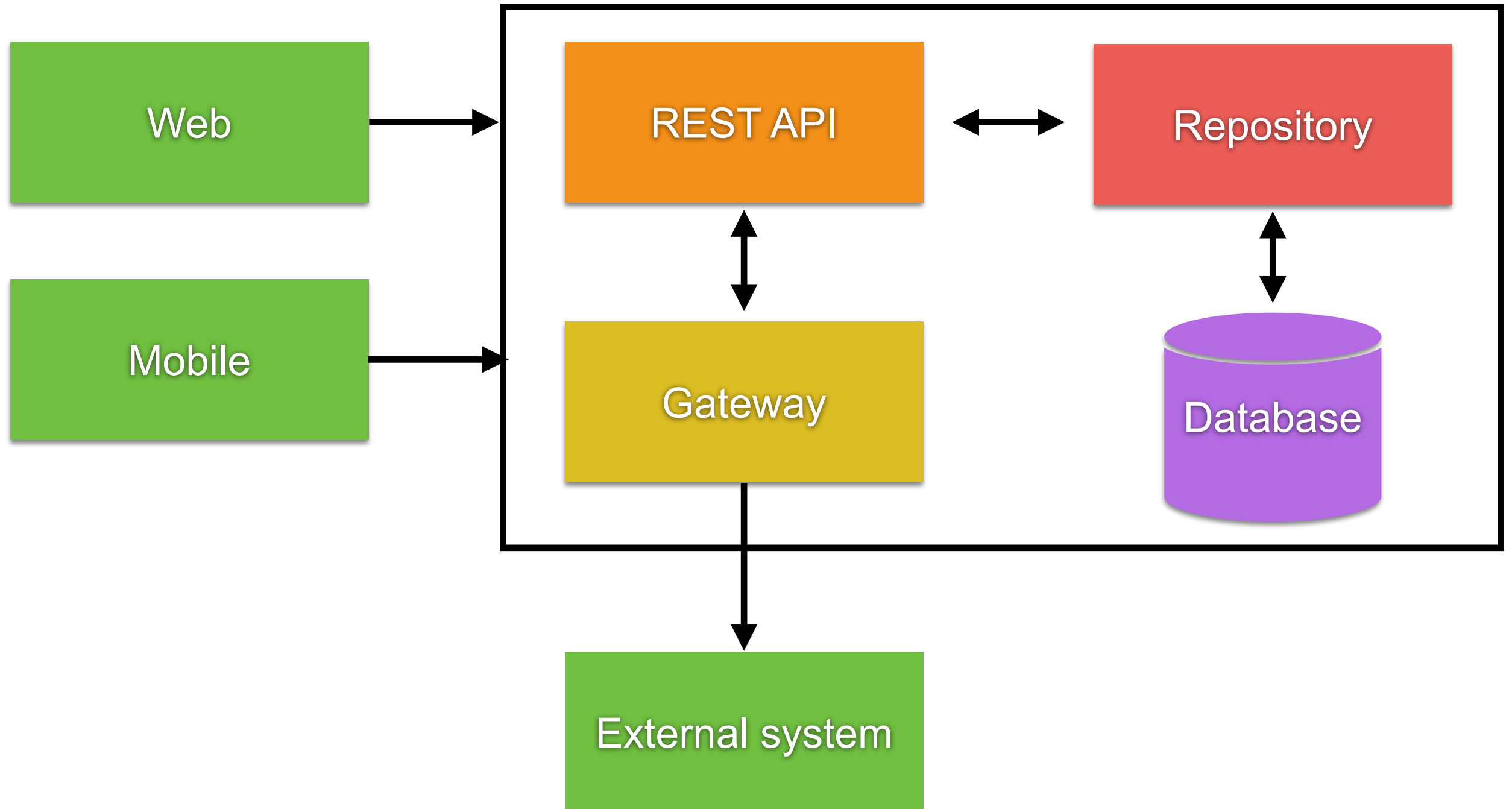
Generate Flow with AI



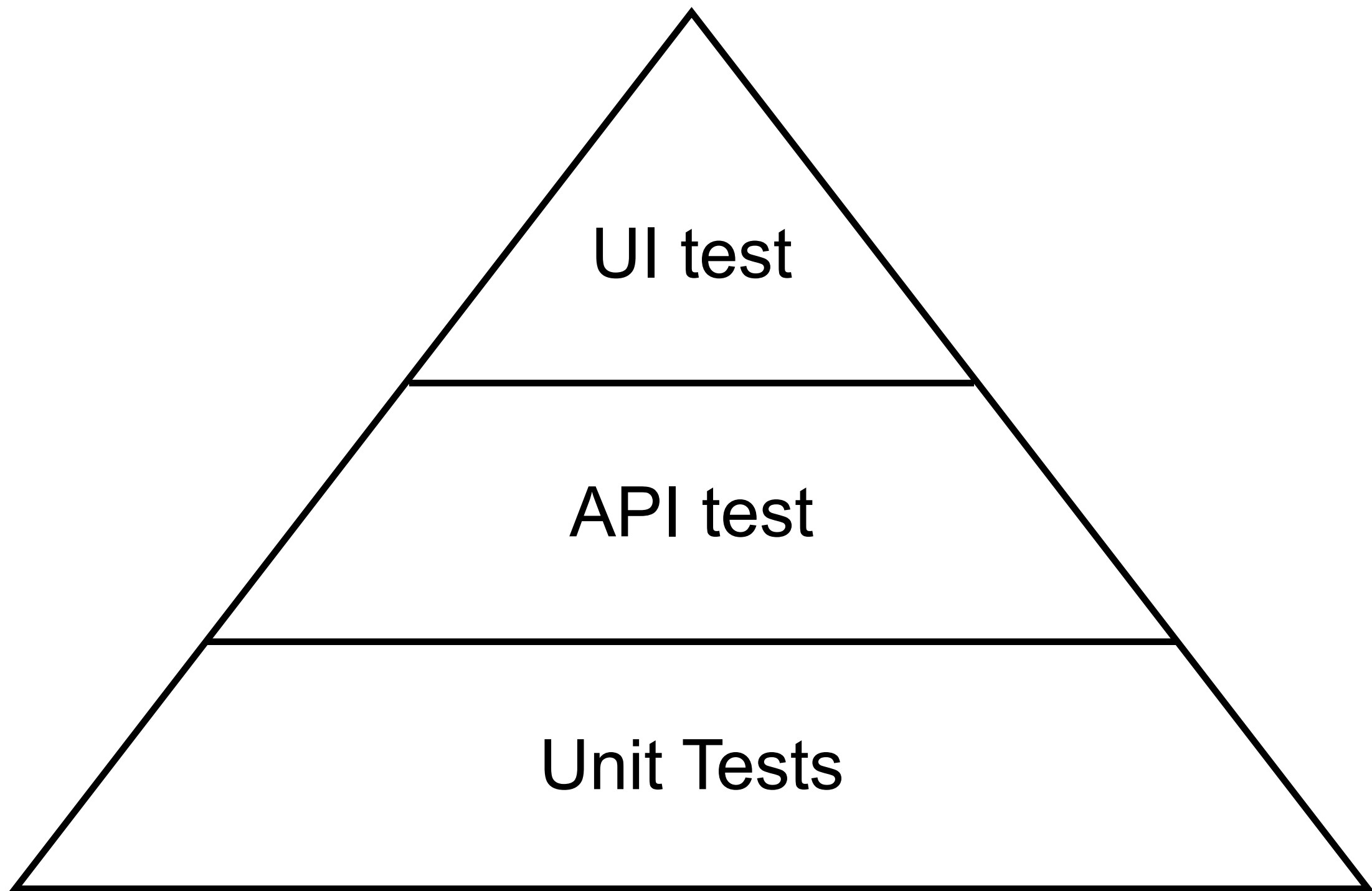
<https://excalidraw.com/>



Architecture



Test cases with Test Level ?



Testing Tools

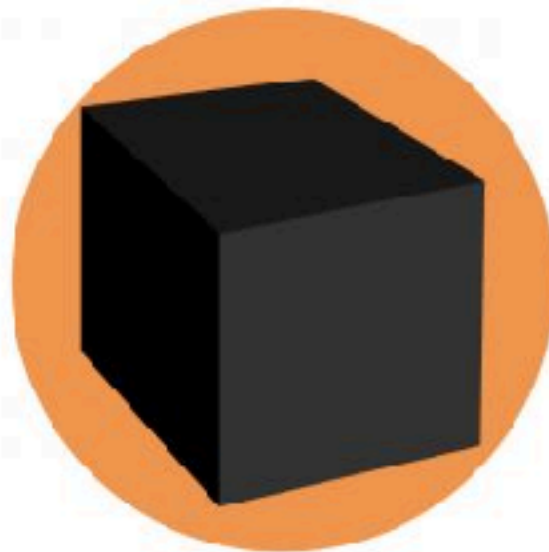
External
testing

Internal
testing



Types of Testing Methods

Black Box
Testing



(Unknown Internal
Code Structure)

+

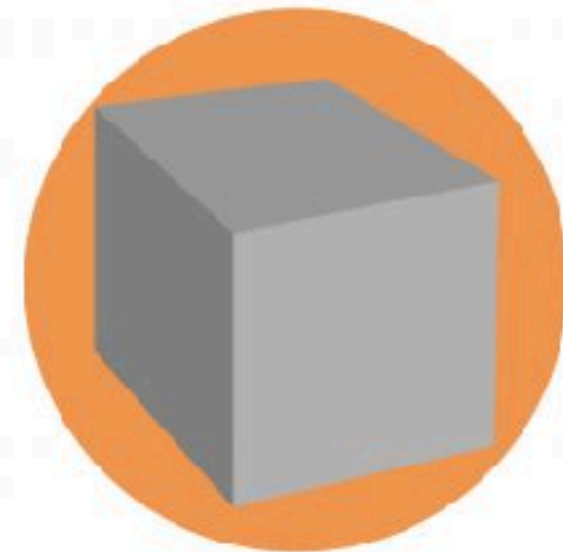
White Box
Testing



(Known Internal
Code Structure)

=

Grey Box
Testing

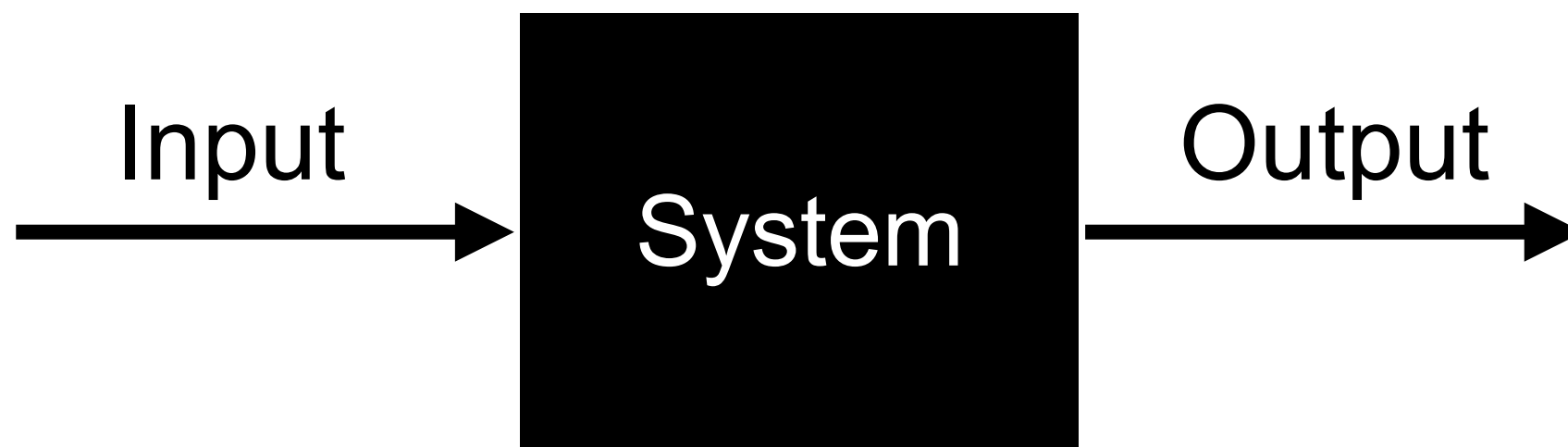


(Internal Code
Structure Partially
Known)



External Testing

Black box testing

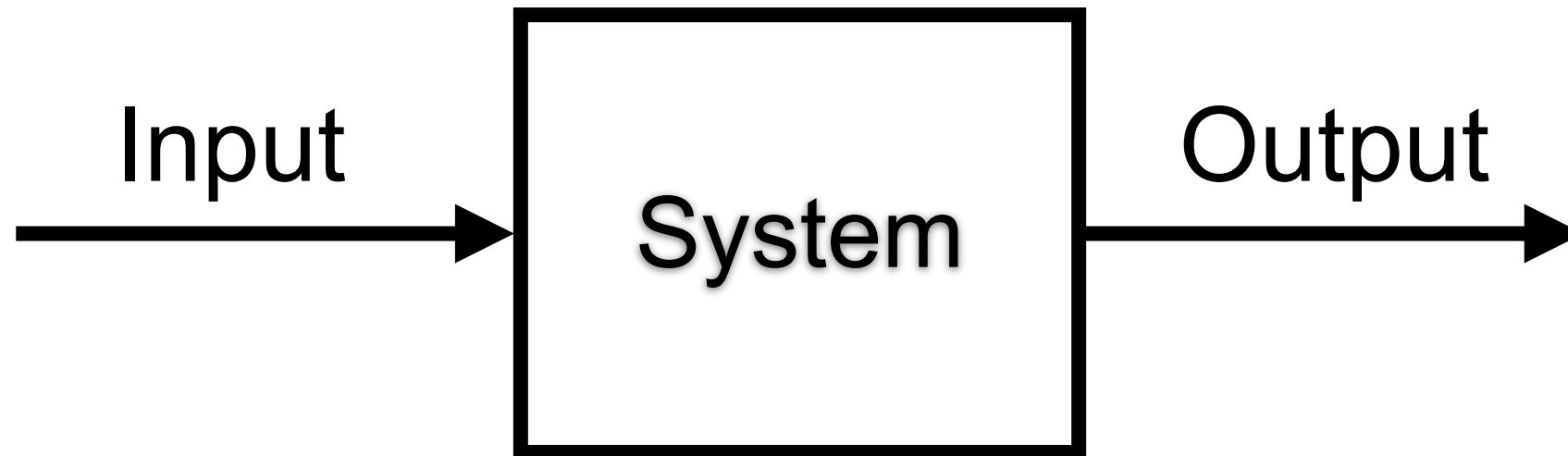


End-to-end testing



Internal Testing

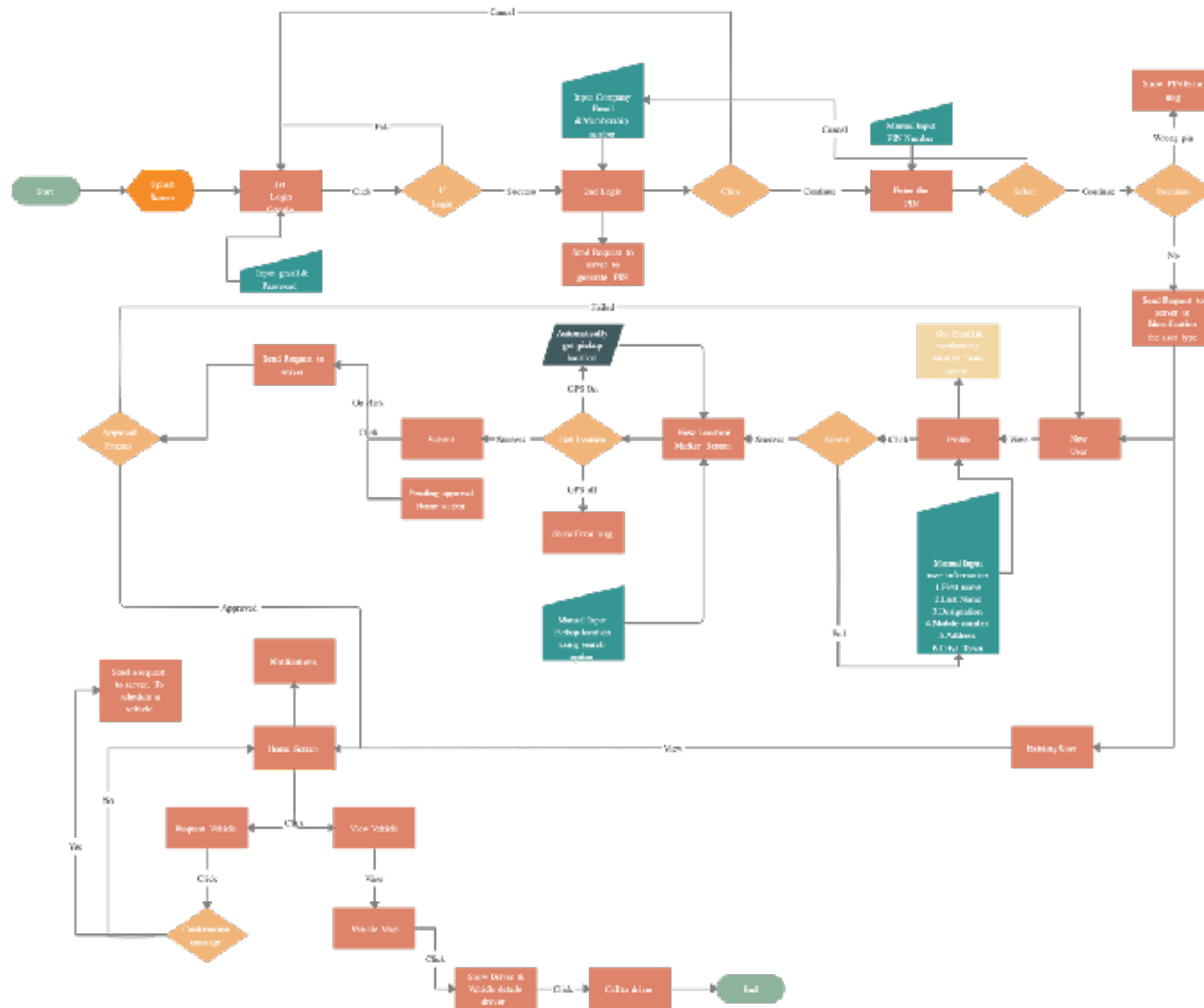
White box testing



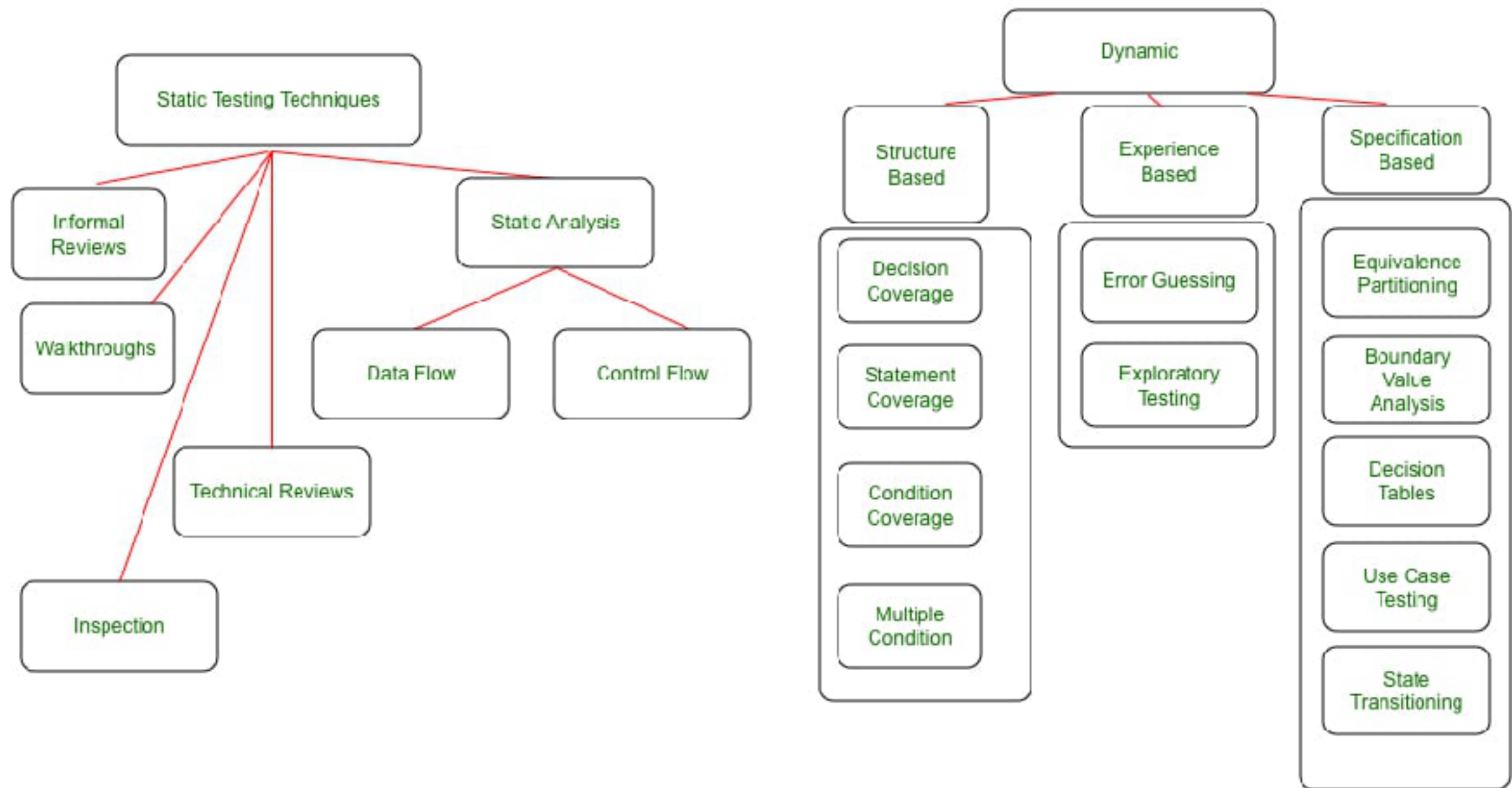
**Unit, Integration,
Contract, component testing**



Internal Testing



Testing Techniques



<https://www.geeksforgeeks.org/software-testing-techniques/>



Let's start your journey



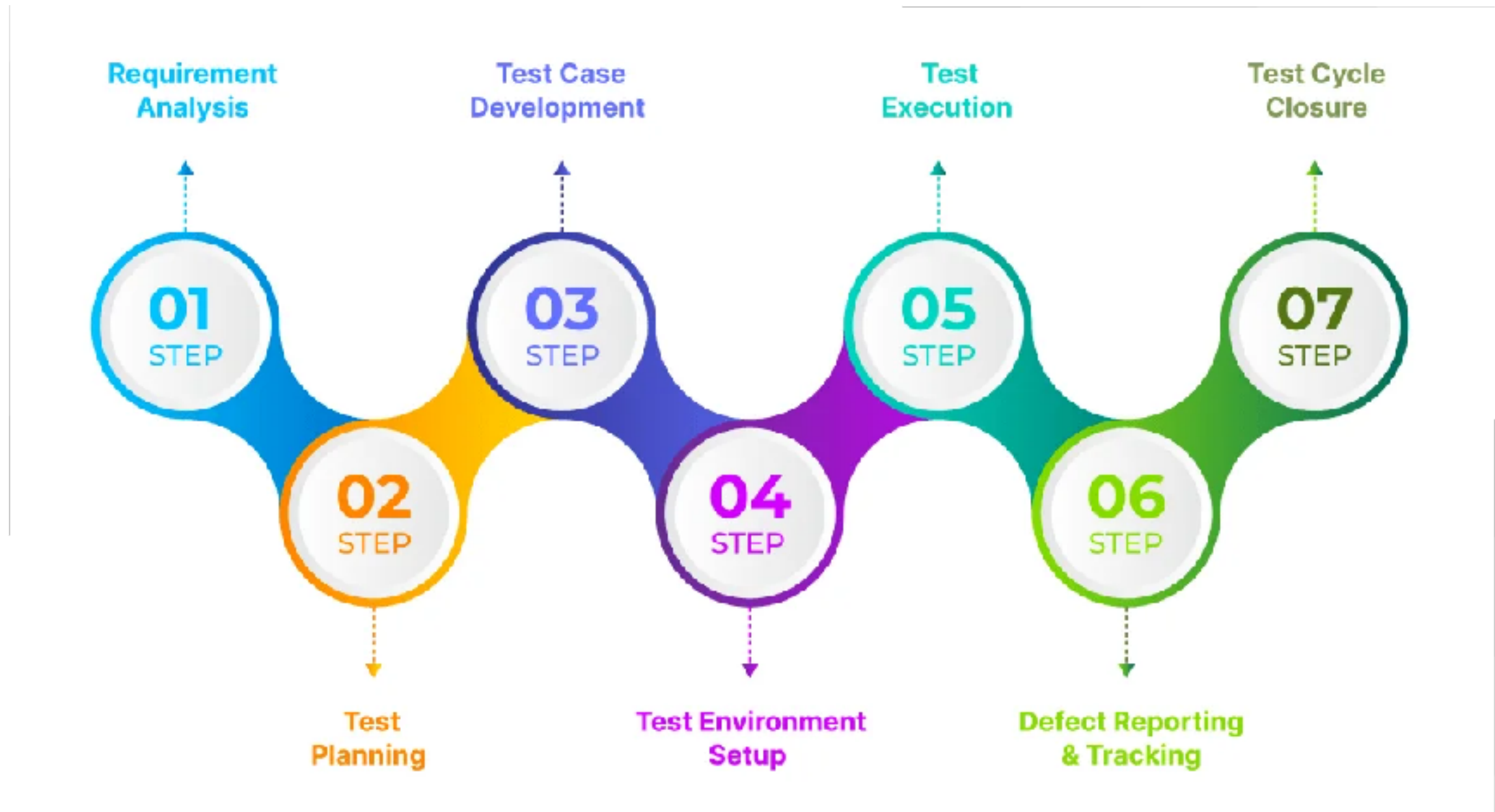
Q/A



Software Testing Life Cycle



Software Testing Life Cycle



<https://www.inexture.com/phases-of-software-testing-life-cycle/>



Software Testing Life Cycle

Improve quality

Reduce cost

Enhance efficiency (avoid rework)

Ensure comprehensive testing

Facilitate continuous improvement



Effective Test Case Design



Effective Test Case Design

Focus on clarify
Completeness
Maintainability

Requirement coverage and early detection



Approach ?

Understand requirements

Identify **test scenarios**

Apply **test techniques**

Prioritize test cases



Identify Test Scenarios ?

Break down features into specific, distinct behaviors or conditions

Think about all the different of **user's interact** with the feature

Happy
paths

Unhappy paths



Apply Test Techniques

Utilize test design techniques to ensure comprehensive coverage

Boundary Value
Analysis (BVA)

Equivalence
Partitioning (EP)

Decision table

State transition



Workshop

